

BEKTOP

The World's No.1 APL Journal

- APL*PLUS II Version 4.0 Reviewed 53
- Smith on Stone-walling in APL 64
- McIntyre on J's Boxed Arrays 92
- Piper on *DOIF* Considered Harmful 133

... plus

- Annual Report & Accounts 13
- Educational Supplement 23
- Statistics Library News 107



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.9 No.1 July 1992

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible. Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, or in workspaces from I-APL, APL*PLUS/PC, APL*PLUS II, IBM APL2/PC or Dyalog APL/386.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1992-93

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£20	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£100	10	5
Overseas Corporate	£155	10	
Sustaining	£430	50	5
Non-voting Student	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 04393-385.

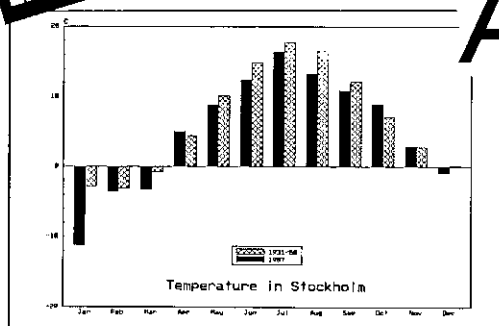
CONTENTS

	Page
Editorial:	
Maximising the Reuse of Code and OOA	Jonathan Barman 3
APL NEWS	
Quick Reference Diary	5
APL Course Diary	7
General Correspondence	9
British APL Association News:	
Committee News & Annual Accounts	13
News from Sustaining Members	Gill Smith 19
The Education Vector	Alan Mayer 23
REVIEWS SECTION	
APL Product Guide	Gill Smith 39
Six Months in the Life of VECTOR	John Searle 50
APL*PLUS II/386 Version 4	David Crossley 53
RECENT MEETINGS	
French Visitors in Napoleon's Footsteps	from the Obninsky News 62
Stone-walling in APL	Adrian Smith 64
GENERAL ARTICLES	
The] <i>HELP</i> Command	Ray Cannon 80
Making a Calendar in J	Keith Smillie 85
Using J's Boxed Arrays	Donald B. McIntyre 92
The Random Vector	David Eastwood 107
TECHNICAL SECTION	
Hackers Corner: Using DOS File Functions	Duncan Pearson 124
Technical Correspondence	Appleton & Thomson 126
• . ? (Random Jottings)	
<i>DOIF</i> Considered Harmful	Dave Piper 133
Index to Advertisers	143

EASY TO USE!

CYBEX

APL Graf



Tools for graphics presentation with the most common type of charts:

- Bar chart
- Line charts
- Point charts
- Added lines
- Histograms
- Pie charts

Well documented in a nice manual. Easy to use

- Simple APL-functions
- Always with defaults for everything
- Extensive parameter control of chart by global variables

Many special options. No runtime fee

APL-systems:

APL*PLUS/PC (CGI graphics)
£ 290

APL*PLUS II/PC (CGI graphics)
£ 390

CYBEX Utility functions

SAVES MONEY!

A general and practical set of utility functions for application programmers.

The special about this package is the machine-coded search routine for IBM:s APL2 mainframe. It is almost exactly like the ROWFIND in APL*PLUS II/PC. This means it performs IOTA for rows in character matrices. It is very efficient for ALL kind of arguments (compared to some other products that suffers from such weaknesses).

This routine alone saves £ 50 000 a year in one big APL-system at Volvo Car Corporation.

Can You afford NOT to use it?

Example: Δ IOTA Row search
 Δ PSUM Partial sums
 Δ UNS Uniques
 Δ BOX Matrix from vector
 Δ ON One object upon the other.

APL-systems:

APL2 mainframe (IBM)
from £ 1 900

APL*PLUS II/PC £ 130



CYBEX AB

Gruvgatan 35B, 421 30 V Frölunda, Sweden
Phone 46 31453740, Fax 46 31452423

Editorial: Maximising the Reuse of Code and OOA

by Jonathan Barman

Reusing code sounds easy, after all we all have masses of utility functions which we copy into a CLEAR WS before starting development of a new system. But getting the maximum reuse is quite difficult, and requires some clever design work. At the last meeting Adrian Smith explained the techniques that help him to get towards a target of only having to write 20% of the code that is required for a system. He uses layers of building bricks which can be assembled to create the final result.

The lowest level of building brick seems to be relatively easy to define, and the majority of code published in Vector falls into this category. Ensuring portability across different varieties of APL also helps to define functions. Each APL vendor has had his own ideas of how to solve a common problem, and one can build on the various design solutions that have been devised. In fact, it really is essential to have basic building blocks to mask the detailed differences in APL interpreters.

Adrian identified a middle range of functions where he uses established designs such as SQL and PostScript as the basis of writing higher level code which can be reused in many situations. Design is much more difficult at this level, and picking up existing ideas, which are familiar to a wide audience, seems a very sensible thing to do.

Identifying common areas which can share code at the top level of a system is the most difficult thing to do. When a big system is being implemented with a team of analysts and programmers, then the problems multiply. Identifying common processes amongst the members of the team needs a clear and simple way of communicating design ideas.

One of the current solutions that is becoming popular is Object Oriented Analysis (OOA). Everything seems to be Object Oriented nowadays, so I suppose that it was inevitable that the term was applied to an analysis technique. OOA seems quite similar to Entity Analysis, as it helps to describe the data requirements and the processes needed to manipulate the data. The general ideas have been about for a long time and the OO tag has been added quite recently.

OOA is not to be confused with Object Oriented Design, or Object Oriented Languages, or Object Oriented x where x can be any word other than Analysis. On the surface OOA seems to me to be intuitive and reasonably easy to use. The fiendish detail required to do a proper job is quite difficult and requires practice, as is so often the case with something that is conceptually simple.

There is no problem using OOA for analysis for APL systems, as it does not depend on any particular language, and you don't have to use Smalltalk or C++. I have seen several APL systems that could have benefited from a bit of formal design work before the developer plunged in and started hacking out code. Prototyping is all very well, but it can become an excuse for doing no design at all! Just carrying out some formal analysis seems to be a major benefit as it forces the clarification of exactly what is required. One of the advantages of OOA seems to be that the end of the analysis stage is well defined, so one does not have the temptation to carry on refining the design until it is ever more perfect.

OOA can go down to quite low levels, so that it would be possible to identify every APL function that is required for a system. However, I found the link between the final result and the implementation to be somewhat vague in the textbooks. This is possibly a price to be paid for a technique which is independent of the programming language to be used.

For those who have not already used analysis techniques, I would recommend that they look at OOA to see if it could help reduce the time taken to write systems. APL is fast, but with good designs, with a maximum reuse of code, it can go at GTi speeds!

Dates for Future Issues of VECTOR

	Vol.9 No.2	Vol.9 No.3	Vol.9 No.4
Copy date	1st Sept 92	1st Dec 92	1st March 93
Ad booking	12th Sept 92	8th Dec 92	12th March 93
Ad Copy	21st Sept 92	21st Dec 92	21st March 93
Distribution	October 92	January 93	April 93

Quick Reference Diary 1992

Date	Venue	Event
6 - 8 July	St Petersburg	APL92
25 September	IEE London	Vendor Forum
23 October	London	Field Trip to Morgan Stanley, at their new Canary Wharf offices at: 25 Cabot Street, E14 4QA
27 November	IEE London	Details to be Announced
August 1993	Toronto, Canada	APL93

British APL Association meetings will be held in the IEE, Savoy Place. Nearest tube outlets: Temple or Embankment. Monthly meetings are normally held on Fridays throughout the autumn and spring.

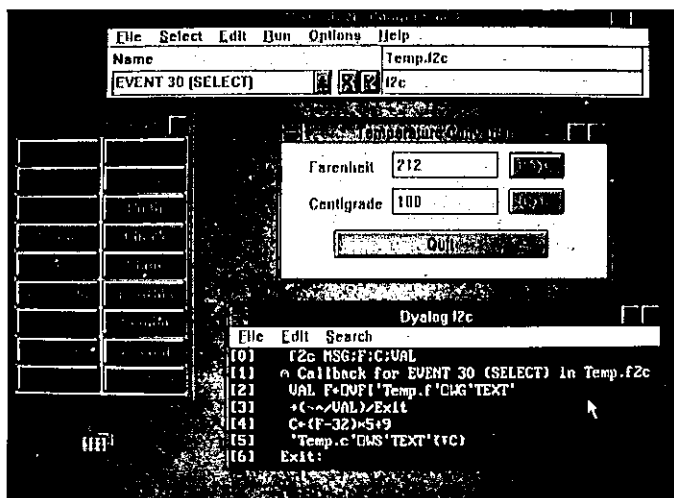
Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association
c/o Gill Smith
Brook House, Gilling East,
YORK YO6 4JJ**

Price £12 per vol (UK); £14 (overseas), £20 (airmail)
including Postage & Packing.

dyalog APL/W



APL for Windows™

Dyalog APL/W is a true 32-bit Windows application and it shows! Click on the APL icon and up pops a powerful multi-window APL development system, with a host of features designed to enhance your productivity and enjoyment. But that's just the start. APL system functions provide you with simple and elegant, yet powerful tools to drive the Windows Graphical User Interface (GUI). In one statement you can create a GUI object such as a PushButton, specify its appearance and behaviour, and attach a "callback" function to be run when the button is pressed. There's even a WDESIGN workspace to help you. Windows programming has never been easier - and your applications will run unchanged under OS/2 and on Unix workstations in future releases of Dyalog APL.

and it's friendly too

Dyalog APL/W supports Dynamic Data Exchange (DDE), using standard APL Shared Variables, so it can talk to any other Windows application that supports DDE. Can you imagine the power of APL combined seamlessly with your favourite databases, spreadsheets and other tools?

DYADIC

All trademarks acknowledged.

APL Training Courses for 1992

compiled by Gill Smith

For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	Cocking & Drury	3	June 16, Sept 8, Oct 13, Nov 17
Beginners	MicroAPL	1	July 23, Sept 17, Nov 12
Intermediate	MicroAPL	1	Jun 11, Aug 6, Sept 24, Nov 19
Advanced	MicroAPL	1	Jun 18, Aug 13 Oct 8, Nov 26
APL*PLUS/PC	MicroAPL	1	June 25, Aug 20, Oct 15, Dec 10
APL*PLUS II/PC Conversion	MicroAPL	1	July 9, Aug 27, Oct 22, Dec 17
Intro to APL2	MicroAPL	1	July 16, Sept 10, Oct 29
Statgraphics	Cocking & Drury	2	June 3, Sept 22, Nov 25

Vendors who wish to be included in the Training Diary, please contact Gill Smith at:

Brook House,
Gilling East,
YORK YO6 4JJ
Tel: 04393-385

APL, TEACH THYSELF

**At last, APL training is available in an effective
computer-based training course.**

At the press of a button.

F1

- This course is **NOT** just an APL textbook where the computer serves as an automated page flipper.
- And it is **NOT** just an interactive drill where you guess the results of APL expressions.
- It **IS** a highly interactive APL instructor with infinite patience **AND** with a sense of humor. Who says learning a programming language has to be dull?
- The Zark APL Tutor installs itself in seconds and remains your partner forever. Whenever you need a quick reminder or some words of APL inspiration, just give the F1 key a poke.
- And teach thyself.

Zark APL Tutor, \$299, APL*PLUS PC format. Includes one year free enrollment in the Zark APL Continuing Education (ACE) program.

Demo disk available for \$10.

From the same folks who brought you the popular textbook, APL ADVANCED TECHNIQUES AND UTILITIES

TO ORDER,
call (203) 872-7806 or write:
ZARK INCORPORATED

53 SHENIPSIT STREET
VERNON, CT 06066
U.S.A.

ZARK

Now available for IBM's
mainframe APL2 and for
STSC's APL*PLUS II.

General Correspondence

The Hungarian Naming Convention

From: Ray Cannon

March 1992

The Hungarian naming convention has been well documented elsewhere, but is basically a naming convention in which the object class or type is used as a prefix in its name. Thus lots of objects of the same class will all start with the same prefix. This can, depending on implementation, have one very big disadvantage in large APL systems: it can slow them down.

The reason for this is the way symbol tables are often designed to work. They use some hashing algorithm to convert the object's symbol into an address to start searching from. However, if lots of symbols produce the same address, as might happen if they all start with the same prefix, the number of symbol entries needing to be searched is much higher than if all the symbols produced very different addresses.

As an example of this type of behaviour, I have seen a system running extremely slowly due to the fact that every file in the file system started with the same letter. Renaming all the files with different letters speeded up the whole process.

So the idea I would like to put forward, is to use a class postfix rather than a class prefix. That way the names would not all start the same and the hashing algorithms would produce different addresses, hence shorter searches. This naming convention could then be called Reverse Hungarian Notation.

With reference to STSC's Double Whammy

From: David Piper

May 1992

Having just installed APL*Plus II Version 4.0, I must give credit to the Conservative party for introducing the perfect phrase to describe what STSC have done to us the poor punters out here in programming land. The last election will go down in (APL) history for that at least.

Evolution has come to programming land at last in the form of a new and deadly weapon)EVLEVEL. With the evolutionary paradigm of survival for those who least use nested arrays, I guess I shall be an early victim! In order to fit in with

the emerging stlBMard (like a standard but somewhat less universal), APL*Plus II is evolving to converge with APL2. For those of us not convinced that APL2 actually has it right (all those functions with axis) this seems like bad news. You better believe it!

As the manual states...

"...STSC is changing the definitions of features that conflict most seriously with APL2 so that the two systems no longer differ. This means that you MUST change existing APL*Plus II applications if they use features whose definitions are being modified."

So what are all the changes? Well, for example, APL*Plus II used the monadic up-arrow as *MIX* (similar to disclose in APL2) whereas APL2 used it for *FIRST*. Monadic epsilon was used as *TYPE* rather than *ENLIST*. You get the general idea.

To make life a little easier, evolution is proceeding slowly. At `)EVLEVEL 1`, all the primitives behave as they used to; though this release introduces some syntactic enhancements:

- strand assignments can be parenthesized
- selective assignment is available

So far so good; selective assignment was always on my wish list.

Evolution can even go backwards — at `)EVLEVEL 0`, any feature which is going to be changed after `)EVLEVEL 1` reports an *EVOLUTION ERROR* (if only Darwin had known about evolution errors!). We seem to have a real temporal twist here — evolution level 0 knows what is going to happen between evolution levels 1 and 2. Perhaps Oolon Colluphid could have used this rather than the Babel fish as the key of his argument in his epic philosophical tome "Well that about wraps it up for God".

Less seriously, perhaps a better approach would have been fuzzy evolution levels. At `)EVLEVEL 1.1` the old *MIX* function gives up the ghost and becomes an *EVOLUTION ERROR*, whilst all the other primitives remain intact. Quite what would happen at `)EVLEVEL 1.32742892` is another matter.

Finally at `)EVLEVEL 2` the behaviour of the language is changed. About half of the changes are syntactic in nature rather than changes to the semantics of functions. The point is you still have to make modifications to your code. Unfortunately not all of the new behaviour is being implemented at `)EVLEVEL 2`. For example, attempts to use *ENCLOSE-WITH-AXIS* in place of *SPLIT*

result in a *NONCE ERROR*, attempts to use partitioned enclose result in *EVOLUTION ERROR*.

Apparently, then, we have lost some functions at *)EVLEVEL 2*. Well not quite. In order to retain functional compatibility with *)EVLEVEL 1* a series of new system functions have been introduced (bringing the grand total to 160, assuming the new system help file is comprehensive). To ensure your code works under all evolution levels, all uses of the *MIX* primitive function must be replaced by *⊠MIX* etc. etc. etc. Helpfully, the new APL2 usages of some of these symbols are also introduced as system functions, for example *⊠ENLIST*.

At this point, I notice that the documentation omits all mention of the changed behaviour of *DISCLOSE* (used to behave like APL2's *FIRST*, now behaves like a sub-set of APL2's *DISCLOSE*). One more little trap for the unwary.

Here is my version of the table that appears on page 7-4 of the new User Manual, I have extended it to include *DISCLOSE*.

First, semantic changes:

Notation	<i>)EVLEVEL 1</i>	<i>)EVLEVEL 2</i>	System fns
$\uparrow w$	<i>Mix</i>	<i>First</i>	<i>⊠mix</i> , <i>⊠first</i>
ϵw	<i>Type</i>	<i>Enlist</i>	<i>⊠type</i> , <i>⊠enlist</i>
$\supset w$	<i>First</i>	<i>Disclose</i>	<i>⊠first</i>
$\uparrow w$	<i>Split</i>	<i>Syntax error</i>	<i>⊠split</i>
$\alpha \subset w$	<i>Partitioned enclose</i>	<i>Nonce error</i>	<i>⊠penclose</i>

Note that at *)EVLEVEL 2*, *DISCLOSE* is implemented as a subset of APL2's disclose function. The subset is compatible with the definition of *MIX* at *)EVLEVEL 1*.

Next, syntactic changes:

Notation	<i>)EVLEVEL 1</i>	<i>)EVLEVEL 2</i>	New notation
$(/)$	<i>Replicate each</i>	<i>Evolution error</i>	<i>(⊠repl)</i>
$(\backslash)$	<i>Expand each</i>	<i>Evolution error</i>	<i>(⊠expand)</i>
<i>A B C[2]</i>	<i>Vector index</i>	<i>Evolution error</i>	<i>(A B C)[2]</i>
<i>A B←X Y</i>	<i>Strand assign</i>	<i>Evolution error</i>	<i>(A B)←X Y</i>

Note that *⊠REPL* and *⊠EXPAND* are functions rather than operators. They cannot be used in place of reduction or scan.

It may seem strange to you that vector index and strand assignment both report *EVOLUTION ERRORS*. It would be much more appropriate for strand assignment to report a *SYNTAX ERROR*. Vector index really is an evolution error

since all that has been disallowed is indexing into strands of variables; it is still possible to use the current syntax to index into numeric strands. For example:

```
1 2 3[2]
```

still works at evolution level 2 (APL2 would report a *RANK ERROR*).

The manual states that a change to this behaviour is planned for a future release of APL*Plus II, so that the above expression will give the same error as APL2. Helpfully, we are even told how to avoid the problem by changing coding practices now.

Unfortunately, there is a window of "opportunity" when, by mistake or for some other reason, code can be written which will generate evolution errors in some future release! In other words, going through all the pain of migrating our code once is not enough for STSC, no, we have to go through the experience twice. This gap could have been avoided simply by reporting an evolution error for all strand index operations.

So what about the new system functions? Are we really safe using them? The manual assures us that they will be available for ever and a day, so systems using them will work at any evolution level. Do I believe them? Well, almost. I just have a sneaking feeling that at evolution level 4 (or maybe 5), shortly after we have all regained our confidence, STSC will announce complete compatibility with APL2. Access to sharefile and DOS files will now be via auxiliary processors; `▢ARBIN`, `▢POKE` and all your other favourite system functions will be replaced by associated functions; the interpreter name will undergo a subtle change to APL*Plus 2...

Another cause of real pain is that apart from the use of `)EVLEVEL 0` there are no tools to help with the migration process. At least IBM had the courtesy to provide `)MCPY` and a workspace full of (relatively) useful functions to help you migrate from VS APL to APL2. Not content to cause us grief, STSC want to rub salt into the mixed metaphor too.

Oh well, mine is not to reason why, I just want to be carried screaming into the future.

Dave Piper.
11 Lingfield Road
Eastwick Park
Evesham
Worcs WR11 6XG

British APL Association News Secretary's Report for 1991-92

by Anthony Camacho, Honorary Secretary, 19 May 1992

This is the Secretary's selection of the interesting items that were dealt with in your committee this year. Intentionally omitted are such things as arranging meetings (they are reported fully elsewhere), settling budgets and reviewing expenditure, booking rooms, maintaining contact with the British Computer Society (your Chairman and Treasurer both attend meetings) and a good deal of regular work such as answering queries, protecting our copyrights and other administration. Also omitted apart from a mention are initiatives which have scarcely begun such as the schools educational initiative or the publicity initiatives arising from a meeting held by the publicity officer.

There were Committee meetings on 7 June 91, 20 September 91, 22 November 91, 14 February 92 and 1 May 92. There will be another on the morning of 12 June 92.

1. APL Conferences

We sponsored a Russian to attend APL 91. We did some preliminary investigation, visited York University, discussed arrangements and contacted conference organisers and are planning to hold APL 94 at York. A nucleus of a committee has been formed under Adrian Smith as conference Chairman and Peter Donnelly who will organise the exhibition.

A proposal has been sent to SigAPL inviting them to collaborate by not asking for financial sponsorship, which will not be necessary if we obtain approval for our plans from the BCS as our funds with them will be adequate. We have helped with publicity for APL 92 and we hope our reports on APL conferences encourage the world to attend.

At APL 91 Vin Grannell paid for video recordings to be made of several sessions. When we heard he had done this we contributed half his costs in return for copies of the tapes and some have already been shown at our meetings.

2. Administration

The Association acquired access to email by paying for the Secretary to join CIX (the email address is acamacho@cix.compulink.co.uk). Many items for Vector have been solicited and received by this route.

The Administrative Assistant, Alison Chatterton, who has served us so well for some years, resigned and her place was taken by two people: Rowena Small who does the membership records and Gill Smith who does the Vector work. The Royal Mail freepost licence was transferred to Rowena's address.

The steps necessary to demonstrate that we comply with the Data Protection Act were taken and appropriate messages printed in Vector and on our membership forms.

The Association's policy on payment of expenses was discussed, clarified and recorded in writing. The Association's policy on membership records was changed. We now wish to hold a name and address for each APLer who is a member by virtue of working for a sustaining or corporate member. It is intended that the collection of the extra records will begin with the membership renewals for the year 1992/3.

3. Sponsorships

As well as sponsoring a Russian APLer at Stanford the Association has agreed to pay for the printing of the Russian edition of the I-APL Manual. We support the Regional Enterprise Grant Scheme proposal of Dr Ian Clark to write enhancements to the I-APL/Mac interpreter, and are considering a contribution towards sponsoring it.

The Association has continued to sponsor the APL Statistics Library with a conference at Gregynog, a presentation at APL 91 and the launch of the first part of the library at one of our meetings.

The outstanding achievement award is to be given to Paul Chapman in recognition of his achievement in writing the portable interpreter for I-APL. The presentation may be made at the AGM on 12 June 1992.

4. Publications

Vector is now fairly well established as the leading APL journal; it appears regularly and contains an excellent mix of articles from all over the world. For some time now it has been 144 pages or more instead of the 128 pages promised.

It is the major expense and main activity of the Association as well as the main benefit to members. The committee has actively supported the Vector working group and done its best to ease their task. APLers (members or not) who feel they might write for it are encouraged to try their hand and to contact the Editor or one of the working group.

The Committee is currently considering suggestions that we might launch a Beginners' Vector as part of Dr Mayer's schools initiative. Also proposed is a fully refereed journal of high academic standard that might be called The Transactions of the APL Association. The title is provisional as if launched it will attract papers from the whole world as there is no other wholly refereed publisher of APL papers. Perhaps the title could reflect this.

Summary of Annual Accounts

Summary of income and expenditure/receipts and payments:

	1991/92 (I&E) £	1991/92 (R&P) £	1990/91 (R&P) £
Income/Receipts			
Subscriptions	14678	14231	11787
BCS services	596	596	313
Bank interest	7583	7583	10652
Vector advertising (incl. VAT)	5939	5385	8331
Conferences and seminars	-	-	248
Other	372	515	782
	-----	-----	-----
Total receipts	29168	28309	32113
	-----	-----	-----
Expenditure/Payments			
Meetings	2679	2614	2223
Administration	3995	4378	4510
BCS services	596	596	313
Outstanding Achievement Award	500	-	500
APL Statistical Library	2155	2155	2738
Vector production and despatch	14549	16530	12308
Education Vector supplement	3434	4222	4444
Conferences and seminars	1128	1128	3358
Projects	4937	5160	839
Other	1512	1874	527
	-----	-----	-----
Total payments	35485	38658	31761
	-----	-----	-----

Assets summary:

Bank and other balances	70773	81121
Debtors	5104	2928
Creditors	(2135)	(1046)
	-----	-----
Net assets	73941	83003
	-----	-----

Notes

1. Pence figures have been omitted, so columns may not add exactly.
2. Debtors are mainly advertising and subscriptions.
3. Creditors are mainly advance subscriptions, unpaid bills, and VAT.
4. The value of stocks of Vector and the Software Library have not been assessed.

Treasurer's Remarks on the Annual Accounts.

In order to present a clearer picture of the Association's finances, the presentation of the accounts has been changed from that used in the past.

Instead of showing just receipts and payments, i.e. what goes in and out of our bank account, there is shown also an income and expenditure account for the year 1991/92, i.e. revenues strictly relating to the activities of that year.

The adjustments made to the receipts and payments figures to arrive at those for income and expenditure may be summarized as follows:

	£
Income attributable to 1990/91, excluded	3029
Expenditure attributable to 1990/91, excluded	5212
Debtors (1991/92), incorporated	3983
Creditors (1991/92), incorporated	2135

Most of the second of these items, expenditure attributable to 1990/91, relates to the cost of production and distribution of Vector 7.4, which was paid for in 1992, along with all four parts of volume 8.

This is a significant factor in the reduction of net assets, which had previously not allowed for the costs related to the fourth issue of the respective volume of Vector. Increased expenditure on projects, and a decline in bank interest account for most of the rest of the decline in net assets.

The increased expenditure relating to Vector can be attributed to three factors other than inflation: production costs were up partly because of an increased number of pages, partly because of the inclusion of colour advertisements (paid for by the advertiser), and distribution costs were up as a result of increased circulation.

BAA PUBLICITY IDEAS

If any of you have ever looked at the inside back cover of VECTOR you might have wondered who the Publicity Officer is and what he does. Well, last year when I joined the Committee in this very role, the same thoughts went through my mind. The way I see it, the Publicity Officer is someone who promotes APL by whatever means possible, to keep existing BAA members happy and to raise the profile of APL. As a result, I have put together a list of possible ideas for publicising APL and the BAA.

Improving Attendance at Meetings

There should be some incentive to come to meetings. There could be an APL brain teaser competition in VECTOR with the various solutions discussed and a prize given out at the next meeting. This type of activity would increase audience participation at meetings. The J workshop was a great success where everyone had a go at J on the machines provided.

Ideas for interesting topics to be presented at meetings are desperately needed. In order to attract people other than the few committed members, and those who have only vaguely heard about APL, it might be a good idea to host meetings with other language specialists, for example Mathematica or Smalltalk. Work would be needed to find out how to contact the potential participants.

The meetings are usually in London, but we need to analyse the membership list and work out other locations. Many people cannot make it to London, so we should go to them. I believe that we have had this type of meeting in the past, and that they have been well attended. The members of the committee that took part in the Publicity Meeting all agreed that this is a good idea and that we should approach people who live and work elsewhere to help us organise such an event.

Rather than going to the pub after the meeting, maybe we could hold a social occasion, such as wine tasting at or near the IEE. This way, members could bring their partners and friends along, or even bring a colleague along to the meeting as well!

Meetings should be more widely advertised so that we attract those who have only just heard about APL.

The corporate membership is being split into individuals, who will then get all news of meetings, etc. A lot of people working in companies probably do not know that they are members!

Some years ago we used to sell APL books at meetings, and this could be started up again.

Education

Do an APL presentation at a teachers' conference or similar. Find out which conferences would be most appropriate and get suitable APL experts to attend to try and spread the word. Education Vector could be used to show what we have done already.

Make sure that schools and colleges are kept informed of BAA activities, and have a permanent invitation to attend meetings.

Outreach

APL diary, general stationery, calendar with meeting agendas.

Communicate with APL groups all over the world and find out what they are doing. Co-ordinate with them to host joint events either in the UK, overseas, or produce joint articles through a medium such as VECTOR.

Past copies of VECTOR could be used as publicity material to send to people who might be interested in APL.

We could hold a 1 day event which is both social, has vendors and speakers. David Eastwood went to one such event in Copenhagen which was very successful. Having this on the weekend would mean that APLers from Europe could attend, people could bring their families. There could be a family event or Cricket match.

Help!

If you have any more ideas, then I would welcome a phone call or letter with your suggestions. Please call me, Misha Jovanovic, the Hon Publicity Officer. Address and telephone number is in the back of Vector.

News from Sustaining Members

Compiled by Gill Smith

HMW Trading Systems Limited

The most advanced foreign exchange dealing system in the world - now the most affordable. With over five years of evolutionary development to its credit, 4XTRA from HMW is now being launched as the leading edge technology capable of maximising profits and maintaining a competitive edge.

Developed as a front end deal capture and management system, 4XTRA provides the functionality required to monitor real time multi-currency positions, enter deals and calculate profit opportunities. With NPV profit and loss accounting as standard, the dealer is able to cover his positions accurately whilst providing his management with a real time picture of the overall risk.

4XTRA's comprehensive position keeping and deal entry screens enhanced to meet the demands of forward, spot and corporate trading together with facilities such as automatically generated spot cover deals, overnight tom-next switching and value added reporting give the trader the tools he needs to deal effectively.

And 4XTRA won't burn a hole in the IT budget. Cost is based on the number of users, and because the system is PC-based, you can forget about mainframe investment and maintenance costs - a simple PC network is all that's required. However, low cost doesn't mean low performance. Already installed in one of the world's busiest dealing rooms, 4XTRA easily manages over 4000 deals a day and can accommodate peak rates of over 60 deals a minute.

The next release of 4XTRA will use Dyalog APL-W, which will allow the system to run on other platforms (UNIX as well as DOS). A Windows version of Dyalog APL is being released later this year, with a full OS-2 version following on.

If you would like to hear more about how 4XTRA can help your FX business expand, please contact: Rob Thurston, Business Development Manager, HMW Trading Systems Limited, 1 Temple Avenue, London EC4Y 0HA. Tel: 071-353-4212; fax 071-353-3325.

MicroAPL Ltd

In the last couple of issues of Vector we have concentrated on our new relationship with IBM UK, focussing on recent product announcements for APL2 both on mainframe and PC systems. David Eastwood is progressively making contact with IBM's mainframe APL users and indeed if any readers have questions relating to IBM's product range they should contact David at our offices.

In this issue of Vector, we shall be reviewing some of the work our developers have been doing on our own APL product range — the APL.68000 family of products.

We have recently issued a minor upgrade for our Macintosh users to support the new Quadra range of Macintoshes. These are the latest 68040-based systems and are the fastest models in the Mac range. In common with other versions of APL.68000 we have had to make minor changes to the interpreter to support the data and instruction caching facilities on the Quadra. This upgrade is available at no charge for owners of APL.68000 Level I Version 7 or APL.68000 Level II Version 2.

Users of APL.68000/X — our version of APL.68000 for the RISC System/6000 — might be interested to note that we have implemented our 'MicroPLOT' business plotting language under APL.68000/X. MicroPLOT has been available for many years both under APL*PLUS/PC and for our multi-user versions of APL.68000. MicroPLOT is designed to simplify the production of routine business graphics and to present a device-independent syntax for APL programmers. We have versions of MicroPLOT available for HP plotters, Tektronix terminals and PostScript devices. Using MicroPLOT on the RISC System/6000 allows APL programmers to produce two dimensional business graphics without having to delve too deeply into the XLib facilities supplied on this system. The code is supplied in an unlocked form to permit customisation.

Our experiences with graphics on the RISC System/6000 coupled with numerous user enquiries have led us to recognise that there is a considerable demand for this sort of software on other windows-based platforms and we are in the midst of producing a Macintosh version of MicroPLOT.

APL.68000/X has been designed to make writing new applications which have a Graphical User Interface (windows, menu bars, dialogs, etc) as simple as possible. At the same time, because the IBM RS/6000 is so fast, APL.68000/X is being used to 'down-size' existing mainframe APL applications. Many of these older applications communicate with the user through AP124, an auxiliary processor performing screen layout and forms entry on IBM 3270 terminals.

MicroAPL have recently developed a windows-based AP124 for APL.68000/X. In APL-programming terms, the new processor is an exact replacement for AP124, so all the screen layout arrays and expressions like *CTL+3* to update the screen will work unmodified. However, the new AP124 is a true widget-based implementation using the Motif/X-Windows user interface.

An AP124 screen under APL.68000/X appears in its own window. It is implemented as a special case of a generalised Motif dialog, in which all the elements are either labels or text entry fields. Under Motif text entry fields will have a border, and our AP124 processor takes this into account when laying out the dialog. As well as using the normal AP124 keystrokes, the user can move around the dialog with the mouse and use it to select and edit text. Also, being a true Motif implementation, the dialog is customisable through the Motif resource mechanism, so you can change the colours and fonts used very easily. The result is that unmodified AP124-oriented programs take on a whole new lease of life and look like true windows applications.

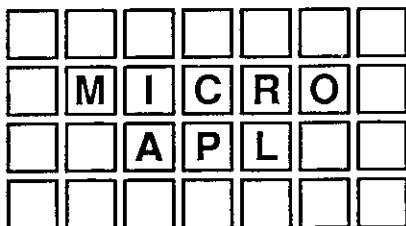
Some things are worth the wait

APL.68000 is not the only APL available for IBM's RISC System/6000. But no other APL running on the RISC System/6000 offers all these features:

- MicroAPL's acclaimed windowing interface
- Seamless integration with X-Windows
- Adherence to OSF/Motif style guidelines
- Very close compatibility with APL2/370
- Binary workspace compatibility across platforms
- High performance through MicroAPL's unique "Portable Assembler"™ technology

APL.68000 for the IBM RISC System/6000

*Fourth to announce.
First to get it right.*



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866
Fax: 071 928 1006



***General Application Consulting
Migration and Downsizing Expertise
Decades of APL Experience
Fortune 500 Clients***

Lingo Allegro U.S.A. Inc.
203 N. LaSalle St.
Suite 2100
Chicago, Illinois 60601
Phone +1 312 558 1342
Fax +1 312 346 9603

THE EDUCATION VECTOR

July 1992

Editor Alan Mayer

This Education Vector has been reprinted from VECTOR Vol.9 No.1. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Rd, Redland, BRISTOL, BS6 6LS Tel: 0272-730036.

Contents

Editorial	Alan Mayer	24
Two Simple Games	Bill McLean	25
Two Mathematical Functions	C.E. Williams	30
Workspace Listing in I-APL	Alain Delmotte	32

Dr. Alan Mayer
European Business Management School
University College of Swansea
Singleton Park
Swansea SA2 8PP Wales, UK

Ysgol Rheolaeth Busnes Ewropeaidd
Parc Singleton,
Abertawe SA2 8PP

Tel: 0792-205678 Ext.4274 Fax: 0792-295626 JANET: MAYER@UK.AC.SWAN.MS

Editorial

Welcome to the "APL92" edition of Education Vector. The British APL Association Journal Vector (of which Education Vector forms a small part) will be available at the APL 92 Conference at St. Petersburg in July. We wish our hosts well, in what may be regarded as the most truly international gathering the world APL community has ever experienced.

If you are not one of the lucky ones who will be attending APL92, you may like to know that you can get your own copy of Vector by becoming a member of the British APL Association. Write to Rowena Small, 8 Cardigan Road, London E3 5HU — the subscription is £12 per annum, for which you get four editions of Vector (144 pages each) and notice of meetings of the Association. Also, if you know of anyone who would like a (free) subscription to Education Vector, please send me his or her name and address.

My thanks to those who have responded to my request for functions. I have included a selection in this edition: some games from W.W. McLean, and some mathematical functions from C.E. Williams. Also thanks to Alain Delmotte, of the Food and Agriculture Organisation of the UN in Rome, for his programs to list the contents of I-APL workspaces. Together, the contributions make a good mixture of entertainment, application and utility — but if we are missing out something, write and tell me about it (preferably enclosing a contribution!).

I am currently assembling APL material suitable for use in schools. Later in the year I will be offering an APL interpreter and sample educational material, free of charge to any school that would like to try introducing it into their curriculum. Schools that have already tried APL have found it useful in a variety of areas, and I want to give as many young people as possible the opportunity of experiencing the power and flexibility of the language.

I look to readers of Education Vector (members and non-members alike) for help in this work. First, if you are involved in education and have used APL, tell me about your experiences, and if you have developed programs you are willing to share, let me see them. Second, bring this initiative to the attention of teachers who may like to take part — if they drop me a line, I will make sure they receive the material. Third, let me know what sort of material you think should be developed for this purpose. Fourth, if you are willing to spare some time and share your own expertise, this may be invaluable to schools in your area — sometimes the availability of a person with some experience of the language can save a great deal of time, particularly when the system is first introduced. If you are interested in helping in this way, please let me know.

A meeting will be arranged later in the year for anyone interested in discussing these ideas. Birmingham has been suggested as a possible venue, but I am open to suggestions of alternatives.

Two Simple Games

contributed by Bill McLean

In response to our request for functions, W.W. McLean (Dunoon, Argyll) contributes two games — the first is called *MAGIC* and involves manipulation of a 3 x 3 matrix, and the second is the well-known *OXO* or noughts-and-crosses (tic-tac-toe) game.

Magic

The instructions for this game are provided as an option in the function. A randomly selected matrix of 0s and 1s is presented, and the player is asked to select from a standard set of matrices (also 0s or 1s). The selected matrix is used to complement some of the values in the original matrix. The aim is to get a matrix of 1s with a lone 0 in the centre.

The instructions are printed after the function for convenience.

```

V MAGIC:A;GRID;K;L;LINE;LINE1;LINE4;M;MARGIN;NOS;WIN
[1]  □←GRID← 3 3 p ((?9p2)-1) o GRID← 3 3 p 0 o GRID[2;1]←1
[2]  WIN← 3 3 p 1 1 1 1 0 1 1 1 1
[3]  A← 1 1 0 1 1 0 0 0 0 1 1 1 0 0 0 0 0 0 1 1 0 1 1 0 0 0 1 0 0
[4]  A←A, 0 0 1 0 0 0 0 1 0 1 1 1 0 1 0 0 0 1 0 0 1 0 0 1 0 0 0 1 1 0
[5]  A←A, 1 1 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 1 1 0 1 1
[6]  NOS← 9 3 3 p A
[7]  'Would you like instructions? Y/N'
[8]  +('YN'=□)/INSTR,START
[9]  INSTR:
[10]  ''
[11]  ''
[12]  ''
[13]  ''
[14]  MARGIN← 3 6 p ' '
[15]  □←LINE← 1 27 p ' '
[16]  □←K←(NOS[1;:]),[2]MARGIN,(NOS[2;:]),[2]MARGIN,[2](NOS[3;:])
[17]  □←LINE
[18]  □←LINE1← 1 27 p ' 1          2          3 '
[19]  □←LINE
[20]  □←L←(NOS[4;:]),[2]MARGIN,(NOS[5;:]),[2]MARGIN,[2](NOS[6;:])
[21]  □←LINE
[22]  □←LINE4← 1 27 p ' 4          5          6 '
[23]  □←LINE
[24]  □←M←(NOS[7;:]),[2]MARGIN,(NOS[8;:]),[2]MARGIN,[2](NOS[9;:])
[25]  □←LINE
[26]  □←LINE4← 1 27 p ' 7          8          9 '
[27]  □←LINE
[28]  'By pressing any number (1 - 9), you can see which group of boxes,'
[29]  'marked 1's, will be complemented: 0 to 1, 1 to 0.'
[30]  'The object is to get a centre 0 surrounded by 1's'

```

```

[31]      □←WIN
[32]      ' '
[33]      □←GRID
[34] START: 'WHICH KEY? 1 - 9'
[35]      □←GRID+(NOS[□:;]*GRID)
[36]      +{^/, (WIN=GRID)}/END
[37]      →START
[38]      END: ' GOOD! A WIN!'
  7

```

Instructions

1 1 0	1 1 1	0 1 1
1 1 0	0 0 0	0 1 1
0 0 0	0 0 0	0 0 0

1 2 3

1 0 0	0 1 0	0 0 1
1 0 0	1 1 1	0 0 1
1 0 0	0 1 0	0 0 1

4 5 6

0 0 0	0 0 0	0 0 0
1 1 0	0 0 0	0 1 1
1 1 0	1 1 1	0 1 1

7 8 9

By pressing any number (1 - 9), you can see which group of boxes, marked 1's, will be complemented: 0 to 1, 1 to 0.
The object is to get a centre 0 surrounded by 1's

```

1 1 1
1 0 1
1 1 1

```

Example Game

```

0 0 0
1 0 0
0 0 0
WHICH KEY? 1 - 9
□:
  7

```

```

0 0 0
0 1 0
1 1 0
WHICH KEY? 1 - 9
□:
  1

```

```

1 1 0
1 0 0
1 1 0
WHICH KEY? 1 - 9

```

```

□:
      6

  1 1 1
  1 0 1
  1 1 1
GOOD! A WIN!

```

Any offers of an algorithm (in APL of course) to solve *MAGIC* problems?

OXO (Noughts and Crosses)

Mr McLean's second game is already designed to be played against the computer. The positions on the noughts-and-crosses board are labelled:

```

 1 2 3
 4 5 6
 7 8 9

```

The player (human) plays *X* and the computer plays *O*. You are offered the choice to play first or second, and the board is displayed after each move.

The variable *REFTABLE* contains all the possible winning positions:

```

 1 2 3
 4 5 6
 7 8 9
 1 4 7
 2 5 8
 3 6 9
 1 5 9
 3 5 7

```

This is used to index other arrays. The *BOTH* array fills up with 1s (for the human i.e. *X*) or 10s (for the computer i.e. *O*)

The algorithm used by the computer is simple. It first checks whether it can win (a row in $8\ 3\ \rho BOTH[REFTABLE]$ which adds up to 20). If a computer win is not possible, it checks for a human win (a row adding up to 2) and blocks if necessary. Otherwise the computer prefers to play the centre (if it gets the chance), a random empty corner, or just any square available.

The computer does not always win. Any offers of a better algorithm?

```

V GO;BOARD;BOTH;COMPUTER;HUMAN;KK;M;NEWBOARD;P;Q;RANDOMC;RC;REFTABLE;ROW;T;Z
[1]  SETUP
[2]  'WHO PLAYS FIRST? (H)uman or (C)omputer or (Q)uit?'
[3]  KK+□
[4]  →(KK='HCQ')/L1,L2,L3
[5]  L1:INPUT
[6]  L2:DEFENSE
[7]  →L1
[8]  L3:→
    V

V SETUP
[1]  BOARD←19
[2]  COMPUTER+HUMAN+BOTH+900
[3]  NEWBOARD←90'.'
[4]  REFTABLE←1 2 3 4 5 6 7 8 9 1 4 7 2 5 8 3 6 9 1 5 9 3 5 7
    V

V DEFENSE
[1]  'MY MOVE'
[2]  ''
[3]  A CHECK 20
[4]  T←+/Z+8 30BOTH[REFTABLE]  A Index into 1's and 10's. Add rows.
[5]  P←T÷20  A Any sum of 20: which row?
[6]  M←T÷20
[7]  →((V/M)=0)0L6
[8]  ROW←(8 30REFTABLE)[P;]
[9]  Q←(+/ROW)-+/ (8 30((COMPUTER+HUMAN)[REFTABLE]))[P;]
[10] →END
[11] L6: A CHECK2
[12] T←+/8 30BOTH[REFTABLE]
[13] P←T÷2
[14] M←T÷2
[15] →((V/M)=0)0L7
[16] ROW←(8 30REFTABLE)[P;]
[17] Q←(+/ROW)-+/ (8 30((COMPUTER+HUMAN)[REFTABLE]))[P;]
[18] →END
[19] L7: A CENTER
[20] →(NEWBOARD[5]≠'.'/)/L8
[21] Q←5
[22] →END
[23] L8: A ANY CORNER
[24] RANDOMC←4?4
[25] RC←(1 3 7 9)[RANDOMC]
[26] T←NEWBOARD[RC]ε'.'
[27] Q←1+T/RC
[28] →(Q=0)0L9
[29] →END
[30] L9: A Any square left
[31] Q←NEWBOARD1'.'
[32] END: A Assign to chosen position
[33] COMPUTER[Q]←Q
[34] BOTH[Q]←10
[35] NEWBOARD[Q]←'0'
[36] ''
[37] SBWC
    V

```

```

V INPUT
[1] 'YOUR MOVE!'
[2] Q←□
[3] →((Q>0)^(Q<10))ρL2      a Q is between 1 & 9
[4] 'TRY AGAIN'
[5] →1
[6] L2:
[7] →(BOTH[Q]=0)ρL3      a Square is not occupied
[8] 'TRY AGAIN'
[9] →1
[10] L3:BOTH[Q]←1
[11] HUMAN[Q]←Q
[12] NEWBOARD[Q]←'X'
[13] SHOWBOARD
[14] WINCHECK      V

```

```

V SBWC
[1] SHOWBOARD
[2] WINCHECK
V

```

```

V SHOWBOARD
[1] 3 3ρNEWBOARD
V

```

```

V WINCHECK;K;L
[1] K←(1-(,BOTH))/19
[2] L←(,REFTABLE)εK
[3] L←+/(8 3ρL)
[4] →(√/(L=3))ρL11
[5] L5:K←(10-(,BOTH))/19
[6] L←(,REFTABLE)εK
[7] L←+/(8 3ρL)
[8] →(√/(L=3))ρL13
[9] →(^(NEWBOARD≠'. '))ρL15
[10] →0
[11] L11:'YOU WIN'
[12] →
[13] L13:'I WIN'
[14] →
[15] L15:'A DRAW'
[16] →
V

```

Two Mathematical Functions

by C.E. Williams

C.E. Williams (Cold Keld, Loweswater, Cockermouth, Cumbria) writes: a couple of functions which may be of use to someone. The function *CHAR4*, to find the characteristic equation of a 4×4 matrix, uses Bocher's formulas which is a good illustration of APL's ease of programming. It is easy to adapt for any order of matrix.

```
[0] RES←CHAR4 A;B;C;D;A1;A2;A3;A4;S1;S2;S3;S4
[1] S1←+/1 1⊞A      ⍝ Sums major diagonal
[2] B←A+.×A        ⍝ Matrix multiplication
[3] S2←+/1 1⊞B      ⍝ Repeat
[4] C←B+.×A
[5] S3←+/1 1⊞C
[6] D←C+.×A
[7] S4←+/1 1⊞D
[8] A1←-S1
[9] A2←(S2+A1×S1)÷-2
[10] A3←(S3+(A2×S1)+(A1×S2))÷-3
[11] A4←(S4+(A3×S1)+(A2×S2)+(A1×S3))÷-4
[12] RES←1,A1,A2,A3,A4
```

To test the function:

```
A←4 4p1 -4 -1 -4 2 0 5 -4 -1 1 -2 3 -1 4 -1 6
A
1 -4 -1 -4
2 0 5 -4
-1 1 -2 3
-1 4 -1 6
CHAR4 A
1 -5 9 -7 2
```

i.e. the solution for this matrix is $x^4 - 5x^3 + 9x^2 - 7x + 2 = 0$.

Romberg's Integration

The second function is a version of Romberg's method for numerical integration. It is slower than Simpson's Rule. I hope someone will improve it — it is a start anyway!

```
[0]  ROMB X;A;B;C;J;M;L;S;T;I;I2;I3;A1;B1;W;N;K;IO
[1]  Romberg integration, function in ΔFN
[2]  A is vector of lower, upper limits and N
[3]  IO←0 ◊ A←X[0] ◊ B←X[1] ◊ N←X[2]
[4]  A1←ΔFN A
[5]  B1←ΔFN B
[6]  I←20p0
[7]  C←B-A ◊ L←1 ◊ M←K←0
[8]  J←(A1+B1)÷2
[9]  I[0]←C×J
[10] L1:L←2×L ◊ M←M+1 ◊ W←1
[11] L3:J←J+ΔFN A+W×C÷L
[12] →{(L-1)≥W+W+2)/L3
[13] I2←J×C÷L ◊ S←0 ◊ T←4
[14] L2: I←I3+((I2×T)-I[S])÷T-1
[15] →(N=K+K+1)/PRINT
[16] I[S]←I2 ◊ I2←I3 ◊ S←S+1 ◊ T←T×4
[17] →(S<M)/L2
[18] I[S]←I2
[19] →L1
[20] PRINT: 'Integral is ', I3
```

Running the function, to integrate the square root of $x^2 + 3x - 4$, with $N=10$, (the exact value is 1.577186) gives:

```
ROMB 1 2 10
1.513789887
1.554609508
1.557330816
1.569181806
1.570153293
1.570356824
1.574352706
1.574697433
1.574769562
1.574786867
Integral is 1.574786867
```

Repeating the process with $N=50$ and $N=100$, gives 1.577181348 and 1.577185956 respectively (but be warned - it takes a long time!)

Workspace Listing in I-APL

by Alain Delmotte, Agrometeorology/FAO, Rome.

It is interesting once in a while to see the content of a workspace: the lists of variables and functions and their size and content.

)*VARS*,)*FNS*, □*WA*, □*LX* and the Del editor (▽) or □*CR* will provide the information but there is no record. It is also handy to have a file with the printed content of the workspaces, for example to be able to copy a function into another workspace.

We will create a *LIST* utility workspace which can be copied into a working workspace, used to print the content and then be discarded.

As with all workspaces, but especially here with the utility workspace, great care should be taken to avoid conflict of names.

The names of the variables, functions and local variables will be systematically prefixed with Δ or ΔΔ. Recall that Δ, and _ can be used in names (N.B. Δ can even be used alone in I-APL — I often use Δ alone for the *WHERE* function, Δ:α, creating a kind of diamond separator and providing shorter lines which fit in the 79 characters length of line of I-APL). Another way would be to use lower-case letters; they can be typed in using the <INS> key: first press <INS> (this switches the keyboard to lower-case), input your text and press <INS> again to return to the normal keyboard.

The use of Δ for the names allows the creation of normally unused names to avoid conflict with the names of the objects of the workspace to be listed. In particular, if objects in the workspace are called *A*, *B*, *C*, ... and if we use these names as local variables in our listing functions, the content of the objects will not be printed but instead the present content of the local variables. The same conflict can arise with the local variables of functions and subfunctions.

A main function, Δ*GO*, will initialize some local variables (Δ*CR* carriage return, Δ*LF* line feed and Δ*FF* form feed) and will call a function to print headings to the report (ΔΔ*HEADINGS*) and a function managing the printout of the content (ΔΔ*WSPACE*). The argument of Δ*GO* is the name of the workspace; other information is passed in global variables:

ΔWA	$\leftarrow \square WA$	<i>a free space</i>
ΔV	$\leftarrow \square NL\ 2$	<i>a variables list</i>
ΔF	$\leftarrow \square NL\ 3$	<i>a functions list</i>

It should be noted that ΔV will contain ΔWA , so we will take off ΔWA from ΔV in ΔGO .

$\Delta\Delta HEADINGS$ prints the title of the workspace which is supplied to ΔGO and passed as argument to $\Delta\Delta HEADINGS$. It also prints the content of the Latent Expression ($\square LX$) and the space used by the workspace. This last value is computed as the difference between the available space of an empty workspace (on my PC 31,644) and the $\square WA$ of the current workspace supplied as the global variable ΔWA .

$\Delta\Delta WSPACE$ first prints the lists of the variables and functions through the $\Delta\Delta VARSFNS$ function and then prints the content of the variables and functions with the $\Delta\Delta LIST$ function.

The $\Delta\Delta VARSFNS$ function lists the names of the variables and functions, six names per line. This restricts the length of the names to 15 characters (for a line of 80 characters). In practice the names are limited to 12 characters which is quite sufficient; only 2 or 3 names were of that length in the workspaces distributed with I-APL (and were placed at the end of the line).

The argument ω ($\omega \leftarrow \square NL\ 2$ or 3) is first padded with blanks by the primitive Take (in this case it overtakes) and then raveled:

```

 $\Delta A \leftarrow 1 \uparrow \rho \omega$ 
 $\omega \leftarrow ,(\Delta A, 12) \uparrow \omega$ 

```

60 blanks are added in case the last line does not have six names and the result reshaped to lines of 72 characters and printed.

```

 $\omega \leftarrow \omega , 60 \rho 1$ 
 $\Delta B \leftarrow \lceil \Delta A \div 6$  a round up for incomplete line
 $\Delta B \leftarrow \Delta B, 72$ 
 $\omega \leftarrow \Delta B \rho \omega$ 

```

ΔV and ΔF provided to $\Delta\Delta LIST$ the lists required as right argument ω ; α , the left argument of $\Delta\Delta LIST$, will specify if ω contains variable names ($\alpha = 'V'$) or function names ($\alpha = 'F'$). We first check if ω is empty, in which case we do nothing with a branch to 0.

```

 $\rightarrow (0 \in \rho \omega) / 0$ 

```

Then in a DO ... WHILE loop we take the top name of the list, print the content of the object and erase the name; we repeat the loop while the list is not empty.

```
NXT:
  ....
  ....
  ω ← 1 0 + ω
  → (~ 0 ∈ ρ ω) / NXT
```

The way to print the content of variables being different from the one for the functions we call a specific function using an IF construct, for example:

```
± (α = 'V') / 'ΔΔVARIA, ω [□IO; ]'
```

We take the first line: $\omega [\square IO;]$ whichever is $\square IO$ and ravel it to a vector.

$\Delta\Delta VARIA$ prints the name of the variable (and its shape as an extra) followed by its content. To find the shape and the content we have to execute ω , argument of $\Delta\Delta VARIA$, containing the name of the variable.

```
ΔΔPRINT '      ', ω, '      (dim: ', (▼ ρ ± ω ), ' )'
```

$\Delta\Delta FUNCT$ only prints $\square CR \ \omega$. We will see later some cosmetics for the result.

The printing function $\Delta\Delta PRINT$ has to handle the printing of a matrix, a vector or a scalar. ω is first formatted as characters:

```
ω ← ▼ ω
```

and then reshaped as a matrix with each line suffixed by carriage return and line feed (this could be printer specific) and printed with $\square TX$:

```
ΔA ← - 2 + 1 1, ρ ω      # reshapes to a matrix
ω ← ΔA ρ ω              # of minimum 1 1 shape
ω ← ω , ΔCR             # appends carriage return
ω ← ω , ΔLF             # appends line feed
```

Cosmetics

$\square CR$ returns a character matrix of the list of a function but the lines are not numbered. In other systems, the $\square VR$ function returns a nicely formatted list of the function:

```

      □VR 'VARIA'
      ▽ ΔΔVARIA
[1]  A Prints the content of the variable of name
[2]  ΔΔPRINT '      ',ω,'      (dim: ',(▽ρ±ω),')'
[3]  ΔΔPRINT ±ω
      ▽

```

We can add this extra touch to our function without difficulty. First we create a character variable with the initial ▽ and the line numbers; we will limit ourself to 30 lines.

```

ΔΔNUM ← ▽ 30 1 ρ 130      A 30x3 character matrix of
                        A the numbers 1 to 30
ΔΔNUM ← ΔΔNUM,']'        A adds closing bracket
ΔΔNUM ← 1 ϕ ΔΔNUM        A suppresses the leading
ΔΔNUM[19:] ← 1ϕΔΔNUM[19:] A blanks
ΔΔNUM ← '[' ,ΔΔNUM        A adds the opening bracket
ΔΔNUM ← (1 5 ρ '      ▽'),[1]ΔΔNUM A adds the line 0 ▽ symbol

ΔΔNUM
  ▽
[1]
[2]
[3]
..... numbers from 4 to 27
[28]
[29]
[30]

```

Note that ΔΔNUM could be created 'on the spot' in ΔΔFUNCT, but this would slow down the system. We use ΔΔNUM in ΔΔFUNCT[3] to add the numbers to □CR if the function is not a direct definition function. After printing the numbered □CR, we add a closing ' ▽'.

After using the LIST workspace, its functions and variables should be removed. This is easily achieved with the Expunge system function □EX. Before saving the LIST workspace we create a variable containing the lists of names:

```

ΔV←''      A dummy variables for ΔV, ΔF and
ΔF←''      A ΔWA which should also
ΔWA←''     A be removed
ΔΔVARL ← □NL 2 3
ΔΔVARL

ΔF
ΔGO
ΔV
ΔWA
ΔΔFUNCT

```

```

 $\Delta\Delta$ HEADINGS
 $\Delta\Delta$ LIST
 $\Delta\Delta$ NUM
 $\Delta\Delta$ PRINT
 $\Delta\Delta$ VARIA
 $\Delta\Delta$ VARL
 $\Delta\Delta$ VARSFNS
 $\Delta\Delta$ WSPACE
    )ERASE  $\Delta$ V  $\Delta$ F  $\Delta$ WA
    )LX+ 'This is the LIST workspace'      * Latent expression
    )SAVE
LIST

```

Using LIST

Before copying *LIST* into the current workspace we create the global variables with the free space and the lists of variables and functions:

```

 $\Delta$ WA ←  $\square$ WA           * before anything
 $\Delta$ V ←  $\square$ NL 2         * before  $\Delta$ F, will contain  $\Delta$ WA which
                        * is removed in  $\Delta$ GO

 $\Delta$ F ←  $\square$ NL 3
)COPY LIST
 $\Delta$ GO 'the name'
    .....
    )EX  $\Delta\Delta$ VARL       * it works!?
                        * expunge the LIST objects
1 1 1 1 1 1 1 1 1 1

```

Limitations

If the name of a variable or function is longer than 12 characters it will be truncated in the list of names, but not later; if it is 12 characters, it will stick to the next one (unless it is the last name of the line). We have limited the length of functions to 30 lines; this can be adjusted. But who writes functions longer than a screen? If the current workspace is big, there could be a *WORKSPACE FULL* message. The solution (tedious) is to split it in pieces, for the listing only. To avoid complexity and to reduce the size of *LIST* there is no provision for the page jump; some lists will be cut. To reduce the size of *LIST*, all the comments could be removed from the working version. Some variables are impossible to print. This is the case with the *FONT* variable of the *EPSON* workspace; it contains control characters for the printer. I replaced it by:

```

FONT ← 'This is a variable containing the font code'

```

P.S. A similar function could be created to print in a file instead of on the printer; it should incorporate some of the functions of *DFILE*, including the opening and closing of the file and *ΔΔPRINT* should be adjusted.

LIST on LIST

```

LIST
=====
□LX ← 'This is the LIST workspace'
Space used: 8704

*****Variables*****
ΔΔNUM      ΔΔVARL
*****Functions*****
ΔGO        ΔΔFUNCT      ΔΔHEADINGS  ΔΔLIST      ΔΔPRINT      ΔΔTITLE  ΔΔVARIA
ΔΔVARSFNS  ΔΔWSPACE

      ΔΔNUM      (dim: 31 5)
      ▽
[1]
[2]
[3] ...
[28]
[29]
[30]
      ΔΔVARL      (dim: 11 10)
ΔF
ΔGO ....
ΔΔVARSFNS
ΔΔWSPACE

      ▽ ΔGO ω;ΔCR;ΔLF;ΔFF;ΔA
[1]  A Main function to list the content of a workspace
[2]  ΔCR←□AV[13+□IO] A Carriage return
[3]  ΔLF←□AV[10+□IO] A Line feed
[4]  ΔFF←□AV[12+□IO] A Form feed
[5]  A Taking off ΔWA from ΔV
[6]  ΔA←~1+ρΔV
[7]  ΔV←(ΔV-.#ΔA†'ΔWA')/ΔV
[8]  ΔΔHEADINGS
[9]  ΔV ΔΔWSPACE ΔF
      ▽

      ▽ ΔΔFUNCT ω;ΔA
[1]  A Prints the □CR of the function in a mode similar to □VR
[2]  ΔA←□CR ω
[3]  Δ(2=ρρΔA)/'ΔA+(((1+ρΔA).5)†ΔΔNUM),ΔA'
[4]  ΔΔPRINT ΔA
[5]  Δ(2=ρρΔA)/'ΔΔPRINT ' '      ' ' '
      ▽

      ▽ ΔΔHEADINGS ω
[1]  A Prints headings for the workspace
[2]  ΔΔPRINT ω

```

```

[3]  ΔΔPRINT (ρω)/'='
[4]  ΔΔPRINT ''
[5]  ΔΔPRINT '□LX ← ',□LX, ''
[6]  ΔΔPRINT ''
[7]  ΔΔPRINT 'Space used: ',(▼31644-ΔWA)
[8]  ΔΔPRINT '      ▼      ▼ α ΔΔLIST ω
[1]  α Successively lists the variables and functions
[2]  →(0<ρω)/0
[3]  NXT:
[4]  z(α='V')/'ΔΔVARIA,ω[□IO;]'
[5]  z(α='F')/'ΔΔFUNCT,ω[□IO;]'
[6]  ΔΔPRINT ''
[7]  ω+1 0+ω
[8]  →(~0<ρω)/NXT
▼

▼ ΔΔPRINT ω;ΔA;ΔB
[1]  α Prints the matrix, vector or scalar
[2]  ω←▼ω
[3]  ΔA←~2+1 1,ρω
[4]  ω←ΔAρω
[5]  ω←,ΔCR ω
[6]  ω←ω,ΔLF
[7]  2 □TX ω
▼

▼ ΔΔVARIA ω
[1]  α Prints the content of the variable of name
[2]  ΔΔPRINT '      ',ω,'      (dim: ',(▼ρzω),')'
[3]  ΔΔPRINT zω
▼

▼ ΔΔVARSFNS ω;ΔA;ΔB
[1]  α Lists the names of variables and functions
[2]  α formatting in lines of six names of maximum 12 characters
[3]  ΔA←1+ρω
[4]  ω←,(ΔA,12)†ω
[5]  ω←ω,60ρ ' '
[6]  ΔB←[ΔA÷6
[7]  ΔB+ΔB,72
[8]  ω←ΔBρω
[9]  ΔΔPRINT ω
▼

▼ α ΔΔWSpace ω
[1]  α Prints the content of the workspace
[2]  α first the lists of variables and functions, then their content
[3]  ΔΔPRINT '*****Variables*****'
[4]  ΔΔVARSFNS α
[5]  ΔΔPRINT ''
[6]  ΔΔPRINT '*****Functions*****'
[7]  ΔΔVARSFNS ω
[8]  ΔΔPRINT ΔLF
[9]  'V' ΔΔLIST α
[10] ΔΔPRINT ''
[11] 'F' ΔΔLIST ω
[12] ΔΔPRINT ΔF

```

APL Product Guide

Compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

Fully Updated for Vector 9.1
Replies are awaited from entries marked with
sidebars: these may be removed from Vol 9.2

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL Visual Display Units and Printers
- APL-based Packages
- APL Consultancy
- Other Products
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 04393-385

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Active Workspace	AWL486	1,950	486 based 33MHz PC, 140MB Disk, 4MB RAM, VGA Colour. (inc. 1 year on site maintenance.)
APL People	IBM PCs & compatibles	poa	includes PC, mono/colour monitor, APL interpreter, operating system software, plus optional printers, graphics boards, additional memory etc.
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,617	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 18" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 18" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
Dyadic	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Interprocess Systems	APL2 Dev't Workstation	poa	Mainframe APL2 supported on a PS/2 via a co-processor card with 16Mb of memory running VM/ESA (370 mode). A complete system includes a PS/2, a P/370 co-processor card, and software licenses for VM/ESA, APL2, GDMM and the full line of interprocess APL2 enhancements.
MicroAPL	IBM RS/6000	12,000+	POWER range of RISC systems running AIX. Dumb terminal or graphical interface.
	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
Optima	IBM Compatible	poa	Complete PC-based station, APL interpreters & all support eq't

APL INTERPRETERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Active Workspace	DYALOG APL DOS 386	poa	Dyadic's PC 386 APL Interpreter.
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time Dyalog APL	poa 1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.

Cocking/Drury	APL*PLUS PC Rel 10	410	STSC's full featured APL for IBM's and compatibles - Version 10 includes the Quad-NA facility to interface to non-APL software, support for Microsoft Windows and mouse devices. The User-command processor has been built in to the interpreter.
			Upgrades to version 10 are available from Version 9 and earlier releases.
	APL*PLUS PC Run-Time	175 for 5	Closed version of the interpreter for developers, prevents user exposure to APL.
	APL*PLUS PC Developer System	950	Gives rights to distribute an unlimited number of copies of Run-Time application.
	APL*PLUS II System	1200	High powered APL interpreter for the 60386 chip. Price includes one year's maintenance and free upgrades - volume discounts VERSION 4.0 NOW AVAILABLE - THE ONLY APL THAT LETS YOU BUILD WINDOWS 7 DOS APPLICATIONS FROM ONE PACKAGE.
	APL*PLUS II Developer System	3200	Unlimited distribution of APL*PLUS II Run-Time applications!
	APL*PLUS II for UNIX	poa	STSC's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers running under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dyaldo	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Aitos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces I-APL/Mac		13	Macintosh version of I-APL
I-APL Ltd	I-APL/PC or clones	8 - 11	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	11	As above
	I-APL/Macintosh	13	As above
	Iverson Software Products		I-APL is the UK agent for ALL Iverson Software products, including: Sharp APL + enhancements to the ISI-APL standard Versions of J (and the associated texts) for PC, Mac, Arc, Atari. Please call for price list and order form. Please note there is a £3 packing charge on all orders.
IBM (APL Products)	APL Version 2 Release 1	poa	Full APL2 System for IBM 370 and 390. Product No. 5688-228. See your IBM Branch Office.
	APL2 Application Env't Ver2 Rel1	poa	Run-time Environment for APL2 Packages (IBM 370 and 390). Product No. 5688-229. See your IBM Branch Office
	APL2 for the RISC System/6000	poa	Product No. 5785-012. See your IBM Branch Office.
	APL2 for the IBM PC (US version)	\$495	Product No. 5799-PGG. Part No. 6242936, PRPQ No. RJB411 (in HONE, RJO411). (from IBM Direct, order by Part Number; from your IBM Branch Office, order by PRPQ Number)
	APL2 for IBM PC (European vers)	poa	Product No. 5604-260. Part No. 38F1753
	TryAPL2	free	Free APL2 for educational/demonstration use. Write to APL Products, specifying diskette size desired.
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. From all IBM dealers, including MicroAPL.
Iverson Software Inc.	APL386	\$30	Sharp APL Release 20 for PC 386, 486 with graphics, and ability to operate under Windows.

	APLIPC	\$30	For PC
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Optima	APL*PLUS/PC	369	
	APL*PLUS II	950	
	APL*PLUS II PC Developers Kit	poa	
	Dyalog APL	999	
Reuters Ltd	SHARP APL	poa	for IBM mainframes
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
	APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL VISUAL DISPLAY UNITS AND PRINTERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	IBM & compatibles	poa	IBM and 'budget' APL VDUs - monochrome/colour/graphics.
APL People	Epson series	200	Inexpensive dot-matrix and NLQ printers
	Quan-data & Qume etc	500	Daisy-wheel printers
Dyadic	IBM 3151	599	Monochrome APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
	IBM 6154	1,228	Colour APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
Dyadic	Printers (Various)	poa	Range of APL printers available.
General Software	Mellordata	poa	
Shandell	HDS Monitors	965-1395	A range of APL monitors. Please call for details.
	X-Terminals	1000-3500	View-stations for use with all X-Windows systems

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Syndicate Manager	poa	Lloyd's managing agent's syndicate / company accounting system. Stamp & Personal accounts (inc. Run offs)
APL-385	APL-385 FSM-385 DRAW-385 DB-385 GEN-385	50(PC), 125(mf)	Including ... Screen development Screen design Relational W.S. Miscellaneous Utilities
The APL Group	Qualedi	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	The First APL compiler!
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet incorporating data exchange between APL, FOCUS, IFPS, SAS, APL/DI, ADRESII, Lotus123, Visicalc, Multiplan & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(for PC's)	APL*PLUS PC Tools	275	Utilities including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	90	327x IRMA support.
	FIN & STAT. LIBRARY	250	Financial & Statistical routines
	SPREADSHEET MGR	150	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
CYBEX AB	APL Graf/PC	290	Presentation graphics for APL*PLUS/PC (CGI)
	APL Graf II/PC	390	Presentation graphics for APL*PLUS II/PC (CGI).
	Utility Functions APL2	1900	For APL mainframe; incl. a very fast search.
	Utility Functions II/PC	130	Same package for APL*PLUS II/PC.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling

	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces	IAC/Graf	15	Graph plotting for I-APL/Mac
	IAC/Vox	15	Spoken APL characters for I-APL/Mac
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
Impetus Ltd	Impetus	poa	Corporate Modelling and Reporting System.
INFOSTROY	Russificator	poa	Drivers and documentation for use with APL*PLUS/PC system and other STSC software with Cyrillic alphabet (PC).
Interprocess Systems	IEDIT	\$3000-5000	Full screen APL2 editor with immediate APL execution, and full-screen debugger
	AOC	poa	APL2 Optimizing Compiler, translates APL2 functions to FORTRAN programs.
(mainframe)	AFM	\$6500-15300	High performance component and keyed file system (VS APL and APL2)
(PC)	AFM	\$175	Single user component and keyed files for APL2/PC.
	Enhanced Format	\$2575	A QuadFMT data formatter for VS APL and APL2
	PowerCode	\$2000	External functions for APL2
	CALL/AP	\$4700	For calling non-APL programs (VS APL and APL2)
	WSORG	poa	Full-screen Workspace Organizer for APL2.
Marcia	STATGRAPHICS 5	635	The old favourite.
	STATGRAPHICS PLUS	895	Now running under APL*PLUS II - so you can have as much workspace as you want!
	LOGOL 90	poa	Logistics management system for 386/486 & RISC computers. Sales Forecasting, Inventory Management, Master Scheduling, Distribution Requirements Planning, Sales & Operations Planning.
	TWIGS	poa	A modular library of tools to teach and explore state-of-the-art materials management concepts. Developed by R.G. Brown.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
Reuters Ltd	STATGRAPHICS Rel 5	590	
	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	75	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	NEWSFLASH	poa	Real time message exchange
	VIEWPOINT	poa	4GL - Info centre product

UNIWARE (for mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	poa	component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	TOOLS & UTILITIES	poa	Including FILEPRINT, FILESORT, FILECONVERT, FILEMANAGER(EMMA) STSC's database package
	EXECUCALC	poa	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL(VM or TSO)
(for APL*PLUS/PC)	APL Debugger 2.1	FF1950 FF9750	A visual APL debugger to help develop applications (site license)
	Menus 3.0	FF2450 FF12250	Complete set of hierarchical menu utilities (site license)
	ETATGEN 2.0	FF1950 FF9750	Page layout report generator (site license)
	UNITAB 2.0	FF4550 FF22750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
(for APL*PLUS II)	UNIASM 3.0 (site license)	FF4950	Assembler utilities to speed up APL*PLUS/PC applications
	UNISTAT 5.1	FF2900	Data analysis add-on module for Statgraphics
	UNIWARE Toolkit II 4.1	FF39000	(site license only). Relational database system and complete set of utilities for APL*PLUS II development
	APL Debugger II 2.1	FF2950 FF14750	A visual APL debugger to help develop applications (site license)
	Menus II 4.0	FF3950 FF19750	Complete set of hierarchical mouse-driven menu utilities (site license)
	ETATGEN II 2.0	FF2950 FF14750	Page layout report generator (site license)
	UNITAB II 2.0	FF6950 FF34750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNISTAT Plus 5.2	FF4300	Data analysis add-on module for Statgraphics
	Warwick University BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	Consultancy	poa	PC Based APL system design, programming and implementation.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
APL People	Consultancy	poa	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL6800, C/Unix, TSO/MVS, VM/CMS, graphics, Operational Research etc. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.

Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	250-500	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-275 275-350 300-450 400-600 450-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Delphi	Consultancy	poa	APL system development on mainframes and micros.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
IAC/Human Interfaces	Consultancy	350	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	Localization of APL software for the Soviet Union software market
Intelligent Programs	Consultancy	poa	Systems development, enhancements, support.
	Documentation	poa	Preparation of new manuals, rewriting of existing materials.
	Training	poa	Training for APL experts through to non-technical system users.
Lingo Allegro USA	Consultancy	poa	General APL consultancy specializing in Prototyping, Migration, Mainframe to PC Downsizing, Performance Analysis, Troubleshooting, and Graphics.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
M.T.I.C.	Consultancy	240-500	Business analysis and APL consultancy
Optima	Consultancy	poa	A range of consultants with 3-15 yrs APL PC and mf experience.
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	350	Specialising in Banking, Financial & Planning Systems.
Reuters Ltd	Consultancy	poa	Consultancy & support service world-wide.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Rex Swain	Consultancy	poa	Independent consultant, 15 years experience. Custom software development & training, PC and/or mainframe.
Unlware	Consultancy (Senior)	FF/day 5000	Consultancy from people with at least 8 years APL experience.
	Consultancy (Senior)	FF/day 7500	Advice and training in Windows programming with APL*PLUS II
	Training	FF10000	5-day class on Windows programming with PLUS II version 4.0
Wickliffe Computer	Consultancy	poa	System design, consultancy, programming and documentation, Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — in English or French.
	The BBS\APL:		301-384-3672, 300/1200/2400/9600 b, N-8-1, 24 hours. APL educational material is downloadable free. An additional 30 megs of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr).
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL in Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)		
		16	185pp by Thomson
	J Dictionary	16	by Ken Iverson
	Programming in J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	An Introduction to J	8	47pp by Ken Iverson
	Tangible math	8	38pp by Ken Iverson
	Sharp APL Reference Manual	18	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
IBM (APL Products)	APL2 Keycaps	poa	Keycaps for the PC and PS/2 (USA and UK standard). Product Number SX80-0270.
	APL2 Keyboard Stickers	poa	Product Number SC33-0604.
Reuters Ltd	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.
Iverson Software Inc. J		\$24	Documentation & shareware for many popular machines, including PC, MAC, SUN, & ARCHIMEDES.
	J Source	\$90	Documented source code in C for porting to other machines (Shareware).
	Programming in J	\$15	78pp.
	Tangible Math	\$12	34pp.
	Arithmetic	\$18	123pp.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.	Visa/Mcd
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15	N/N
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60	N/N
Ass. Francophone pour la promotion d'APL France					
		Les Nouvelles d'APL			
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)	
CPC UG APL S/G (Capital PCUG)	Washington, D.C.	Capital PC Monitor	Monthly meetings, occasional classes	free	
Dutch APL U.G.	Holland	-	Mini-congress, APL ShareWare Initiative		
APL Club Austria	Austria	-	Quarterly Meetings	200AS per person, 1000AS per company	N/N
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings		

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245)-496647, Fax: 0245-496646.
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: 31-3474-2337, Fax: 31-3474-2342
APL-385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APLBUG	Jorge Mezei	110 Alma Street, #4, Menlo Park, CA 94025, USA
APL Club Austria	Erich Gall	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Club Germany	Dieter Lattermann	IBM Germany, Wickensstrasse 1a, D-6900 Heidelberg, Germany. Tel: (49) 6221-404-243
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: (203) 762-3933 Fax: (203) 762-2108
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: (33) 1 69 08 79 63
BACUS	Joseph de Kerf	Rocinberg 72, B-2570 Duffel, Belgium.
Anthony Camacho		11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-730036.
Paul Chapman		7 Queen Court, Queen Square, London WC1N 3BA. Tel: 071-278-5268.
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE. Tel: 071-436 9481 Fax: 071-436 0524
CPC UG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850, USA. Tel: 301-762-9372.
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: (46) 31-45 37 40. Fax: (46) 31-45 24 23.
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 6EX. Tel: 0804-760241
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 0367-710384
Dutch APL U.G.	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: 03474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256-811125 Fax: 0256-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0669-824741
General Software Ltd	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS. Tel: 081-864-9537
H.M.W. Trading Systems	Stan Wilkison	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA. Tel: 071-353-4212; Fax: 071-353-3325

HRH Systems	Dick Holt	Box 4496, Silver Spring, Maryland 20914, USA.
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics., LE16 8QL. Tel: 0536-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX Tel: 0388-527190. Email: clark.i@applelink.apple.com
I-APL Ltd	Anthony Camacho (for queries, order forms) J C Business Services (for pre-paid orders only)	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-760038 56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU
IBM (APL Products)	Nancy Wheeler	APL Products (M46V/D12), IBM Santa Teresa, PO Box 49023, 555 Bailey Avenue, San Jose CA 95161-9023, USA. Tel: 1-408-463-APL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
INFOTROY	Alexei Miroshnikov	3 S. Tulenina Lane, Leningrad 191186 USSR. Tel: 812-238-6392 Fax: 812-319-9709
Interprocess Systems Inc.	Stella Chamberlain	9040 Roswell Road, Suite 690, Atlanta, Georgia 30350-1131, USA Tel: (404) 992-8400. Fax: (404) 998-4869
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 071-285-1120
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: (416) 925-6096 Fax: (416) 488-7559
Lingo Allegro USA Inc.	Steven J. Halasz	203 North LaSalle St., Suite 2100, Chicago IL 60601 USA Tel: (312) 558-1342 Fax: (312) 346-9603
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX. Tel: 021-359-5096. Fax: 021-359-0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866
Ellis Morgan		Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants. Tel: 0730-263843
M.T.I.C.	Ray Cannon	21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS Tel: 0252 874697
Optima Systems Ltd	Paul Grosvenor	15a George St, Croydon, Surrey CR0 1LA Tel: 081 680-5958 Fax: -5856
Parallax Systems Inc.	Kevin Weaver	Avery Road, Box 319, Garrison, NY 10524, U.S.A. Tel: 914-424-4265
Peter Cyriax Systems	Peter Cyriax	22 Hereford Road, London W2 4AA. Tel: 071-229-5344
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB. Tel: 0276-20769. Fax: 0276-683427. Mobile: 0831-307548
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A. Tel: (212)-864-3078
Reuters Ltd	Paul Jackson	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA. Tel: 0203-256562.
The Rochester Group Inc.	Robert Pullman	50 S. Union St., Rochester NY 14607-1828, U.S.A. Tel: 716-454-4360. Fax: 716-454-5430
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax: 02407-3118
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel: 203-868-0131
Sydney APLUG	Rob Hodgkinson	GPO Box 1425, Sydney, NSW 2001, Australia
Unlware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France. Tel: (1) 45-27-20-61. Fax: (1) 45-27-20-71. Telex: 648348F UNIWARE
Wickliffe Computer Ltd	Nick Teifer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel: 0946-692588
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523369
Zark Incorporated	Gary A. Bergquist	53 Shenipsit St, Vernon CT 06066, USA. Tel: (203) 872-7806

Six Months in the Life of VECTOR

by John Searle

Conception

Firstly, a bit of APL. Let us consider the creation of the issue Volume X number Y (X.Y), and refer to the next issue as (X.Y)+1. However, since VECTOR is a quarterly journal, the previous issue may be more exactly specified as:

```

      V R←A PLUS B
[1]  R←0.1×10⊥ 0 1 + 0 4 ⌈B+4⊥ 0 -1 + 0 10 ⌈10×A
      V
      8.3 PLUS 2
      9.1
  
```

An issue of VECTOR has a gestation period of about six months. The working group meeting discusses the next issue (X.Y+1) in some detail and a draft contents page is created. This gives the Editor enough time to identify the holes so that we can all help to fill the gaps. If all goes well, we can then get down to planning the issue after, X.Y+2.

Material for an issue of VECTOR comes from a variety of sources, and much of it now has its own momentum, meaning that the Editor does not have to write it. The Editor will decide if this to be a thematic issue, and request articles relevant to the theme well in advance. Other articles are either requested or arrive through the post with a wonderful irregularity.

One of the main aims of recent VECTORs has been a drive to have regular features appearing either in every issue or in every annual issue. VECTOR X.1 is now always produced in time for the annual APL conference, and is given away free to each delegate. Advertisers: this is a very good issue to aim for! The following issue X.2 always has a comprehensive report of the conference, with summaries of the main papers and lots of photographs. The January issue X.3 is generally a thematic issue with invited papers. The April issue X.4 can be relied upon for the abstracts of the forthcoming conference, and may contain an article that wanders from the truth in the best of April 1st traditions.

The Meeting

VECTOR requires a quarterly Working Group gathering; a meeting of minds and travelling bodies prepared to risk the motorways and railways of Britain in order to bring you the latest in the world of APL. It is an amalgam of creative thinking, a free orange juice at BAPLA's expense, another opportunity for a hotbed of crazy ideas for the next April 1st issue piece. Verily, a travelling show from Yorkshire to Berkshire, from York to London to East Garston (where's that??) to St. Albans and back again.

The best thing about the meeting is getting a copy of the pending issue hot from the printer's press. If you are very lucky, Adrian will let you take it home afterwards. Fingers quickly flick through this pristine copy for the vital signs - does my latest contribution appear? Does it have any (more than the usual number) of misprints.

The tangible output from the meeting will be a fleshed out Contents Page and Anthony's minutes, with hopefully my name in bold Action font not too often repeated.

Afterbirth

Who are you? Who actually reads VECTOR? We know who we send it to, but who eventually reads it? As the BAPLA does not keep the names of APLers in corporate memberships, we do not really know the full story, although steps are being taken to change this. My reading of the BAPLA membership numbers suggests there is an increasing number of you, but feedback to the Editor is very low. *Please write!* A brief note of con-, in- or de-structive criticism would be very valuable to the working group.

Another question for you - how do you store your VECTORs? (Assuming you keep them!) In chronological order or in rainbow colour order?

The working group is very proud of your journal. It is all done for love, a lack of money, and a feeling of doing something professionally worthwhile. However, the group constantly needs fresh blood. Your interests could range from the technical, statistical, commercial or sociological, to the not to be forgotten right of just having a say in the industry that employs you.

And finally, the last and most important question: **what colour should the cover of the next issue be?**



Cocking & Drury Ltd

**Your sole UK supplier
of the entire APL*PLUS software range**

including:

APL*PLUS PC

APL*PLUS II for the 386

APL*PLUS II for UNIX

APL*PLUS VAX/VMS

APL*PLUS Mainframe

**as well as Run-Time and Run-Time Developer Kits
and all delivered ex-stock!**

**The only dealer authorised to provide maintenance and support for APL*PLUS
All software prices include 12 months free maintenance, with free upgrades (except
APL*PLUS PC).**

**We also offer a wide range of courses in APL and the most experienced APL
consultancy service in the country.**

Whatever your APL needs - call the experts first.

Cocking & Drury Ltd 180 Tottenham Court Road, LONDON W1P 9LE

Phone 071-436 9481 Fax 071-436 0524

APL*PLUS II/386 Version 4

reviewed by David Crossley

Background Review

Although I have used STSC's APL*PLUS extensively on mainframes, I had not used any of their PC-based products before being asked to undertake this review. Consequently the opportunity to look at the latest version of STSC's top-end APL for 80386/486 machines was an exciting prospect.

For many years, STSC has very successfully provided APL*PLUS/PC running under MS-DOS for 8086/88 (and 80286) machines. It is currently in its 10th version. It is however limited by MS-DOS's conventional memory restrictions, effectively placing a 640k-byte ceiling on workspaces. In 1988, STSC launched APL*PLUS II with the Phar Lap DOS Extender to take advantage of extended memory above 1 Megabyte available on 80386 machines. This chip supports RAM up to 16 Megabytes. Workspaces could occupy most of that extra memory.

APL*PLUS II/386 is an advanced APL supporting nested arrays. A very pragmatic approach has been taken in providing 'hooks' for interfacing with the world outside APL. Various facilities have been added or improved with subsequent versions. The original product and Version 3 have been reviewed in previous editions of Vector [Cyriax 89; Askoolum 89; Pearson 91]. Version 4 brings a host of minor improvements and three major extensions:

- the ability to run within MS Windows;
- language changes aimed at compatibility with IBM's APL2;
- extra windowing features including editing subwindows.

The latter two features may impact on existing applications code.

The Physical Product

A sizeable box contained three manuals (User, Reference and Utilities), an installation guide and a Windows Interface booklet. I was impressed with the documentation. It was easy to read and easy to refer back to later. It is a pity that the manuals are not supplied in ring-binders; keeping the page open with one's elbow whilst typing in cryptic system function arguments requires some skill. But this is a minor criticism. The software was provided in compressed files on 6

diskettes in both 3½" and 5¼" formats. Installation was quite involved but generally worked the first time. Decompressed, the software uses about 10.5 Megabytes, but in practice this could easily be reduced to 5 Megabytes or less by picking out what you really need; there are over 4 Megabytes of provided workspaces alone. This compares well with other PC products such as Windows and Wingz.

There is also a rather nasty gadget called a Sentinel which fits on the printer port. The software will not work without it, and it is checked regularly. In my case, with a tower system beneath a work surface and the usual short leads all screwed to the back, fitting this device involved a bumped head, various scratches, and air so blue that even the dog felt obliged to leave the room. Although it can be left in place, I hope vendors of other software products I might install do not insist on a similar contraption.

The APL Language

STSC and IBM developed their nested array versions at about the same time in the early 80's, broadly along the same theoretical basis (or heresy, some would say) adopting the "floating" methodology and strand notation. I believe STSC was the first vendor to provide a research system (NARS). There were conflicts with IBM's APL2 in the final implementations in that some primitives acted differently and there were syntactical anomalies. STSC have nobly decided to move to the APL2 way "thus allowing the APL community to be more in agreement on an APL standard".

Version 4 of APL*PLUS II/386 will be the last to support the old (i.e. current) behaviour. It will also support the new (i.e. APL2) behaviour. This is achieved by an "evolution level" which may be set as a configuration flag or by the system command `)EVLEVEL`. To help detect occurrences of deviant behaviour in existing code, an evolution level of 0 will report an *EVOLUTION ERROR* on execution. Level 1 assumes the old and 2 the new behaviour. Primitives that are affected can be replaced by a system function that is independent of level, such as `⊖MIX` or `⊖FIRST` for monadic `↑`. Strand assignment and indexing both require the strand to be within parentheses in level 2, e.g. `(A B C)←data`.

I know already from my mainframe experience with APL*PLUS that the nested array environment is both practical and productive. I am pleased to see that a blind-spot in the implementation - the inability to modify nested elements of an array below the top level - has been rectified with the introduction of selective specification for a subset of primitive functions.

For example:

```

      A←(1 2 3){'abcd'}(2 2ρ4)
1 2 3  abcd  4 4
      4 4

      ((3(2 1)ρA)+5
      A
1 2 3  abcd  4 4
      5 4

```

Otherwise the basic language remains unchanged, although the code for dyadic epsilon and iota is reputedly much faster at the expense of more temporary workspace.

The number of system functions has been further expanded. As well as the evolution level primitive substitutes, such as `⊖MIX`, `⊖FIRST`, `⊖SPLIT`, etc., there are new functions to support windowing extensions (see below). Altogether there are now 172 documented system functions, variables and constants plus a few that are deliberately not documented. This compares with about 50 symbols used for primitive functions and operators. Adrian Smith's controversial reference to `⊖KITCHENSINK` springs appropriately to mind [Smith 87 and follow-up letters in Vector 4.4]. Too many system functions have arcane, highly parameterised arguments which contradict the underlying elegance of APL. Practical they may be but surely there must be a better way.

The APL Environment

STSC has developed a very nice text-based windowing environment which has seen further improvements in this release. Just about everything is configurable by the user, from the keyboard to the screen with your own choice of colours (from the admittedly small palette of 16 colours imposed on text VGA modes). You can set these parameters in a configuration file so that your preferred style is always presented on calling APL. However the default settings are reasonable, presenting a familiar APL desk-calculator look.

The session is recorded up to the limit of reserved disk space which you can set. Thus scrolling backwards and forwards is allowed. One or more lines of text may be edited and 'dragged down' to the current line for re-execution. Sideways scrolling has also been introduced so line wrapping can be avoided.

Previous reviewers have commented on the myriad of keystroke commands that allow you to do just about anything, such as opening lines, copying blocks of text, moving blocks of text and so on. Of course this requires a significant, non-

transferable learning curve. Fortunately, mouse support has been greatly improved and most actions can be achieved with much less effort using the rodent.

Navigation of the edit environment has been greatly enhanced by a new box feature that provides facilities familiar to Windows users. The boxed surround includes vertical and horizontal scroll bars where appropriate, a zoom-in/zoom-out icon, and the edit window may be re-sized and moved by mouse drag actions. All very slick. A menu bar can be brought into view by mouse or keystroke action with pull-down menus to provide a variety of services. There is also a much improved help facility to which you can link your own customised help files for applications.

The keyboard can be configured in the familiar APL form or in standard ASCII form (or indeed in other standard or customised forms). In the former, ALT and ALT-Shift key combinations make the alternative APL/ASCII configurations available. Another way which I found easier is to switch character sets using ALT-CapsLock when mixing APL and ASCII text, e.g. in function definitions.

The full power of the editing facilities is extended to sub-windows in which functions, text or numeric arrays (up to rank 2), and text DOS files may be edited or simply browsed. Up to 32 sub-windows may be open at any time, and they may be rotated using keystrokes or moved to directly by mouse click. There are numerous ways of initiating an edit sub-window. The simplest is to double-click a name with the mouse or enter a keystroke combination with the cursor on the name. If the object exists, a sub-window is opened for editing. If not, a configurable default type is assumed which you may alter. Another way is to use the system command `)EDIT`. But perhaps most powerful is the ability to invoke a sub-window using one of the system functions `⎕ED` or `⎕EDIT`. Along with the new window functions, `⎕ECTRL` and `⎕WCTRL`, a great deal of flexibility is provided to build and manipulate edit windows under program control.

There is, though, a lack of debugging aids relying mainly on the now old-fashioned `⎕STOP` and `⎕TRACE` mechanisms. However, `⎕MF` (monitor function) is made more useful by linking to a more refined timer.

Graphics Support

As in previous versions, APL*PLUS II/386 supports both the `⎕G`-style of graphics inherited from the PC product, and also GSS*CGI graphics, a device-independent set of external routines interfaced through the undocumented system function `⎕CGI` which you access using the utility functions in workspace

VDI. The range of graphics drivers for the latter have been updated as "an optimal mix" from GSS*CGI Version 2.20 and earlier.

The External Environment

Those of you who have grown up with the home computer will be familiar with peeks, pokes and interrupts. Direct access to CPU memory locations was a way of life, and the only way to get the computer to perform certain tasks. STSC has provided the requisite tools for such direct control via system functions `□PEEK`, `□POKE` and `□INT` in all of its PC products. Personally, I am not of this fraternity. It is with trepidation that I issue a `□POKE` or `□INT` knowing that it could seriously affect the health of my computer. STSC warns you of possible dire consequences, but then frequently suggests uses to achieve desired actions though normally relating, admittedly, to more advanced programming control such as windowing. You can set a flag via `□SEG` to use the `□POKE` location codes originally devised for the PC product, or you can directly access actual memory locations up to 1 Megabyte.

However, if you need to do these things there may be no other choice. The wider range of mouse options available through `□INT` compared with `□MOUSE` has been noted [Pearson 91]. I believe that the VGA colour potential has been ignored by APL but `□INT` has proved useful in rectifying this omission [Smith 90; Crossley 91].

Two facilities for the more serious low-level coder are `□CALL` and `□NA` (for Name Association, I believe), the latter introduced in Version 2.0. `□CALL` allows 80386 protected-mode machine code to be run directly from APL. `□NC` allows programs written in a compiled language to be associated with an APL name which may then be run as a monadic function. Various C and FORTRAN compilers are supported. (Note that a new version of Phar Lap 386 is bound to APL*PLUS II/386 and Version 4.0 of the debugger may be required.) I accept the value of these facilities and will take advantage of supplied utilities, but personally I prefer to write my code in APL.

Microsoft Windows Interface

Does it work? The answer is yes, but the reality is rather like playing Snakes and Ladders; the elation of ascending each ladder is tempered by the disappointments of the snakes.

Of course, Version 3 of APL*PLUS II/386 could not even co-exist in the same DOS session as MS Windows 3. But a new version of the Phar Lap DOS Extender

has cleared the memory conflicts. Version 4 runs in 386 Enhanced Mode of Windows 3. You can run APL as a non-Windows full-screen application. This allows you to use the mouse within APL and control the environment just as normal. To switch to another application, you must press the standard ALT-ESC key combination to return to Windows. The session you left is iconised. Double-click with the mouse on the icon to return to the APL session. You can have more than one APL session going if you like. All well and good, but is this really Windows?

Well, you can also run APL within a window, but then the mouse becomes transparent to APL so you will have to learn all those edit keystrokes after all. You must also reserve keystroke combinations in the PIF file to prevent interception by Windows. Just nine vivid colours are available which, it seems, cannot be altered.

However, you can write Windows applications driven by APL from a multi-window environment, which is probably an improvement on writing them in C. A very nice little booklet starts you on your way to becoming a Windows application writer, though you are expected to have MicroSoft's Software Development Kit or equivalent to do the job properly. It explains the philosophy of Windows programming.

Essentially, your APL driver window (with defunct mouse) communicates with the (non-APL) application window via an APL Agent window. Windows programming instructions, issued through the supplied C-routines via utility functions, do things to the application window, such as creating menus, buttons, scroll bars and so forth. Events in the application window, such as keystrokes or mouse actions, are passed to the APL Agent window in coded form where they are displayed. The APL Agent can filter out events to pass on to the APL driver and leave Windows to take default action on others. For example, the application may be interested in a mouse left-button click but no others (and certainly not mouse movements without clicks which can generate hundreds of events per second). Thus the APL driver can take some appropriate action, such as perform a calculation or alter something else in the application window. The APL driver and Agent windows would of course be hidden when the application becomes operational.

The example application performed some quite complex tasks, such as building pop-up windows, yet was easy if long-winded to program. It is not really APL but it is moving in the right direction.

Yet More Interfaces

A great deal of software is bundled with APL*PLUS II/386. The *Interfaces and Utilities* diskettes amount to some 6 Megabytes when decompressed. There is far too much to attempt a review but here are a few snippets:

- an APL interface to Borland's *Paradox* database product.
- full supporting workspaces for the *IRMA 3270* emulation board.
- a user command processor to build your own system commands, plus a useful library of pre-defined commands.
- a Screens Interface Toolkit that enables you to build text-based screens based on *WIN*.
- a report generator.
- workspace and file maintenance facilities.
- lots of programming utilities and development aids.

Conclusion

I may be wrong but I would guess that IBM's APL2 holds the lion's share of the APL market for small machines if for no other reason than a conservative tendency to devolve from the mainframe. However, STSC must dominate the non-tied market. And to quite a large extent, the development of APL*PLUS II/386 shows distinct signs of being market-driven. It satisfies a very diverse range of needs. There is probably a way to interface with most external devices and software, yet present it through APL.

Nonetheless, there does seem to be a lack of structure in the development process. The proliferation of system functions and variables is surreptitiously turning APL into yet another parameter-driven language. There is too much duplication and overlap — witness for example *ED* and *EDIT*. The commercial reasoning is understandable. Customers demand support for this or that facility now, if not yesterday. They may have invested much effort in writing software and having to change code to migrate to a higher version would not be popular.

The beauty of APL lies in its concise, orthogonal elegance which makes it such a highly productive language. To lose these qualities begs the question: why use APL at all? For APL to ultimately succeed, its underlying strengths must be exploited, new additions must sit comfortably within the APL framework. This

requires properly thought-out design by APLers, not transplanted glitches from other orthodoxies. This is not solely an indictment of STSC but rather a serious question for all implementers to ponder. STSC has taken the bold step of moving towards APL2 in the interests of a common standard. Should not other language features be viewed in the same light?

But back to pragmatics. APL*PLUS II/386 is already a tried and tested product with a successful pedigree. What I had time to try was fast (though admittedly using a 40836/33 MHz machine). The session and object editors, particularly with the enhanced mouse and boxed windowing features, are very impressive. But the move towards integration with MS Windows 3, albeit not perfect, must for many existing users be the biggest inducement to upgrade. For first-time buyers, you get a very versatile and proven product with an enormous quantity of support software and distributed workspaces.

References

- [Smith 87] Adrian Smith, *Dyalog APL on the IBM6150*, Vector 4.2, p53
- [Cyriax 89] Peter Cyriax, *APL*PLUS II: STSC's Second Generation APL System*, Vector 5.4, p55
- [Astoolum 89] Ajay Astoolum, *APL*PLUS II for 80386PCs*, Vector 5.4, p61
- [Smith 90] Adrian Smith, *Lots More VGA Colours*, Vector 7.1, p110
- [Pearson 91] Duncan Pearson, *APL*PLUS II Release 3*, Vector 8.1, p73
- [Crossley 92] David Crossley, *Panel Design: An APL Programmer's Toolkit*, Vector 8.3, p74

RECENT MEETINGS

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

French Visitors in Napoleon's Footsteps

Translated from Obninsky Newspaper 20 February 1992

by Michel J. Dumontier and Ludmilla Lemagnen

In February a group of French scientists visited Obninsk: Gérard Langlet, crystallographer and theoretical physicist at CEA, Saclay; Jean-Jacques Girardot, Professor at the Ecole des Mines de Saint-Etienne; Michel Dumontier, Engineer from the Research Centre of the Defence Ministry. The Manager of the group was Ludmilla Lemagnen.



The Delegation at the Atomic Centre: A. Skomorokhov, G. Langlet, D. Lukyanov, L. Lemagnen, Mrs. Kuprianova and J.J. Girardot

At the invitation of the Union of APL Users, the group took part in a seminar in Obninsk together with specialists from CEI. APL occupies a unique place in the computer industry world-wide. APL was invented by K.E. Iverson at the beginning of the 1960s to be a 'tool of thought' for the Computer Age. Now APL has spread all over the West. In the USSR (we will use the old name) a Union of Soviet APL Users was created in Obninsk and registered with the Town Council in 1990. USAU is a union of non-commercial organisations for the development and expansion of computer technology with a strong APL bias, for its introduction into scientific, industrial and educational domains and for the promotion of APL Russian and International seminars.

Among the six members of the Committee, three are from Obninsk: V. Kuprianov, A. Skomorokhov and O. Luksha.

When a journalist does not fully understand his subject he must rely on a specialist, and trust him to explain it to the reader. So I asked Oleg Luksha to describe the seminar.

"Immediately after the creation of USAU we began methodically to develop international contacts. In 1990 Soviet programmers took part in the annual international APL conference, that year in Copenhagen. Two papers were presented by us and they caused a sensation. It proved that Soviets were not born yesterday! Also in 1990, in Leningrad, there was a Soviet-Finnish seminar to which 30 Finnish and 50 Soviet specialists contributed. Thereafter events followed one another rapidly. The decision was taken to hold the 1992 international APL conference at St Petersburg.

Last year, in Obninsk, we celebrated the fifth anniversary of APL activity in our country. We have been visited by specialists from several countries, one of whom is the chief APL architect of APL in IBM, Jim Brown. Our contacts are still widening greatly: 11 people from Obninsk took part in the conference 'Using APL in Statistics' that was held in Finland. We have established a high level of international scientific relations and today have up-to-date information on the problems in which we are interested.

The French-Russian seminar that was held in Obninsk could be called more precisely a school for our specialists. Gérard Langlet gave a lecture: 'Towards a New Information Theory'. His new theory is based on algorithms. The lecture caused a shock. For us it is an interesting and revolutionary theory. Langlet is a polymath. He works in biology, chemistry, botany, physics and linguistics. Afterwards Ludmilla Lemagnen was overheard saying he is a genius.

Jean-Jacques Girardot has created a new programming language called CTalk, based on APL concepts. Girardot is a man of great authority in modern computer technology. Michel Dumontier's lecture was also a shock because of the amount of research in it that was new to us. Roughly, evaluating all the work of the school, we can say that Dumontier has opened new horizons and new domains in the use of APL. We consider our seminar was a significant step forward which will promote the success of the international conference at St Petersburg."

Apart from the science, the seminar had other fruitful results. French people have visited the IATE, and they made contact with the Ecole des Mines of Saint Etienne and will now exchange groups of students and undertake joint projects. Our guests have greatly appreciated the atmosphere of the Humanitarian Centre. They thought the standard of the students of the Centre to be higher than the mean standard in France. The French delegation was entertained by the Mayor of the town, Yuri Kyrillov. They had the idea of twinning Obninsk with Saclay, and our guests returned to France with a letter from our Mayor to the Mayor of Saclay.

Stone-walling in APL

A Talk to the British APL Association

by Adrian Smith

Introduction

This talk falls naturally into two parts: the first is about philosophy, and the second about practical detail. In order to be an effective wall-builder, you need to get both parts right: it is no use having all the right attitudes if you can't fit the stones together; conversely all the craft skill in the world will avail you nothing if you lack the vision to build your wall in the right place!

Incidentally, the title (and some of the subject matter) arises from an unguarded comment I recently made to Ken Iverson (on holiday in Yorkshire) that I thought that the most useful bits of my education were at primary school in the Durham dales. In particular, dry-stone walls are an intrinsic part of the local culture, and the skills involved in building them are somehow "in the blood" of any dales native. Being Ken, he immediately picked me up on this rash statement, and I spent a tough few minutes rationalising just why the art of stone-walling is helpful in designing APL systems. What follows is a first attempt to put these thoughts together into a coherent essay.

Philosophical Stuff

The world is full of hype and fluff about OOPs. You can design your systems with an OOM (Object-oriented methodology), set up the data with an OOD, and hack out the code in an OOL. What no-one seems to realise is that none of these fancy ideas will do any good unless the people who use them have the right attitude of mind. Let me illustrate with three approaches to wall-building:

- the project plan involves creating a visible barrier from Bowness-on-Solway to Newcastle-upon-Tyne. The barrier must be a firm statement about the power and prestige of the builders, as well as serving a practical purpose of keeping out the Scottish riff-raff. The 'design and build' cycle went something like this:

Measure the total distance: every mile set down a milecastle; between each milecastle set down three equally spaced turrets (a Roman mile is roughly 1611 modern yards).

For each yard of wall, calculate the number of standard stones (each 1ft by 9" by 6"), and hence calculate the number of stones required for the entire wall (approx $150 \times 1681 \times 85 = 21,432,750$).

Add the extra materials required for the milecastles (each with a fancy arched gateway etc.) and set up the appropriate management structure (Quarrying, Transport, Engineering, etc.) to deliver the right quantities of the right materials to the right place at the right time (these days it would be called Logistics).

This approach is appropriate if: you know exactly what you want; you control the future; you have an unlimited supply of slave labour. For a modern example, look at the way IBM built OS/2. It leads to fascinating local anomalies like the gate at Milecastle-37 (just along from Housesteads fort) which opens directly over a 200ft cliff.

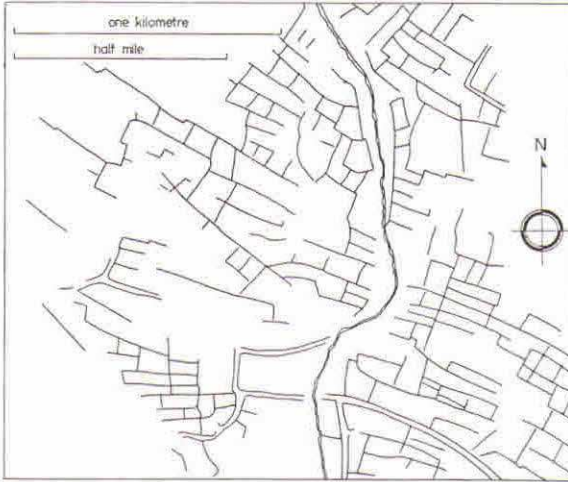
- For a completely opposite approach to the art of wall-building, you need to travel to Connemara in Western Ireland. Here the ground is littered with large boulders, and in order to create fields you must pile them up into walls. The result is a fascinating landscape, where many fields have no gateways, and the only access is by temporarily unpicking a section of wall!

This way of working is typical of the Apple Computer Corporation. It is appropriate if: tomorrow may never come; you are having far too much fun building walls to stop and ask yourself why; you have lots of rocks to get rid of. For an APL example ... "Oh look, there's an interesting rock with $A++$ / written on it ... we could fit that in here ... now how about this one with $\circ$ surely we can't leave that out, it's far too pretty" (with apologies to Dyalog fans).

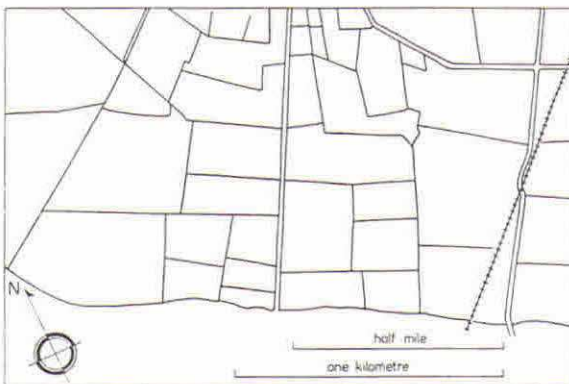
- There is a middle way! Take a weekend break anywhere in the Yorkshire Dales, and look and learn. The first thing you will see is **Vision**: walls which run in delightful patterns up valley sides and along the edge of the unimproved moorland. The second thing you will see is an **Awareness** of existing materials and resources, coupled with the **Skill** needed to fit them together. Finally (if you look hard) you will begin to see that each stretch of wall is the outcome of a **Negotiation**; where there was a soggy piece of ground the wall went round it; walls tend to meet streams at places which are easy to bridge, and so on. The client got what he wanted (the wall kept most of the sheep in the right place, most of the time); the builder got some fun out of picking the most sensible route; the whole thing was done as cheaply as possible by using the stone that just happened to be lying around!

This way of working typifies my OOA (*Object-Oriented Attitude*). It works well when: you are allowed to negotiate the spec; you have limited time and resources; you might have to live with the result (and patch it up occasionally).

As a final illustration, here are three sets of field patterns [1] which stem from the three approaches to wall-building:



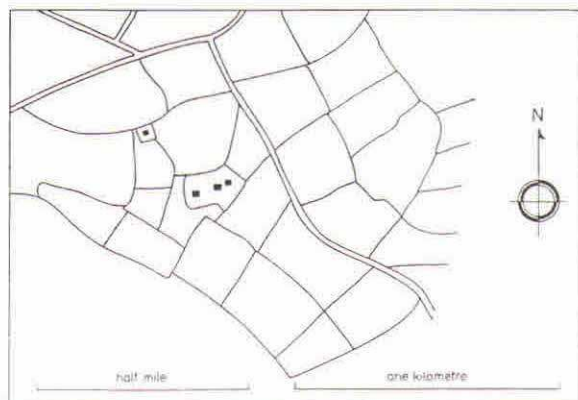
Prehistoric field system, Piddletrenthide, Dorset. Bounded by earthen banks and scarps.



Fields formed by parliamentary enclosure of common arable, Slawston, Leicestershire. Thorn hedge boundaries.

The haphazard layout of the prehistoric field system is in stark contrast with the ruler-straight lines and carefully partitioned areas of the enclosure fields. I find it hard to blame modern farmers for ripping out most of these recent quickthorn hedges and reverting to the previous open-field system!

The last example shows the OOA in action: here you can see both vision and sensitivity. The fields are more or less the same size, the lane is more or less straight but the contours of the underlying landscape are still clearly visible and the radial pattern round the farmstead merges elegantly with the grid pattern further away.



Fields formed for sheep farming, late fifteenth century, Little Rollright, Oxfordshire. Botanically mixed hedge boundaries.

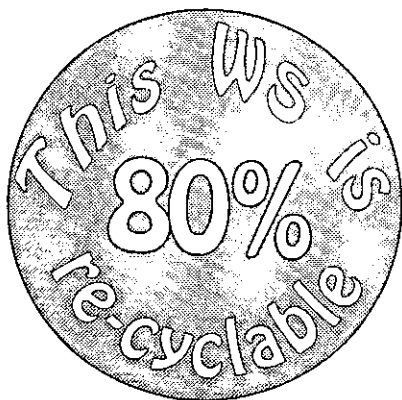
To summarize part-1:

- Object-orientation is in part about techniques, but mostly about an attitude of mind.
- Techniques are easy to teach, but I don't know how you inculcate attitudes. I have always had an OOA, and yet my 8-yr old (in spite of a hefty dose of parental guidance) will take the Roman approach every time. (The 6-yr old seems to be going down the anarchist route, but it is too early to be sure.)
- It may help to get out into the country and try and understand why things are the way they are. Go and walk a section of Roman Wall, go and find some prehistoric fields in Cornwall, go and drive along some of the enclosure roads in Lincolnshire. You may not come out with a fully developed OOA, but you'll surely feel better for it!

Practical Stuff

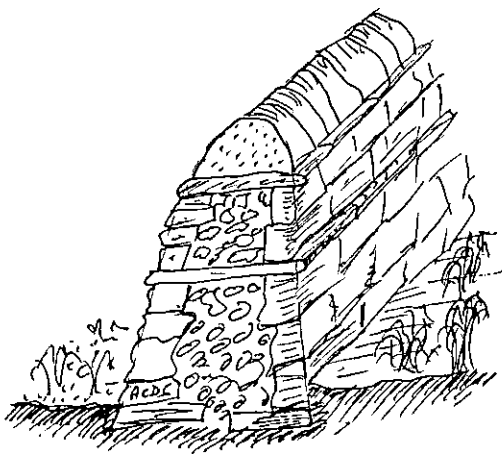
How Green is your workspace? I think 80% recyclability is achievable in almost any APL application, and if we can do that, we have a right to join the Object Oriented club! What I want to do is take a complex application (an interactive planning board, with inbuilt data-maintenance and graphics) and unbuild it layer by layer to show how you can put your OOA into practice.

This particular example is by no means perfect, and in fact it only achieves 58% recyclability. Suggestions for improvement welcome.



Here is a short length of wall: there are a few things about it which you should notice:

- there is a foundation layer below ground.
- the wall is broader at the base than at the top.
- there are two clear strata of 'through stones' which both bind the wall together horizontally and isolate the layers vertically.
- the top layer of cap-stones is well isolated from the rest of the wall. It is to this layer that the majority of damage occurs; it is designed to fall apart relatively easily without causing severe structural damage to the wall below.



The need to cover APL idioms (e.g. $(+/\wedge\backslash)' '=X)\phi X$ by $\Delta IJUST$) is less obvious, and clearly there are numerous grey areas. I have a utility called $\Delta WIDTH$, which just does a $\uparrow$ on the columns (i.e. the last axis) of an array. This saves me a bunch of parentheses at some minimal overhead in execution time; whether you want this kind of thing in your code is mostly a matter of taste. On the other hand, it is well worth covering 'table find' (mine is called $\Delta IOTA$) as the idiom is pretty hairy, and there are numerous fancy algorithms to make it go faster. These algorithms are very environment dependent (on APL*PLUS/PC you just bury $ROWFIND$) so it is important to isolate them from your application.

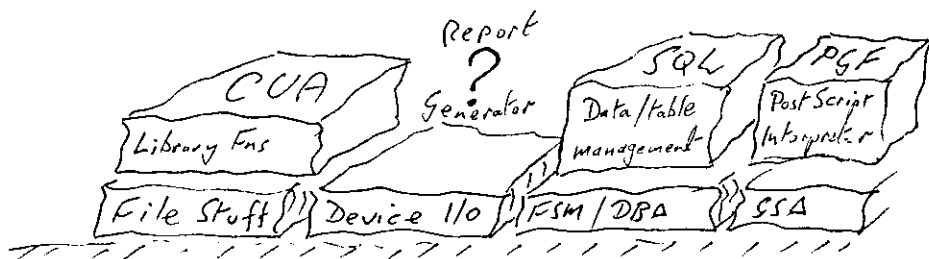
Of course you sometimes find that a 'utility' collapses into a primitive (e.g. ΔSS to $\underline{\epsilon}$) but the behaviour is often different in some circumstances (which way round do the arguments go, how does it handle empty objects, does it extend to higher dimensions ... $\Delta IOTA$ does) and I still think you should treat anything remotely suspect as the lowest visible layer of your wall, and not risk burying it in the foundations.

To summarize, the lowest visible layer provides environment-independent functions to allow your application to:

- get data from files, and write it to files. You will probably need to handle at least 'native' files (QSAM, VSAM etc. on mainframes, ASCII text on PCs) and 'component' files. Increasingly, you may also need to talk to relational databases like Oracle and Paradox.
- send things to printers. This may be a subset of the above (you can usually treat a printer just like a file on DOS and Unix), or it may require a whole raft of really horrible code if you have a bunch of Xerox 4045s attached to your mainframe.
- send stuff to the screen. This covers everything from formatting numbers to some reasonably device-independent way of writing text to the right part of the display. I started my *FSM* functions on dear old AP124, and they have been going strong ever since. It was fascinating to see how easily they could be moved to Dyalog/W ... suddenly all my old mainframe code came up looking quite authentically Windows. (Duncan has taken time off to get married, but he will definitely be writing up his notes on Dyalog/W for the next Vector!)
- manage tables of data. I like to handle groups of related variables by giving the group a name, and then treating it as a single logical table. I only need 4 functions [2] to do the obvious manipulations ($DB\Delta RHO$, $DB\Delta TAKE$, $DB\Delta COMP$ and $DB\Delta INDEX$) each applied to the namelist (in $\square NL$ format) of the variables in the table.

- handle the basic graphics functions. At this level, I mean drawing lines, marking points, filling polygons. Many of these are trivial in many interpreters, but try writing a robust polygon-fill in APL*PLUS $\square G$. . . and see how far you get!
- any other obvious generics, where either the idiom is too hard to remember (e.g. 'delete trailing columns') or the function is highly machine-dependent (e.g. *VGA PALETTE*). I err on the side of making everything a utility, if only because I have twice been through the pain of moving interpreters, and I don't want to inflict the same suffering on those who may come after me.

The next layer of the wall looks like this:



It is much tougher to define this layer well, but if you are serious about that 80% target, you need to do it. Let me start with the easy bit!

Library Management (the CUA layer)

This fits directly on top of the file functions (which file functions you use is up to you). Until recently, there has been no sensible standard for getting data to and from an application, so many APL systems simply copied the `)LOAD`, `)SAVE`, `)DROP` convention established by the interpreter. More recently, people have begun to copy the Lotus-123 style (File-retrieve etc.), but as of the release of Windows-3, there is really no alternative:

File ...	New
	Open
	Save
	Save as ...
	Exit

... and anything which might result in loss of data (e.g. 'Save as ...' with an existing file name) must be confirmed by the user. I have a little raft of functions

LMΔNEW, *LMΔSAVE* and so on, which offer something very close to the CUA standard behaviour. I also included *LMΔDROP* and *LMΔLIB* as users sometimes need to get rid of data too (a fact not currently acknowledged by the majority of windows applications).

The advantage of decoupling these functions from the raw file handling should be obvious! As an example, I recently converted the entire set to use the APL*PLUS/PC *Δpack* workspace [3] for a 10-fold increase in speed. On Plus-II and Dyalog, you are better off with component files, APL2/PC users might like to use AFM, and so on.

Report Generation

This is the one big gap in my wall! The data-management layer (see next section) offers the basics, but it does not go beyond simple tabular listings, with basic paging across and down. The problem is that most APL systems are not simple, and that much of the printed output is highly specific to the application. Often it involves some kind of timetable (days across the page, machines down the side, activities noted in the appropriate cells) which is a structure I have never managed to genericize. Sorry!

Data Management (the SQL layer)

As with the CUA approach to file handling, there is an accepted standard for data management, so the simple message is use it!! This layer also begins to illustrate the importance of those 'through stones' which isolate the layers from each other. Whatever access method you use, you need to apply a lot of self-discipline to stop yourself bypassing it!

The great advantage of a 'real' SQL database (or Paradox database or whatever) is that you physically can't get at the data other than through the authorised channels. The problem with doing all this in APL is that you know that database is actually just a bunch of variables sitting around in the workspace, and the temptation to update them directly can be overwhelming!

Typically, this layer looks like:

```
CREATE TABLE 'EMP'      ⍝ Define a new table
CHANGE TABLE 'EMP'      ⍝ Change the table definition

SELECT '*' FROM 'EMP' ⍝ Returns a simple formatted list
                      ... WHERE ΔSALARY>20000 ⍝ etc

CHANGE '*' FROM 'EMP' ⍝ A simple full-screen edit
```

The use of the SQL syntax is fine if you give this layer directly to users (as long as you front-end it with a little routine to patch in the quotes!), but I think it is probably a mistake from an application-builder's point of view. Better would be:

```
DBΔCREATE 'EMP'
DBΔCHANGE 'EMP'
```

```
{sel} DBΔSELECT 'EMP:★'
```

```
{sel} DBΔVIEW 'EMP,DEPT:name,deptno,deptdesc,salary'
```

This keeps all the rats in one trap! the trouble with the SQL clone (apart from the need to put in the quotes) is that it has to keep remembering things in global variables (ΔROWΔSEL is updated by *WHERE*, and so on). If you are foolish enough to type '*name,salary*' *from* '*emp*' it never gets the chance to sweep up.

Below the surface, these functions do a great deal of work (much more than a standard relational model does). For example, where tables are associated (EMP:Deptno → DEPT) the knowledge of that link is maintained explicitly in the table definition. In fact there are two sorts of links, those which allow implicit deletion and those which do not. As an example, suppose I had:

Employee History → *Emp* → *Dept* → *Location* → *Country*

... if all the links are implicitly deletable, the removal of Switzerland would automatically eliminate Nestlé HQ, any departments which reported directly to it (e.g. Nestec, York), Debbie, and Debbie's salary history. If the link from Dept to Emp was marked 'non-deletable', the previous deletion would be bounced as long as there remained any employees in departments which reported to any location in Switzerland.

I think this is called 'referential integrity' in the jargon; whatever you call it, it is very useful, and saves an enormous amount of code!

Another advantage of remembering where the links go is that you can provide automatic 'pop-up' lists of valid options when the user can't remember the department names:

Update: Employee Table		Screen 1 of 2			
Ct	Grade Deptno	Sal	Commission		
.. 42/Accounts	-----	-----	-----		
.. 50/Sales	M-11 78/COBOL Stuff	4780.00	3456.00		
.. 20/Marketing	M-11 42/Accounts	6780.00	-----		
.. 30/APL Stuff	M-11 50/Sales	6780.00	-----		
.. 78/COBOL Stuff	M-11 77	6780.00	-----		
.. <Enter> select; Esc-quit	M-11 50/Sales	6780.00	-----		
.. 80 ___22 Jones B.A.	M-11 42/Accounts	6780.00	-----		
.. 90 ___24 Jones F.A.	M-12 42/Accounts	15005.00	100.00		
.. 100 ___25 Jones E.A.	M-12 42/Accounts	6780.00	-----		
.. 110 ___36 Zark	M-12 42/Accounts	6780.00	-----		
.. 120 ___37 Jones E.A.	D-14 42/Accounts	6780.00	-----		
.. 130 ___39 Jones C.A.	M-11 42/Accounts	6780.00	-----		
.. 140 ___48 Jones F.A.	M-11 42/Accounts	6780.00	-----		
.. 150 ___49 Hyde LM	M-11 42/Accounts	6780.00	-----		
.. 160 ___51 Kit Kat	S-8 20/Marketing	12345.00	-----		
.. 170 ___52 Jones C.A.	M-12 42/Accounts	6780.00	-----		
		6780.00	-----		
'ADR' in Ctl col - Add/Del/Rpt row.		F3:Save and Exit		PgUp	PgDn
F4:Locate F5:Sort F6:Remove dups F9:Table		F2:Save F2:Rollbk		←Ctl	Ctl→

It is also rather nice to allow a 'dive down' option, so that if you are halfway through an edit on the Employee list, and suddenly remember that the grade table needs fixing ...

Update: Employee Table

Screen 1 of 2

Update: Grade list

Ctl/Sq	Level	Grade	Desc
.. 10	C	6	Bum
.. 20	C	7	Smart Bum
.. 30	S	8	Good Type
.. 40	S	9	Fairly Sound
.. 50	S	10	Getting Known
.. 60	M	11	Spoken of in high places
.. 70	M	12	Flash Car
.. 80	M	13	Very Flash car
.. 90	D	14	Flash Secretary
.. 100	D	15	Runs the place
.. 110			
.. 120			

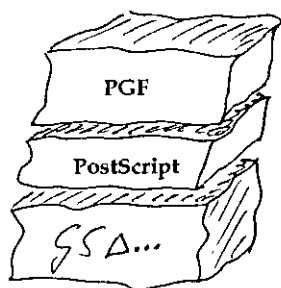
.. 160	51	Kit Kat	M-12	42	6780.00	6780.00
.. 170	52	Jones C.A.	M-11	42	6780.00	6780.00

'ADR' in Ctl col - Add/Del/Rpt row.				F3:Save and Exit	PgUp	PgDn
F4:Locate F5:Sort F6:Remove dups F9:Table				F2:Save F2:Rollbk	←Ctl	Ctl→

... you can drop down (i.e. follow the link to the next table in the chain), fix it, and exit back to where you were before. Of course (*this is APL after all*) this is nicely recursive, so you can go hunting down the links as far as you need, make the changes and back all the way out again.

Again, I should stress the value of keeping this Table Management layer separate from the physical database layer, and minimising the calls your application makes to the underlying tables ($\Delta \text{SALARY} > 0$) $\text{DB} \Delta \text{COMP} \Delta \text{EMP}$). This gives you the future option of pulling out one physical layer, and plugging in something completely different (APL*PLUS II users might like to try the Paradox engine, Dyalog users would probably go to Oracle) without any knock-on changes to the application.

The Business Graphics Layer (PGF/PostScript)



I have still to find a better set of business graphics primitives than the original GDDM/PGF package from IBM Hursley Park [4]. I am also a firm believer in PostScript as the best (device independent) way of describing a page [5]. Accordingly, my business graphics layer itself rests on a PostScript layer!

As a very brief example of the power of this graphics language:

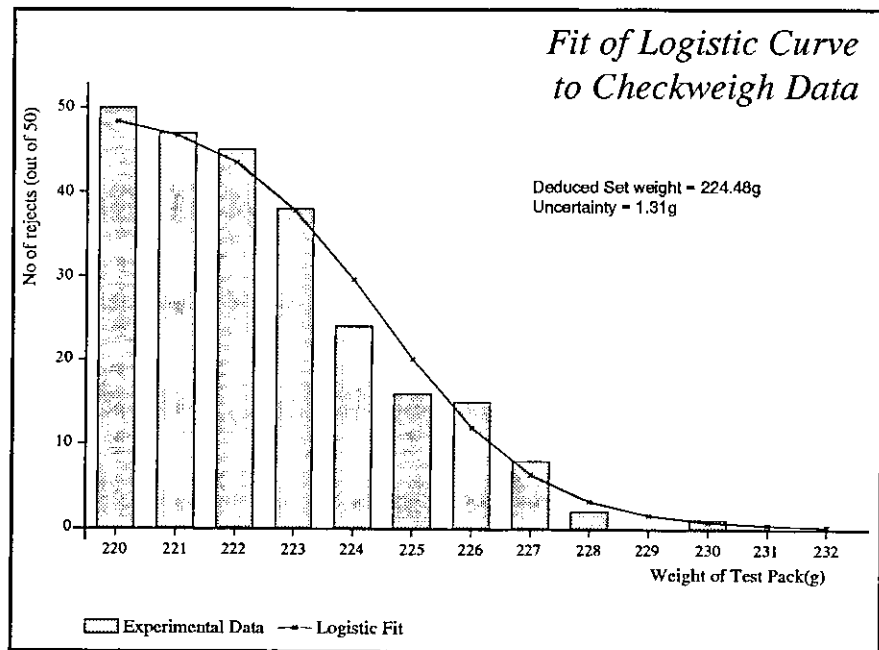
```

▽ R←CHW;PACK;EXP;FIT
[1]  A VECTOR example (Vector 8.4 page 78)
[2]  PACK←219+113
[3]  EXP← 50 47 45 38 24 16 15 8 2 0 1 0 0
[4]  FIT←50×÷1+*(PACK-224.48)÷1.31
[5]  CHSET 'CBAR,CBOX,HRIGHT'
[6]  'HMAR' CHSET 5 3
[7]  CHHEAD 'Fit of Logistic Curve;to Checkweigh Data'
[8]  CHXTTL 'Weight of Test Pack(g)'
[9]  CHYTTL 'No of rejects (out of 50)'
[10] CHXLAB 'I3' □FMT PACK
[11] CHKEY 'Experimental Data;Logistic Fit'
[12] CHBAR EXP ◊ CHPLOT FIT
[13] 8 40 CHNOTE 'Deduced Set = 224.48g;Uncertainty = 1.31g'
[14] CHTERM
[15] R←'PS PG A to see it'

```

▽

The resulting vector of PostScript code is a mere 2220 bytes long, and can very easily be wrapped around with a .EPS cocoon so that you can simply include it in documents like this ...



Or of course you can simply fire it straight to a PostScript printer. At least, with PostScript there is very little temptation to fiddle with the intermediate data (which looks a bit like this):

```

%!PS-Adobe-2.0 EPSF-1.2
%%Creator: APL-385
%%Title: Fit of Logistic Curve to Checkweigh Data
%%BoundingBox: 0 0 426 320
%%CreationDate: 1992 5 23 18 54
%%EndComments
/APLDict 50 dict def
APLDict begin

% Standard prologue for APL charting routines =====
% Version 1.3 Feb 1991
% Assumes b/w output device ... append chrgb.h for colour support!
/M {moveto} def
/L {lineto} def
/rL {rlineto} def
/patscale 1 def

```

```

/currentmark {} def % default marker is null
/currentpatn {} def % default pattern is no shading
/setcolour {pop} def % dummy for b/w devices ... see chrgb.h

/cshow % centre text round current point
{ dup stringwidth pop
  neg 2 div 0 rmoveto show
} def

....
....
....

14 8 4 settext
213.5 163.2 M
(Deduced Set weight = 224.48g) show
213.5 153.6 M
(Uncertainty = 1.31g) show

0 -51.4 M
12 2 16.2 6.3 bK
11 9 0 settext
( Experimental Data ) show
6 1 0.4 1 16.2 9 lK
11 9 0 settext
( Logistic Fit ) show

grestore

showpage

%%Trailer
end

```

Again, I should repeat (to the point of irritating repetition) the value of keeping the interpreter-specific code well and truly bottled up in utilities. I gave Alan Sykes a copy of my PostScript interpreter a couple of weeks ago, and within a week he had a pretty complete version running in Dyalog APL.

All he had to redo were half a dozen easily-understood functions (*GSΔDRAW*, *GSΔMARK* etc.); if I had embedded *ΔGLINE* in my interpreter, he would have had to understand (and then fix) *PL*, *STROKE*, *FILL*, *PB*, *SHOW*, *SETPATTERN* and several dozen more. Of course, Alan's version does shading patterns properly (has anyone out there ever made *ΔGSHADE* fill with anything other than solid colour?) and makes a rather better fist of the text than *ΔGWRITE*.

Wrapping it Up

If you are willing to live within the confines of someone else's imagination, just how far can you go towards that initial goal of an 80% recyclable workspace? Just for interest, I did a quick analysis of that planning board system, which comes out as follows:

Application-level stuff	97K	
High-level SQL stuff	35K	(packaged)
Low-level DB Δ stuff	6K	
Mouse support; MS Δ	3K	
FSMA screen handling	11K	
WIN Δ pop-ups etc	22K	
LM Δ library management	8K	
PGF level Graphics; CH Δ fns	22K	(packaged)
PS interpreter	19K	(packaged)
GSA drawing level	2K	

I reckon that this workspace is 56.89% recyclable! Most applications should come in higher than this; the problem with planning boards is that they have a lot of very application-specific code. This was compounded by the lack of any really effective report-generator (reporting functions tend to get horribly large), and by the need to trade re-usability for speed (the cursor repeat-rate is crucial ... if your code can't get around its loop in the time it takes the next keystroke to arrive ...!).

Yes, I can and will do better. The reporting issue will go away (who needs paper these days), and advances in interpreters will do away with all that $\square WPUT$ stuff and will maybe eventually let us treat everything as objects. Maybe one day we will all have Display PostScript, and I can finally throw out all my lower level? At least, if you build your systems this way, it doesn't hurt quite so much when you have to throw half your code away.

References

- [1] Taylor, Christopher, *Fields in the English Landscape*, Dent & Sons, 1975
- [2] APL Business Technology 83, Conference Proceedings
- [3] Smith, Adrian, *Fast Filing with $\Delta pack$* , Vector Vol.8 No.2 page 125
- [4] Smith, Adrian, *Getting the best out of GDDM*, Vector Vol.1 No.4 page 65
- [5] Smith, Adrian, *APL and PostScript Graphics*, Vector Vol.7 No.3 page 76

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

The]*HELP* Command

by Ray Cannon

This article could have been called "Useful information retrieval from structured documentation via user commands". It discusses some of the problems of providing intelligently selected information extracted from an on-line documentation system and a solution which has been partially implemented at a large mainframe site.

The Problem

Finding the right question you have to ask to get the answer you need. As an example, "What question do I ask if I wish to find out the name of the utility function used to convert a delimited text vector into a matrix?"

The Solution

Asking an imprecise question needs to produce a useful answer, or a set of more precise alternative questions that you might ask. For the above example, I could ask for help on *TEXT*, from its reply select the sub-topic *TEXT MANIPULATION*, and from its reply ask for help on *MAKEMAT*, *VTOM*, *BOX* or whatever.

The solution as implemented, involves User Commands to ask the imprecise question; a text data base (in the form of a well structured documentation system complete with relevant documents) to hold the answers; and a set of look-up methods to match them.

The Text Data Base

The key to data is its structure.

Let me call the data the Library. The library consists of Books (or alternatively Documents) and an Index to them. In the index, each book is cross-referenced by its Author; Title; Subject matter; Formal fixed reference; and Abstract.

Documents also have an internal structure. The document is divided into 3 sections and each section sub-divided into units.

The first section is the control section and it contains all the formal units such as the title and contents.

The **title page** contains a formatted copy of the information used in the library index.

The **contents page** contains a list of all the units in each section and controls the layout and ordering of them. Each unit is defined by 3 items, the unit number (which controls order and significance), the unit title (topic) and an external reference (a unique key).

The middle section contains the main text of the book split up into chapters which themselves can be divided up into smaller parts.

The last section is the appendices which can also be divided into smaller parts.

The parts are the units defined on the contents page.

A unit consists of one or more paragraphs. Parts of a unit may be tagged by markers for subsequent selection (a quote). The full address of some tagged text would be given by a 3 level reference, the Book reference (unique to the book within the library), the unit reference (unique within the book) and the tag reference (unique within the unit).

For example, the syntax of a user command *FOO* could be held in a book with a subject of *USER COMMANDS*, within a unit with a reference of *FOO* marked with the tag *SYNTAX*.

The Look-up Methods

In this context, we are trying to match a defined search string with a set of strings. Near misses may be useful. Perfect matches are the exception rather than the rule.

You need to be able to search for sub-strings, key-words, strings with wild-cards (pattern matching), and incomplete or incorrectly spelt strings (best match). A sub-string look-up must be able to find, for example, the sub-string *FMT* embedded within all the function names used as external references to the units documenting the functions. A key-word search must be able to find, for example, the word *PRINTER* from all the topics.

Pattern matching is normally used to show groups of results, for example a list of all commands starting with *ARCH*.

Lastly there is Best Match. This is the type of thing that spelling checkers use. However in this context, it needs to work with phrases not just individual words. For example, the selection text *PRINT* should match up with: *print*, *printing*, *printers*, *PRNT*, *PRT*, *]PRINT*, *Laser Printing*, *line-printer* etc., as used within topics.

The User Commands

User commands are required both to ask the final question and to ask the questions used to find out what the final question is. One command could be created to handle them all, but its syntax would probably become too difficult to use with ease. That is why I implemented several related commands (which all use the above look-up algorithms) to:

List selective document titles and references: *]DOCS selection*

List selective user commands and descriptions. *]CMDS selection*

List selective topics in a document. *]HELP topic-seln /LIST*

Return the content of a unit or quote from its topic. *]HELP topic*

Return the document or part thereof, from its formal references. *]DOCTEXT*

Display help from a specific command. *]cmd /HELP*

Return help on a specific command. *]HELP]cmd*

Notes on User Command Syntax as Implemented at this Site.

To simplify the use of user commands, they have all been written to use a standard syntax as follows:

]<cmd-name> <parameter string> <key word options>

Command names are standard APL-type names and are terminated by a space.

Parameter strings may be enclosed in quotes (normal APL rules about quotes in quotes apply). They may not contain the */* character unless in quotes.

Key-word options all start with a */*. They can take two forms: flags, for example */PRINT*; and assignments, for example */MSG=OFF*.

Certain key-word options are standard and apply to almost all user commands. These are:

<i>/HELP</i>	Display help about the user command and abort the command
<i>/?</i>	Display the command syntax and abort the command
<i>/PRT</i>	Print the results of the command on your printer
<i>/PRT=node</i>	Print the results of the command on printer <i>node</i>
<i>/PW=nn</i>	Format the result using a $\square PW$ of <i>nn</i>
<i>/MSG=</i>	Can be <i>OFF</i> or <i>RES</i> (return as part of the result)
<i>/RES=</i>	Used to redirect the result into a named APL variable

Like *)SYSTEM COMMANDS*, *]USER COMMANDS* cannot be called directly from within functions. However, the system function $\square UCMD$ can be used to action a user command.

User commands can display messages, return results and action processes. For example, a user command might be asked to read 3 functions from file. The message produced might be that only 2 functions were found, the result could be the names of the functions found, and the action would be the reading and fixing in the workspace of the functions found. Alternatively, the message might be that the command failed as the file system was down, the result was empty, and no action was performed.

Note that *]HELP* returns, by default, help on the command *]HELP* as does *]HELP]HELP*. On the other hand *]HELP /HELP* requests help on how to use the user command *]HELP*, as does *]HELP]HELP /HELP*. Alternatively, *]HELP HELP /DOC=HELP* would return the unit whose topic was called (or contained the best match to the word) *HELP* from the document with the name *HELP*. By coincidence, the text displayed or returned in these cases is all the same.

So what is "The ultimate imprecise question"?

Answer: *]HELP*

APL

Complete APL system for PC386 or PC486. Usable with or without WINDOWS; Compatible with ISO APL and SHARP APL; Arrays of arrays; Component and native files; Editor; Help; On-line documentation; Large workspaces in extended and virtual memory; APL printer support; BGI graphics; and more.

PC version also available.

Software \$30; Documentation package \$30.

Iverson Software Inc

33 Major St. Toronto, Ontario, Canada M5S 2K9
Phone: 416 925 6096 Fax: 416 488 7559

Authorized Dealer

I-APL, 11 Auburn Road, Redland, Bristol BS6 6L7

J is Ken's new APL. Ideal for teaching and research. Uses ASCII characters only, and runs on a wide range of machines, with workspaces compatible across machines.

J shareware and documentation \$24; Shareware source (in C) with documentation \$90

Texts: *Tangible Math* (34 pp) \$12; *Programming in J* (76 pp) \$15; *Arithmetic* (123 pp) \$18.

J

Making a Calendar in J

by Keith Smillie

The Gregorian calendar was introduced in 1582 by Pope Gregory XIII to bring the date of Easter closer to the vernal equinox since the error in the mean year length in the Julian calendar introduced in 45 BC was causing Easter to gradually move towards the summer. In the Gregorian calendar leap years were still those years whose numbers were divisible by 4 except for centennial years where divisibility by 400 was also required. To compensate for the cumulative error in the Julian calendar, when the Gregorian calendar was adopted Thursday, October 4, 1582 was followed by Friday, October 15 resulting in a loss of ten calendar days.

A program in J is developed for calculating and displaying in any one of several formats the Gregorian calendar for any year after 1600. The sequence of verbs in J is compared with the APL functions in McDonnell (1979) which motivated the present paper.

Preliminaries

Since leap year is determined by the divisibility of the year number by the integers 4, 100 and 400, it will be useful to define verbs for finding the integer quotients and integer remainders with these divisors. Since the monad `% 4 100 400` gives the quotients when its argument is divided by 4, 100 and 400, the verb

```
div=. <. @ % 4 100 400
```

will give the integer quotients, and, for example, `div 1992` is `498 19 4`. Similarly, the verb

```
mod=. 4 100 400 & |
```

gives the integer remainders, and `mod 1992` is `0 92 392`.

We shall also use the verbs

```
dayno=. 7 & |
```

in the calculation of the day of the week since the days are numbered from 0 to 6, inclusive, and

```
pi=. >: @ i.
```

to give the positive integers, so that, for example, $p1 \ 4$ is the list 1 2 3 4 .

Finally we shall define the constants

```
DAYS=. ' Su Mo Tu We Th Fr Sa'
```

for the day names,

```
MONTHS=. ' JanFebMarAprMayJunJulAugSepOctNovDec'
```

for the month names, and

```
LENGTHS=. 31 28 31 30 31 30 31 31 30 31 30 31
```

for the lengths of the months where the leap year adjustment for February will be made later.

Leap Year and its Consequences

To find the calendar for any month of any year we shall define a New Year's date for any year as the sequence number for January 1 of that year beginning with Monday, January 1, 1601 as 1, and an ordinal date for the first day of any month as the number of days from January 1 of the year. For example, January 1, 1992 has a New Year's date of 142810 and an ordinal date of 1, and May 1, 1992 has an ordinal date of 121 which is the sum of the lengths of the months previous to May. If we number the days of the week from 0 to 6, inclusive, with Sunday as 0, then the number of the day of the week for any date is the 7-residue of the sum of the New Year's date for the year and the ordinal date for the month. For example, May 1, 1992 falls on a Friday since

```
dayno 142810 + 121
```

where the verb `dayno` defined in the previous section gives the 7-residue, is equal to 5.

The calculation of the New Year's and ordinal dates requires a test for leap year. A simple test follows from the observation that a year is a leap year if its number is evenly divisible by either one or three of the divisors 4, 100 and 400, and is given by the verb

```
ly=. 0&(~:/ . =) @ mod
```

where the verb `mod` defined in the previous section gives the remainders and the monad `0&(~:/ . =)` provides a parity check on the number of zero remainders.

For example, 1y 1992 is 1 and each of 1y 1991 and 1y 1990 is 0. Given this leap year test we may define the monadic verb

```
LENGTHSadj=. LENGTHS&+@((LENGTHS=28)&(*. 1y))@]
```

which gives a list of the month lengths with the length of February adjusted for leap year. We may also define the dyadic verb

```
m1=. {LENGTHSadj
```

which gives the length of the month given as the left argument where 0 represents January, 1 represents February, etc. and the right argument gives the year number.

The New Year's date for any year is the number of days from January 1, 1600 to January 1 of the given year, and is one more than 365 times the number of years plus the number of leap years in this interval. This latter number may be found as the integer quotient of the year number minus 1600 divided by 4, less the integer quotient of this same difference divided by 100 to adjust for all centennial years, plus the difference divided by 400 to account for leap years on every fourth centennial year. Thus the number of leap years is the alternating sum of the appropriate quotients. Therefore, the New Year's date is given by the monadic verb

```
nydate=. >:@(365&* + -/@div)@([@-1601) .
```

The ordinal date for any month, the sum of the lengths of the months preceding the given month, is given by

```
odate=. +/ @ ({.LENGTHSadj)
```

where the left argument gives the month number in 0-origin indexing and the right argument gives the year number. Finally, the negative of the day number of the first day of any month of any year is

```
mb=. - @ dayno @ (odate + nydate@])
```

where the left and right arguments give the month number and year number, respectively.

Now the numerical part of a month calendar consists of the consecutive integers from 1 to an upper limit determined by the month length arranged in a table of six rows since any month except February may extend over six consecutive weeks, and seven columns since there are seven days in the week, and beginning in the first row in a column determined by the first day of the month. This array

may be constructed simply by first generating the appropriate list of positive integers as determined by the month length, converting to a one-column table, formatting to a three-column character table, extending this table to forty-two rows, rotating the rows by an amount given by the day number of the first day of the month, and finally reshaping into the appropriate table. These operations are performed by the verb

```
mctable=. , "_1@(6 7&$)@
(mb !.(42&{.)@({3&":)@,.@pi@ml)
```

where the left and right arguments give the month and year numbers.

Assembling the Calendar

To obtain the desired year calendar the twelve month calendars must be constructed each with column headings for the names of the days of the week and headed overall with the month name, and finally displayed in one of several possible rectangular arrangements. Thus

```
mc=. {mn@[], (DAYS&,@mctable) ,
```

where

```
mn=. _12&{.@]@{&(_3)\MONTHS)
```

centres the month name, gives a labelled month calendar. The dyadic verb

```
cal=. (i.@[]mc"0)
```

will give month calendars as determined by the left argument for the year given by the right argument. For example, `3 4 cal 1992` gives a 1992 calendar arranged in three rows and four columns as the 2-cells of a table of shape `3 4 8 21`, and `3 cal 1992` gives the first three months of the 1992 calendar as the 2-cells of a table of shape `3 8 21`. Finally,

```
calendar=. <"2@(4 3&cal)
```

gives a four-by-three calendar for the year specified by the right argument, and has been used to construct the 1992 calendar given in Figure 1.

Figure 2 gives the script file containing all of the verbs discussed in this paper.

Comparison with APL functions

The method used to calculate the Gregorian calendar in this paper is similar to that given in APL by Eugene McDonnell. The test for leap year using a dot product for a parity check and the calculation of the number of leap years using an alternating sum are due to McDonnell. Zero suppression, which McDonnell said might give a "spot of trouble" in APL, has been handled very simply in J. The main difference between the two methods lies in the approach to constructing the calendars for all months of the year. The APL functions required what may be termed a global approach where all month calendars were constructed at the same time and then reshaped into the appropriate array.

With J the approach is local with attention being given to the construction of a typical month calendar and then extending the calculations to all months of the year using the rank conjunction. A similar approach was taken in calendar calculations done by the author in Nial where the transformer EACH was used. Indeed, the defined conjunction `each=. &.>` was used in an earlier version of the J verbs. We might note in conclusion that Trenchard More has compared the local and global approaches to computation by remarking that "Data are simple when viewed locally."

Acknowledgement

I would like to thank Eugene McDonnell for bringing to my attention simplifications in some of the verbs used in an earlier version of this paper. His suggestions included the use of the conjunction rank in place of the defined adverb each, and a nice use of the verb take for zero suppression in place of an inelegant defined verb that was originally used for formatting.

References

Iverson, K. E., 1992. An Introduction to J. Iverson Software Inc., Toronto.

McDonnell, E. E., 1979. "Making a calendar." APL Quote Quad, vol. 10, no. 1, pp. 23 - 26.

Jan							Feb							Mar						
Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa
			1	2	3	4							1	1	2	3	4	5	6	7
5	6	7	8	9	10	11	2	3	4	5	6	7	8	8	9	10	11	12	13	14
12	13	14	15	16	17	18	9	10	11	12	13	14	15	15	16	17	18	19	20	21
19	20	21	22	23	24	25	16	17	18	19	20	21	22	22	23	24	25	26	27	28
26	27	28	29	30	31		23	24	25	26	27	28	29	29	30	31				
Apr							May							Jun						
Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa
			1	2	3	4							1		1	2	3	4	5	6
5	6	7	8	9	10	11	3	4	5	6	7	8	9	7	8	9	10	11	12	13
12	13	14	15	16	17	18	10	11	12	13	14	15	16	14	15	16	17	18	19	20
19	20	21	22	23	24	25	17	18	19	20	21	22	23	21	22	23	24	25	26	27
26	27	28	29	30			24	25	26	27	28	29	30	28	29	30				
							31													
Jul							Aug							Sep						
Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa
			1	2	3	4							1			1	2	3	4	5
5	6	7	8	9	10	11	2	3	4	5	6	7	8	6	7	8	9	10	11	12
12	13	14	15	16	17	18	9	10	11	12	13	14	15	13	14	15	16	17	18	19
19	20	21	22	23	24	25	16	17	18	19	20	21	22	20	21	22	23	24	25	26
26	27	28	29	30	31		23	24	25	26	27	28	29	27	28	29	30			
							30	31												
Oct							Nov							Dec						
Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa
				1	2	3	1	2	3	4	5	6	7			1	2	3	4	5
4	5	6	7	8	9	10	8	9	10	11	12	13	14	6	7	8	9	10	11	12
11	12	13	14	15	16	17	15	16	17	18	19	20	21	13	14	15	16	17	18	19
18	19	20	21	22	23	24	22	23	24	25	26	27	28	20	21	22	23	24	25	26
25	26	27	28	29	30	31	29	30						27	28	29	30	31		

Figure 1. Calendar for 1992.

NB. Utilities

```
pi=. >: @ 1.
div=. <. @ % & 4 100 400
mod=. 4 100 400 & |
dayno=. 7&|
```

NB. Constants

```
DAYS=. ' Su Mo Tu We Th Fr Sa'
MONTHS=. 'JanFebMarAprMayJunJulAugSepOctNovDec'
LENGTHS=. 31 28 31 30 31 30 31 31 30 31 30 31
```

NB. Month name centered

```
mn=. _12&{. @ ] @ {&{ _3 ] \ MONTHS}
```

NB. Leap year test

```
ly=. 0&{~:/ . =) @ mod
```

NB. Month lengths adjusted for leap year

```
LENGTHSadj=. LENGTHS&+@((LENGTHS=28)&{*. ly))@]
```

NB. Month length

```
ml=. {LENGTHSadj
```

NB. New Year's date

```
nydate=. >:@(365&* + -/@div)@([@-&1601)
```

NB. Ordinal date

```
odate=. +/ @ ({.LENGTHSadj)
```

NB. First day of month

```
mb=. -@dayno @ (odate + nydate@])
```

NB. Month calendar without headings

```
mctable=. , "_1@{6 7&$}@{mb |. (42&{.)@{3&":})@,.@pi@ml)
```

NB. Month calendar

```
mc=. (mn@[ ], (DAYS&,@mctable)
```

NB. Calendar for given number of months

```
cal=. (i.@[ ]mc"0]
```

NB. Calendar with 4 rows & 3 columns

```
calendar=. <"2@{4 3&cal)
```

Figure 2. Script file.

Using J's Boxed Arrays

by Donald B. McIntyre

Introduction

APL, which has been an implemented computer language for more than 25 years, grew out of the concise mathematical notation devised by Kenneth Iverson. J is his powerful new dialect [1]. He has defined it in a Dictionary that is an essential reference [2]. Iverson has extended the language in very interesting ways. He has also rationalised features which, benefiting from so many years of hindsight, he recognised as anomalies in other versions of APL.

Users accustomed to older APL can therefore expect to be puzzled by some changes of behaviour. For example, the sum over a table (+/table) formerly gave **row** totals (summing over the last axis), but the same expression in J gives the **column** totals (summing over the first axis). The reason is that the table is considered an array of **items**, an item being a row. The number of items is given by the first number in the shape; a 4 by 12 table (matrix) is looked on as a collection of 4 lists (vectors), each of length 12. Consequently the mean (the sum divided by the tally) is the sum down the columns (across the items) divided by the number of rows.

Another anomaly was the use of square brackets and the semicolon in indexing. Do expressions such as `V[I]` or `M[I;J]` qualify as verbs (functions), and if not then what are they? In J selections like these are pure functions: e.g. `1 { v` (read this "i from v"). In higher rank objects, which need pairs (or triplets, etc.) of indices to identify one atom in the array, the syntax is the same, but the indices of the pair (or triplet, etc.) are boxed together. How that is done will be seen in the following examples.

Thinking that my experience of using J might help others, I have given accounts of various features with which I had difficulty when I began [4-7]. This is a further example. One of my first attempts in using J was to use boxes to create a small database. When I consulted Ken Iverson, Eugene McDonnell happened to be visiting Toronto, and he responded to my questions in a letter so helpful that, with his permission, I included it in my paper for APL91 at Stanford [4].

Basic Techniques Relating to Boxes:

APL's index generator (`iota`) has been extended:

```

]a=. i.2 3
0 1 2
3 4 5

```

After being boxed, this array of 2 rows, each of length 3, can be treated as an atom (scalar):

```

$a
2 3
]b=. <i.2 3

```

0	1	2
3	4	5

```

$b

```

This illustrates the use of the verb `box` (`<`). Ignoring special cases, we can say that the dyadic verb `link` (`:`) boxes its arguments:

```

]c=. 'mary';'jones';a; 1 3 5

```

mary	jones	0 1 2	1 3 5
		3 4 5	

We now have a list of 4 items, from which we can take a selection or a permutation:

```

$c
4
3 1 2 0{c

```

1 3 5	jones	0 1 2	mary
		3 4 5	

The individual words in a character string can be boxed by the monadic verb `word formation` (`;;`)

```

;;s=. 'here we go gathering nuts in may'

```

here	we	go	gathering	nuts	in	may
------	----	----	-----------	------	----	-----

The length of each word is determined by the tally under open; that is, each box is opened, the tally is taken, and the box is closed again. under (&.) is a conjunction combining here the verbs tally (#) and open (>).

```
#&.> ;: s
```

4	2	2	9	4	2	3
---	---	---	---	---	---	---

The mean is a fork [4-7]. We find the average word-length by applying the verb mean after opening the boxes of word lengths.

```
mean=. +/%#
mean > #&.> ;: s
3.71429
```

Or, defining the verb mw1 we get the mean word-length of any string. This is a tacit definition, a pure functional form, in which there is no explicit reference to arguments.

```
mw1=. mean @ (>@(#&.> @ ;:))
mw1 s
3.71429
```

When we open a list (vector) of boxes we get a table (matrix):

```
s=. 'here we go gathering nuts in may'
s> ;:s
7 9
> ;:s
here
we
go
gathering
nuts
in
may
```

The table is alphabetised by the dyadic sort (/:). This is a remarkably concise statement!

```
/:~> ;: s
gathering
go
here
in
may
nuts
we
```

Should we want to box numbers, we must distinguish between boxing the list as a whole and boxing the individual items in it. The rank conjunction (") instructs the box verb to enclose rank-0 elements:

```
v=. 2 3.4 5.67
<"0 v
```

2	3.4	5.67
---	-----	------

The items of the list (individual numbers) are made into vectors (from rank-0 to rank-1) by `ravel item (.)`

```
$,. v
3 1
,. v
2
3.4
5.67
```

A rank-3 array can be boxed with any rank from 0 to 3:

```
]m=. 1.2 3 4
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
16 17 18 19
20 21 22 23
```

```
<"2 m
```

0 1 2 3	12 13 14 15
4 5 6 7	16 17 18 19
8 9 10 11	20 21 22 23

```
<"1 m
```

0 1 2 3	4 5 6 7	8 9 10 11
12 13 14 15	16 17 18 19	20 21 22 23

Construct a Simple Database:

Use "link" (:) to create an array of boxed elements:

```
d=.,:'E.E.':'McDonnell':'Palo Alto';27;10000; 8 3 12 25 10
d=.d,'Ken':'Iverson':'Toronto';55;15000; 4 19 32 1 15 10
d=.d,'Donald':'McIntyre':'U.K.';61;12000;'
d=.d,'Roger':'Hui':'Toronto';49;20000; 32 4
d=.d,'Anthony':'Camacho':'U.K.';45;35000; 19 23 45 4 17 13 5
d
```

E.E.	McDonnell	Palo Alto	27	10000	8 3 12 25 10
Ken	Iverson	Toronto	55	15000	4 19 32 1 15 10
Donald	McIntyre	U.K.	61	12000	
Roger	Hui	Toronto	49	20000	32 4
Anthony	Camacho	U.K.	45	35000	19 23 45 4 17 13 5

The columns can be taken as **First Name**, **Family Name**, **Location**, **Age**, and **Salary**. The final column illustrates a ragged array (with one empty element). The content of the database is entirely fictitious.

To manipulate the database we need to have a convenient way of extracting columns:

```
col=. >@{"1
locality=. 2&col
locality d
Palo Alto
Toronto
U.K.
Toronto
U.K.
```

Or we can use pronouns:

```
name=. 1 [ age=. 3 [ salary=. 4
```

Because `x /: y` is the same as `(/:y) { x` "x is sorted into an order specified by y" [2]:

```
sort=. ] /: col          NB. Fork
```

It is important to remember that `right` (]) and `left` ([) are verbs and not merely placeholders; they take arguments and produce results.

name sort d

Anthony	Camacho	U.K.	45	35000	19 23 45 4 17 13 5
Roger	Hui	Toronto	49	20000	32 4
Ken	Iverson	Toronto	55	15000	4 19 32 1 15 10
E.E.	McDonnell	Palo Alto	27	10000	8 3 12 25 10
Donald	McIntyre	U.K.	61	12000	

salary sort d

E.E.	McDonnell	Palo Alto	27	10000	8 3 12 25 10
Donald	McIntyre	U.K.	61	12000	
Ken	Iverson	Toronto	55	15000	4 19 32 1 15 10
Roger	Hui	Toronto	49	20000	32 4
Anthony	Camacho	U.K.	45	35000	19 23 45 4 17 13 5

To sort on the basis of the ragged column (5), we might use the mean:

```
mean=. +/%#
mean age col d
47.4
mean salary col d
18400
```

Defining the adverb each (using under):

```
each=. &.>
meanr=. mean each @ (5&{"1)      NB. Means in ragged column
5
meanr d
```

11.6	13.5	0	18	18
------	------	---	----	----

Sorting on the basis of the means in the ragged column, and appending the means:

```
sortr=. ] /: > @ meanr
sortr d,"1 0 meanr d
```

Donald	McIntyre	U.K.	61	12000		0
E.E.	McDonnell	Palo Alto	27	10000	8 3 12 25 10	11.6
Ken	Iverson	Toronto	55	15000	4 19 32 1 15 10	13.5
Roger	Hui	Toronto	49	20000	32 4	18
Anthony	Camacho	U.K.	45	35000	19 23 45 4 17 13 5	18

The catenation (,"1 0) is performed so that rank-1 cells (rows) on the left are catenated to rank-0 cells (atoms) on the right. It is worth experimenting to see that J is capable of catenating the list of means to each row of the table, but the result is not reproduced here.

To display the subset of employees at a given locality, apply the adverb each to the verb match (-:). Because the data are boxed, the search-key must be boxed too. Copy (the dyadic #) does the same as compress or replicate in older APL. The adverb cross (~) interchanges the arguments, thus cutting down on parentheses:

```
d #~ >(<'Toronto') -: each 2("1 d
```

Ken	Iverson	Toronto	55	15000	4 19 32 1 15 10
Roger	Hui	Toronto	49	20000	32 4

We can define a verb that makes such selections easier:

```
place=. ] #~ > @ (< @ [ -: each 2&("1 @ ])
```

```
'U.K.' place d
```

Donald	McIntyre	U.K.	61	12000	
Anthony	Camacho	U.K.	45	35000	19 23 45 4 17 13 5

Replacement and Insertion:

This is done using the adverb amend, formerly "merge" (}). Three pieces of information are required: the array to be changed; the indices identifying the affected cells; and the new data to be inserted. Both the indices and the new data must be boxed.

Thus to place an array into the cell at row-2 column-5:

```
x=. 9 8 7 6,:5 4 3 2
(<x) (<2 5)} d
```

E.E.	McDonnell	Palo Alto	27	10000	8 3 12 25 10
Ken	Iverson	Toronto	55	15000	4 19 32 1 15 10
Donald	McIntyre	U.K.	61	12000	9 8 7 6 5 4 3 2
Roger	Hui	Toronto	49	20000	32 4
Anthony	Camacho	U.K.	45	35000	19 23 45 4 17 13 5

Scattered indexing presents no problem:

```
(x;'London') (2 5;4 2)} d
```

E.E.	McDonnell	Palo Alto	27	10000	8 3 12 25 10
Ken	Iverson	Toronto	55	15000	4 19 32 1 15 10
Donald	McIntyre	U.K.	61	12000	9 8 7 6 5 4 3 2
Roger	Hui	Toronto	49	20000	32 4
Anthony	Camacho	London	45	35000	19 23 45 4 17 13 5

Expansion in General, and a Digression on Reading J:

To insert new items (new rows) the array must be expanded. In older APL dialects the back-slash (\) was used along with a boolean string (U), as in `U\0]D` to expand along the first axis. J uses the back-slash for other operations, but Iverson has defined a verb that performs the operation of expansion [4]:

```
exp=. /:@\:@[#{#@[{.}
1 0 1 exp 7 8
7 0 8
```

To someone who does not know the notation, this expression is, by definition, incomprehensible; i.e. it cannot be read with understanding. This is, of course, no reason for concluding that it is difficult to read.

The first step towards understanding any sentence is to parse it. In J any character immediately followed by a period (.) or a colon (:) is to be treated as a

2-character symbol. Make this clear by inserting spaces and we see a sequence of 11 symbols:

```
exp=. /: @ \: @ [ { # @ [ {. ]
```

The square brackets (`[`) and (`]`) are not symbols of punctuation (as round parentheses are), but verbs. They return their **left** and **right** arguments respectively. Now identify **conjunctions** (dyadic operators) — in this case only **atop** (`@`) — and insert parentheses to show that a conjunction combines the item (verb or noun) on either side of it to compose a new verb. Put the right parentheses in first:

```
exp=. /: @\:) @ [ ] { # @[ ] {. ]
```

Because conjunctions have long left-scope and short right-scope, put in the left parentheses while working from left to right:

```
exp=. ((/:@\:)@[ ] { (#[ ] {. ]
```

Confirm that this gives the same result as before:

```
1 0 1 exp 7 8
7 0 8
```

Because the parentheses present enclose (composed) verbs, we can view the definition as a sequence of 5 verbs, symbolically:

```
exp=. p q r s t
```

Such a sequence of verbs would have no meaning in ordinary APL. Iverson calls it a **train**: “an isolated sequence of parts of speech that does not resolve to a shorter sequence through the normal application of verbs” [2, p.5]. He assigns meanings to trains of two verbs (**hooks**) and three verbs (**forks**), and by repeated resolution to any train of verbs [2, 8, 9]. Thus

```
(s t)          NB. Hook
```

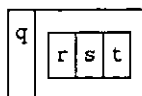
s	t
---	---

```
(r s t)        NB. Fork
```

r	s	t
---	---	---

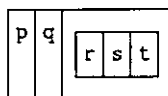
(q r s t)

NB. Hook containing a Fork



(p q r s t)

NB. Fork containing a Fork



Thus, if we define the following:

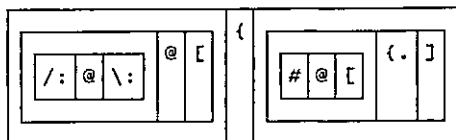
```
p=. /:@\:@[
q=. {
r=. #@[
s=. {.
t=. ]
```

then:

```
1 0 1 (p q r s t) 7 8
7 0 8
```

The J system provides two important tools for investigating parsing. The directly displayed form shows clearly the fork within the fork.

exp

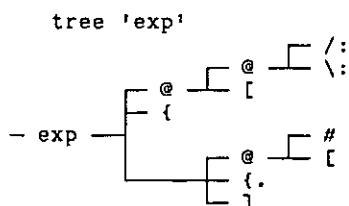


The conjunction (5!:4) displays a tree representation. If the verb `tree` is defined in the profile, it is always conveniently available:

```
tree=. 5!:4 @ <
f=. p q r s t
tree 'f'
```

```

      p
      q
- f ---
      r
      s
      t
```



In reading the tree, remember that atop (@) is a conjunction that makes verbs; hooks and forks are trains only of verbs.

Returning to the original form without parentheses:

```
exp=. /: @ \: @ [ { # @ [ { . ]
```

In Iverson's words: "Notice how this tacit definition reads in English. The inverse of the downgrade of the left argument permutes the (over)take of the right argument by the number of items of the left argument" [4, p.270].

Had an adverb (a monadic operator) been present, we would insert parentheses to show that it modified the verb to its left. This is shown by the verb `meanr`, defined above to compute the means of the ragged items in column 5:

```
meanr=. mean each @ (5&{"1)
```

Note first that parentheses are required to prevent the atop (@) from grabbing the 5 to its right. Next, the conjunction with (&) takes the 5 along with the verb from (t) to compose a new verb. The rank conjunction (") takes this composed verb (including the 5 that is part of it) and combines it with the noun (1) to its right. The result (5&{"1) is a verb that returns the fifth column of the array to which it applies.

`each` is a defined adverb, and, like any other adverb, it modifies the verb to its left (`mean`) forming a new verb (`mean each`). It is this verb that is the left argument of the atop.

Had we defined `meanr`:

```
meanr=. +/%# each @ (5&{"1)
```

then `each` would have modified `tally` (#) alone, and the definition would have been parsed as a fork, which would have been quite wrong:

```
(+ /) % (# each @ (5&{"1))
```

Adding New Items to the Database:

First expand the array:

```
] e=. 1 0 1 1 0 1 1 exp d
```

E.E.	McDonnell	Palo Alto	27	10000	8 3 12 25 10
Ken	Iverson	Toronto	55	15000	4 19 32 1 15 10
Donald	McIntyre	U.K.	61	12000	
Roger	Hui	Toronto	49	20000	32 4
Anthony	Camacho	U.K.	45	35000	19 23 45 4 17 13 5

Then amend by inserting the new items one at time:

```
e=. e 1&{ }~ 'Bob';'Bernecky';'Toronto';35;22000; 4 5
]e=. e 4&{ }~ 'Graham';'Woyka';'U.K.';62;35000; 14 31 5 7
```

E.E.	McDonnell	Palo Alto	27	10000	8 3 12 25 10
Bob	Bernecky	Toronto	35	22000	4 5
Ken	Iverson	Toronto	55	15000	4 19 32 1 15 10
Donald	McIntyre	U.K.	61	12000	
Graham	Woyka	U.K.	62	35000	14 31 5 7
Roger	Hui	Toronto	49	20000	32 4
Anthony	Camacho	U.K.	45	35000	19 23 45 4 17 13 5

Or define another array, from which to amend the original by merging several items in one step:

```
x=. ('Graham';'Woyka';'U.K.';1;2;3),:'Vin';'Grannell';'Los Angeles';4;5;6 7 8
```


Acknowledgements:

Kenneth Iverson and Roger Hui provided the language and the system. I am particularly indebted to Kenneth Iverson and Eugene McDonnell for sharing their trenchant insight at crucial points in my experience with J.

References

- [1] J is available from Iverson Software Inc., 33 Major Street, Toronto, Ontario, Canada M5S 2K9. Phone (416) 925-6096; Fax (416) 488-7559. Also from I-APL Ltd in the UK (see Product Guide).
- [2] Kenneth E. Iverson, *ISI Dictionary of J*, Version 3.3 with Tutorials, Iverson Software Inc., Toronto (1991) 32pp.
- [3] Kenneth E. Iverson, *Arithmetic*, Iverson Software Inc., Toronto (1991) 119pp.
- [4] Donald B. McIntyre, *Mastering J*, APL91 Conference Proceedings, Stanford, California, August 1991. APL Quote Quad Vol. 21 Number 4 (August 1991), pp.264-273. This paper includes valuable statements by K.E. Iverson and E.E. McDonnell, for which I am very grateful.
- [5] Donald B. McIntyre, *Hooks and Forks and the Teaching of Elementary Arithmetic*, Vector, Vol.8 No.3 pp.101-123 (1992)
- [6] Donald B. McIntyre, *Using J with External Data: two examples*, Vector, Vol.8 No.4 pp.97-110 (1992)
- [7] Donald B. McIntyre, *Language as an Intellectual Tool: From hieroglyphics to APL*, IBM Systems Journal, Vol. 30, Number 4 (1991).
- [8] Kenneth E. Iverson, and E.E. McDonnell, *Phrasal Forms*, APL89 Conference Proceedings, New York City, August 1989. QuoteQuad, Volume 19, Number 4, (1989) pp.197-199.
- [9] Roger K.W. Hui, Kenneth E. Iverson, Eugene E. McDonnell, *Tacit Definition*, APL91 Conference Proceedings, Stanford, California, August 1991. APL Quote Quad Vol. 21 Number 4 (August 1991), pp.264-273.



RENAISSANCE DATA SYSTEMS

Enlightenment Thru Information Processing

Specializing in Books and Software on APL, J
and other Curiosities of merit.

HAVE YOU GIVEN A COPY OF IAPL OR J TO A TEACHER OR STUDENT?

Name: _____ Date: _____

Street: _____

City: _____ State: _____ Postal Code _____

Country: _____ Telephone _____

For a copy of our most recent catalog,
please send a self-addressed, legal-
sized envelope (stamped if from U.S.)
or mail this form to:

Renaissance Data Systems
Park West Finance Station
P. O. Box 20023
New York, New York 10025-1510

Learn APL

APL Shareware/Demos and Related Publications

APL and Mathematics

APL AS A TOOL OF THOUGHT Proceedings

Other Important Sources of APL Information.

APL History

APL References and Techniques

Special Subjects In APL

APL Interpreters and Software

J=: A Powerful Dialect of APL

Other Software for Use with APL

To Russla with Love and for APL92

Other Curiosities of Merit

*I-APL: An International voluntary project of individuals and
companies seeking to share their love of APL.*

THE RANDOM VECTOR

The Newsletter of the APL Statistics Library
Editor David Eastwood

July 1992

Contents

Editorial	David Eastwood	107
Nonlinear Regression	Peter Hingley	109

Editorial

This issue of Vector coincides with the annual international APL conference, which this year is being held in St Petersburg, and our regular readership will be increased to include delegates to APL92 who are being given copies of Vector. Alan Sykes, the ASL Deputy Chairman, will be one of the delegates from the UK and will be giving a presentation about ASL at the conference. Alan will be basing his presentation on areas of application for the ASL software, in Education, Research and Commerce and will be discussing some of the future plans for ASL. We will seek to print his presentation in a future issue but in the meantime I hope that it proves an interesting introduction to the ASL software for delegates at APL92 and in particular our Russian colleagues.

We have, at long last, begun distributing copies of ASL software and by the time this issue of Vector appears we will have added the Basic Statistics shelf to our release set. Order forms are available from the BAA or from members of the ASL Management Committee. It has been interesting to note that a substantial proportion of our initial 'orders' for ASL software has been from overseas APLers and we hope to encourage our overseas colleagues to contribute to ASL both in terms of additions to those shelves we have released and also by contributing to new shelves for ASL. Of necessity the initial ASL offerings have come from a relatively small nucleus of UK-based statisticians. Our conference last autumn included contributions from a number of new faces, and I shall be publishing some of their work in Random Vector, starting with Peter Hingley's article in this issue. At that conference, ideas for a number of new shelves for ASL were discussed (see Vector 8.3, January 1992), and as the first phase of the ASL project is completed we are keen to turn some of these suggestions for new shelves into tangible offerings.

At the time of writing this copy, the ASL Management Committee has not settled on a venue for the next ASL conference, which means that we will probably miss the traditional autumn slot we have booked for our conferences in years gone by. We would be interested in canvassing views from readers as to whether we should alter the format of the ASL conference from a three-day residential conference to a one-day conference (say in London), thereby possibly allowing a wider attendance.

As a final point I would like to record our thanks to John Searle for the work he has put into the ASL project over the last few years. John is being posted overseas by his employer and has had to stand down from both the BAA Committee and the ASL team. Pending the results of the current BAA Committee elections, we would envisage the incoming Projects Officer of the BAA joining the ASL Management Committee.

As usual, I give contact addresses for the ASL Management Committee:

Chairman: Jake Ansell, Business Studies, University of Edinburgh
William Robertson Bldg, 50 George Square, Edinburgh EH8 9JY
Tel: 031-650-3806

Deputy Chairman: Alan Sykes, European Business Management School,
University of Wales, Singleton Park, Swansea, SA2 8PP
Tel: 0792-295296

BAA Representative: David Eastwood, MicroAPL Ltd, South Bank Technopark,
90 London Road, London SE1 6LN Tel: 071-922-8866

Nonlinear Regression Modelling in APL

by Peter Hingley

This article is a summary of a talk I gave at the ASL meeting in Gregynog in October 1991. It was suggested that a nonlinear regression modelling volume might be written for a future edition of the library, to augment the functions already available in the Regression shelf for analysing generalised linear models (GLIMs). Since then a similar suggestion has been made by Appleton [1].

In regression a data set is related to an underlying set of independent variables by an appropriate function, and the parameters of the function are to be estimated. One of the simplest examples of regression is of course the straight line model, $y = a + bz$, solvable on many statistical calculators. In this case the independent variable is z , the dependent variable is y , and the parameters are a and b .

There is a bewildering variety of types of nonlinear model, including time series models [2] and dynamic models exhibiting chaotic behaviour [3]. I shall consider the restricted situation where observed data are imagined to be generated by a regression model consisting of a function of independent variables, assuming fixed but unknown parameters and independent, identically distributed errors.

The aim is to find a method for estimating the values of the underlying parameters. In the functions and examples given below I will further restrict attention to models which are differentiable with respect to parameters, and have a single set of independent variables. Hence the APL functions would need to be greatly augmented to provide general purpose nonlinear estimation routines.

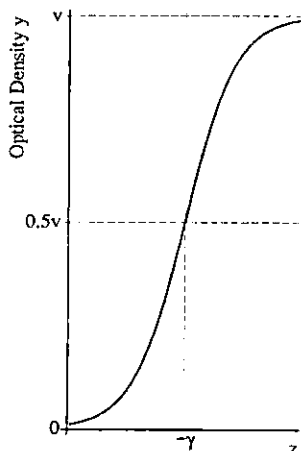
A Logistic Function for ELISA Data

As an example of a nonlinear regression model, consider the logistic function. This has often been suggested as being appropriate for biochemical measurement systems where the possibility exists of saturation of the system at high levels of dosage input [e.g. 4]. We suggested [5] that one particular formulation of a logistic might be usefully applied to *Enzyme Linked Immunosorbent Assays* (ELISAs) for characterisation of antibodies against Foot and Mouth Disease in animal serum.

In this kind of assay fixed concentrations of serum antibody are reacted with varying concentrations of a standard virus antigen preparation [6]. The extent of reaction between antigen and antibody is given by a coloured indicator, the depth of colour being measured as optical density in a spectrophotometer.

One way of writing the logistic function is:

$$y = v \cdot \frac{10^{a(z+\gamma)}}{1 + 10^{a(z+\gamma)}} + e = f(\theta, z) + e$$



Here y is the optical density assay response, z is the applied log dosage of antigen, e is experimental error, v is the optical density at saturation, γ represents the log dosage giving 50% saturation, and a represents antibody heterogeneity.

It is of course obvious to APL users that a data set comprising n observations can be represented by $n \times 1$ vectors y , z , e ; and that a set of p parameters like γ , a , v can be represented by a $p \times 1$ vector θ (here $p = 3$). To keep things simple we will assume that the constituents of e are all independent and have zero expectation.

Now it is possible to transform the logistic equation so that it becomes a straight line model.

$$\log \left[\frac{f(\theta, z)}{v - f(\theta, z)} \right] = a(z + \gamma)$$

However in doing this the error structure is perturbed. That is, the error structure of the new model becomes itself a function of inputs, outputs and parameters, leading to considerable complications in estimation. It is far better to use an estimation method based on the original model, but taking into account the inherent nonlinearity. In any case many other model formulations can not be so easily linearised as this example.

Nonlinear Estimation

The following method has been found appropriate for the logistic model, and should be applicable to other cases also. In an earlier publication [5] APL functions were provided to solve a logistic model specifically. In what follows I will generalise the approach to deal with any model of a similar type. I will describe APL functions to implement a Gauss-Newton estimation method [7,8]. Other possible approaches that could be used include Newton-Raphson, Grid Search, Random Search or Derivative free methods, and the interested reader is encouraged to consult Seber and Wild [9] for a description of these. Other useful books include references [10, 11, 12 and 13].

We shall search for parameter estimates, $\theta = \hat{\theta}$, which minimise the Residual Sum of Squares (RSS), defined as

$$RSS = [y - f(\theta, z)]^T [y - f(\theta, z)]$$

$\begin{matrix} 1 \times n & & n \times 1 \end{matrix}$

where T indicates transposition, and $f(\theta, z)_{n \times 1}$ is the nonlinear function giving the expected value of $y_{n \times 1}$ for a given vector $z_{n \times 1}$ of independent variables.

The method will redefine the problem in terms of an approximate linear model. A linear model is a special case of the nonlinear model in which the function $f(\theta, z)$ is written as a product of a design matrix $X_{n \times p}$ and a parameter vector $B_{p \times 1}$.

$$y = X \cdot B + e$$

$\begin{matrix} n \times p & p \times 1 \end{matrix}$

It can easily be proved, as is shown in most statistics textbooks [e.g. 7], that the estimate of $\hat{B}$ that exactly minimises the RSS is:

$$\hat{B} = [X^T X]^{-1} \cdot X^T y$$

$\begin{matrix} p \times p \end{matrix}$

where $[\]^{-1}$ indicates inversion of a square matrix.

Returning now to the nonlinear case, it is possible to approximate $f(\theta, z)$ by means of the first two terms of a Taylor expansion.

$$f(\theta, z) \approx f(\theta^a, z) + f'(\theta^a, z)^T (\theta - \theta^a)$$

$\begin{matrix} n \times 1 & & n \times 1 & & n \times p & & p \times 1 \end{matrix}$

where θ^a is a guess for θ , and $f'(\theta, z)_{p \times n}$ is the derivative of $f(\theta, z)$ with respect to θ .

The nonlinear model for y can now be approximated as

$$y \approx f(\theta^a, z) + f'(\theta^a, z)^T (\theta - \theta^a) + e$$

Rearranging, we can get an expression for the approximate residuals r^a .

$$\begin{aligned} r^a &= y - f(\theta^a, z) \\ &\approx f'(\theta^a, z)^T (\theta - \theta^a) + e \end{aligned}$$

$\begin{matrix} n \times p & p \times 1 \end{matrix}$

This expression is formally equivalent to a linear model, as defined above, with $X = f'(\theta^a, z)^T$, and $B = (\theta - \theta^a)$. We can therefore write an approximate solution for $(\hat{\theta} - \theta^a)$ as:

$$(\hat{\theta} - \theta^a) \approx [f'(\theta^a, z)^T f'(\theta^a, z)]^{-1} \cdot f'(\theta^a, z)^T r^a = d^a$$

Enough now exists for an estimation algorithm (similar to that described in [1]). Make a first guess for θ^a . Then calculate d^a , and update θ^a to give θ^{a+1} as

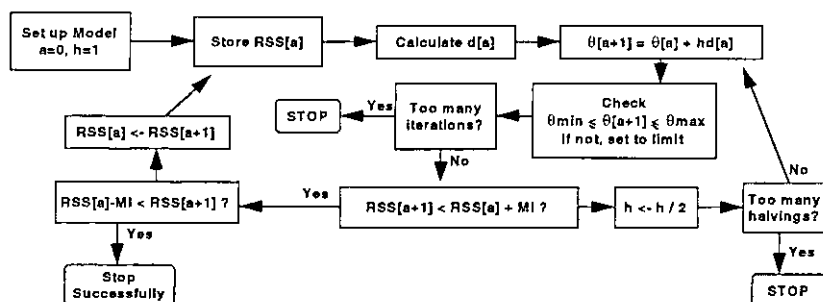
$$\theta^{a+1} = \theta^a + d^a$$

Repeat the process iteratively, and declare convergence, $\hat{\theta} = \theta^{a+1}$, when the next iteration modifies RSS by less than a given tolerance MI. That is when

$$RSS^a - MI < RSS^{a+1} < RSS^a + MI$$

Overshooting

The iterative process defined above suffers from the difficulty that the linearised parameter increment on any particular iteration can accidentally increase RSS, either by overshooting the point of minimum RSS or by some other peculiarity of the RSS surface. A simple modification to the algorithm can easily stop this happening. If RSS rises by more than the tolerance factor MI on any particular iteration, then the parameter increment is successively halved until an acceptable value of θ^{a+1} is found.



The flow diagram shown opposite summarises the operation of the algorithm, including limits on the numbers of iterations and halvings, and limits to the range of parameter estimates. It should be noted that the system technically only searches for local optima, and may become confused if the RSS surface has more than one minimum within the allowable range of parameters. The process is not guaranteed to converge successfully, but in regular problems with sensible starting estimates it usually seems to work. The main function *ESTIMATE* calls two subsidiary functions; *MFIZ* specifies the model $f(\theta, z)$, while *MDFIZ* specifies the derivatives $f'(\theta, z)$. After executing *ESTIMATE* the parameter estimates are found in the vector *PI*, the fitted values $f(\hat{\theta}, z)$ in *FIZ*, while RSS is, not surprisingly, given by *RSS*. All the functions in this article were implemented in TRYAPL2, but should be easily transferable to other interpreters.

```

V ESTIMATE
[1]  A CARRIES OUT NONLINEAR PARAMETER ESTIMATION
[2]  A CAN BE USED INDEPENDENTLY OR CALLED FROM SIMLATE
[3]  A THE FOLLOWING GLOBAL VARIABLES ARE EXPECTED:-
[4]  A Y, THE VECTOR OF DEPENDENT VARIABLES
[5]  A P0, THE P*1 VECTOR OF INITIAL PARAMETER ESTIMATES
[6]  A PU, PL P*1 VECTORS SHOWING UPPER AND LOWER BOUNDARIES
[7]  A HM THE MAXIMUM NUMBER OF HALVINGS
[8]  A MI THE MINIMUM RESIDUAL INCREMENT FOR CONVERGENCE
[9]  A IM THE MAXIMUM NUMBER OF ITERATIONS ALLOWED FOR EST'N
[10] A CALLS FUNCTIONS MFIZ, MDFIZ
[11] MFIZ P0
[12] RSS+/( (Y[;1]-FIZ[;1])^2)
[13] I+0
[14] HIS=WSS+RSS
[15] I+I+1
[16] XH+1
[17] YOF=Y-FIZ
[18] MDFIZ P0
[19] PINC=YOF*(MDFIZ)
[20] J+1
[21] IIS=PI+P0+(PINC*XH)
[22] PI+(PI*(PI<PU))+(PU*(PI>PU))
[23] PI+(PI*(PI>PL))+(PL*(PI<PL))
[24] MFIZ PI
[25] RSS+/( (Y[;1]-FIZ[;1])^2)
[26] RD=RSS-WSS
[27] +JIS*(I=IM+1)
[28] +NIS*(RD<MI)
[29] +OIS*(J=HM)
[30] XH+XH+2
[31] J+J+1
[32] +IIS
[33] OIS:'TOO MANY HALVINGS'
[34] NIS:P0+PI
[35] +HIS*( (((RD*2)*0.5)>MI)^(XH<1))
[36] +KIS
[37] JIS:'TOO MANY ITERATIONS'
[38] KIS:
V

```

```

V MFIZ P
[1]  R CALCULATES EXPECTED VALUES FOR DEPENDENT VARIABLES OF ESTIMATION MODEL
[2]  R SETS UP A N*1 MATRIX FIZ
[3]  R THE FOLLOWING GLOBAL VARIABLES ARE EXPECTED:-
[4]  R Z, A VECTOR OF INDEPENDENT VARIABLES
[5]  R Y, A VECTOR OF DEPENDENT VARIABLES
[6]  FIZ+(pY)pFIZ+0
[7]  A+B
[8]  R CALCULATIONS FOR LOGISTIC ESTIMATION MODEL
[9]  FIZ[;1]+10*(P[2;]*(P[1;]+Z[;1]))
[10] FIZ[;1]+P[3;]*FIZ[;1]/(1+FIZ[;1])
[11] +C
[12] R CALCULATIONS FOR EXPONENTIAL DECAY ESTIMATION MODEL
[13] B:FIZ[;1]+P[1;]*(1-(10*(1*P[2;]*Z[;1])))
[14] C:
V

V MDFIZ P
[1]  R CALCULATES DERIVATIVE MATRIX OF EST MODEL WRT PARAMS
[2]  R SETS UP A P*N MATRIX DFIZ
[3]  R THE FOLLOWING GLOBAL VARIABLES ARE EXPECTED:-
[4]  R Z, A VECTOR OF INDEPENDENT VARIABLES
[5]  R Y, A VECTOR OF DEPENDENT VARIABLES
[6]  ZIPY+pY[;1]
[7]  R CALCULATIONS FOR LOGISTIC ESTIMATION MODEL
[8]  A+B
[9]  DFIZ+(3,ZIPY)pDFIZ+0
[10] ZIP+10*(P[2;]*(P[1;]+Z[;1]))
[11] DFIZ[1;]+P[3;]*(10*P[2;]*ZIP/((1+ZIP)*2))
[12] DFIZ[2;]+DFIZ[1;]*(P[1;]+Z[;1])+P[2;]
[13] DFIZ[3;]+ZIP*(1+ZIP)
[14] +C
[15] R CALCULATIONS FOR EXPONENTIAL DECAY ESTIMATION MODEL
[16] B:DFIZ+(2,ZIPY)pDFIZ+0
[17] ZIP+10*(1*P[2;]*Z[;1])
[18] DFIZ[1;]+1-ZIP
[19] DFIZ[2;]+(10)*P[1;]*Z[;1]*ZIP
[20] C:
V

```

Example 1: ELISA of Antibody/Antigen Interaction

First I will demonstrate the operation of *ESTIMATE* using the logistic model for ELISA specified above. The data for estimation are explained above, and in more detail in [14]. The following output shows a possible terminal session. The plots were drawn using Anscombe's *SCATTERPLOT* function [15], and show visually that the fitted values *FIZ* agree well with the data.

```

bZ
1.81 1.81 2.11 2.11 2.41 2.41 2.71 2.71 3.01 3.01 3.31 3.31
    3.61 3.61 3.91 3.91

bY
0.157 0.144 0.303 0.229 0.538 0.41 0.62 0.599 0.863 0.817 1.049
    1.039 1.147 1.158 1.184 1.274

```

```

IM+500
MI+0.0001
HM+20
PL+3 1p-10 0 0
PU+3 1p10 10 10
PO+3 1p-2.5 1 1

```

```
ESTIMATE
```

```
qPI
```

```
-2.753996549
```

```
0.9207034535
```

```
1.340651261
```

```
qFIZ
```

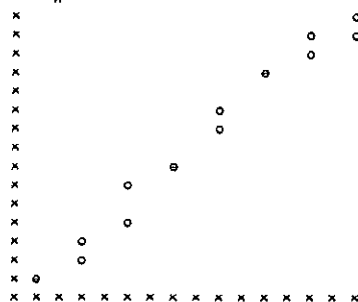
```

0.1596307509 0.1596307509 0.272668213 0.272668213 0.4361869199 0.4361869199
0.6390867678 0.6390867678 0.8478902192 0.8478902192 1.025221088 1.025221088
1.152868976 1.152868976 1.234222932 1.234222932

```

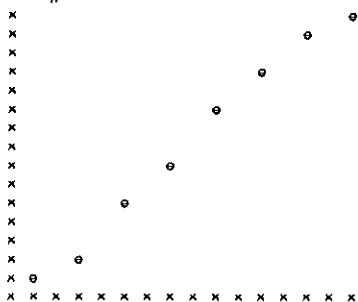
```
W+(,Z) SCATTERP (,Y)
```

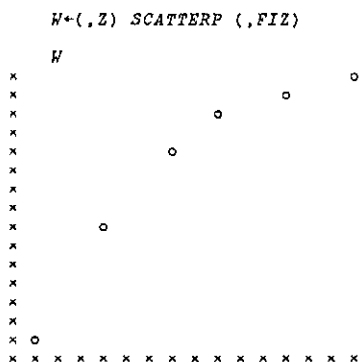
```
W
```



```
W+(,Z) SCATTERP (,FIZ)
```

```
W
```





Apart from giving a confirmation that *ESTIMATE* works well in another situation, this example is of interest because Bates and Watts report that the expectation surface in parameter space is moderately curved, and the linearised Gauss-Newton steps do not match well to the surface. This might well lead to difficulties in obtaining precise estimates, but actually the results shown above seem adequate. Incidentally the estimates given here have an RSS of 6.701, slightly lower than the RSS of 6.707 obtainable from Bates and Watts' estimates, though they quote a value of 6.498 which is clearly incorrect. Some degree of indeterminacy in final estimates is always to be expected in nonlinear estimation because convergence is declared at a certain level of acceptable tolerance. It is necessary for the user to ensure that the tolerance value *MI* in any particular case is not too large to lead to inaccurate estimates, but not so small that an excessive number of iterations are required.

Probability Densities for Least Squares Estimates

There are a variety of statistical diagnostics that can be calculated after fitting a model. For the most part these are analogous to quantities used with linear models, and APL functions could easily be developed for them. However some extra restrictions are usually required on the error terms, typically that the vector *e* follows a multivariate normal distribution.

In order to demonstrate the advantage of using APL for extensions, I will show how functions can easily be established to simulate many sets of data, estimate parameters from each set, and then display the results in the form of a multi-dimensional frequency table. This kind of table gives a good impression of the shape of the multivariate probability distribution for the estimates, provided that a sufficiently large number of simulated data sets have been used to iron out sampling effects.

Consider the problem of simulating sets of data from the ELISA test logistic curve that has been described above. Assume that each data point from a simulated experiment has a normally distributed error, and that the errors for each point are independent and have constant standard deviation. The following global variables should first be specified: *SIG*, the standard deviation of error assumed; *NSIM*, the number of simulated experimental data sets; *PIN*, the true values of the parameters for generating the curves; and *PST*, the initial guess values for the estimation of parameters for each curve. (In practice we would probably set *PST* = *PIN*, except in the case of estimation when using a different model to that generating the data.)

The function *SETUP* allows the midpoints of the multi-dimensional frequency table to be defined, and then calls *SIMLATE*. Then for each of the simulated data sets, the true data generating model is specified by *MFOZ*, the random normal errors are added by *RNORMAL* (taken from [15]), while the parameter estimation is carried out using *ESTIMATE* and its associated functions as described earlier.

```

V SETUP
[1]  A THIS SETS UP A MATRIX BIN IN WHICH TO STORE SIMULATION COUNTS,
[2]  A AND A MATRIX BINL TO STORE CLASS BOUNDARIES.
[3]  A THE FOLLOWING GLOBAL VARIABLES ARE EXPECTED:-
[4]  A PIN, A P*1 MATRIX CONTAINING TRUE PARAMETER VALUES
[5]  A
[6]  A CALLS FUNCTION SIMLATE
[7]  BIN←1
[8]  DIN←DIN2←1
[9]  BINL←(1,(P PIN[;1]))P BINL←0
[10] CPIN←0
[11] IK←1
[12] CLA:CPIN←CPIN+1
[13] 'ENTER CLASS DATA FOR PARAMETER ',CPIN
[14] WDIN2←DIN2
[15] WDIN←DIN
[16] DIN2←0
[17] DIN←((P DIN2)+1)P DIN←0
[18] DIN[1]←(((3*DIN2[1])−DIN2[2])÷2)
[19] K←2
[20] ILA:→ILB×1(K=((P DIN2)+1))
[21] DIN[K]←(DIN2[K−1]+DIN2[K])÷2
[22] K←K+1
[23] →ILA
[24] ILB:DIN[(P DIN2)+1]←(((3*DIN2[P DIN2])−(DIN2[(P DIN2)−1]))÷2)
[25] BIN←(((P DIN)+1),(P BIN))P 0
[26] →DLB×1((P DIN)≤(P BINL)[1])
[27] WBINL←BINL
[28] BINL←((P DIN),(P PIN[;1]))P 0
[29] ZE←(((P BINL)[1])−((P WBINL)[1]))P ZE←0
[30] I←1
[31] CLB:BINL[;I]←(WBINL[;I]),(ZE)
[32] I←I+1
[33] →CLB×1(I<CPIN)
[34] BINL[;CPIN]←DIN
[35] →ELB
[36] DLB:ZE←(((P BINL)[1])−(P DIN))P ZE←0
[37] BINL[;CPIN]←(DIN),(ZE)
[38] ELB:IK←IK+1
[39] →CLA×1(CPIN<(P PIN[;1]))
[40] SIMLATE
V

```

```

V SIMULATE
[1] A THIS CARRIES OUT NSIM SIMULATIONS OF DATA SETS. ESTIMATES ARE
[2] A GENERATED AND STORED AS COUNTS IN BIN. A MATRIX BIND IS SET UP IN
[3] A WHICH SIMULATED APPROXIMATION OF P.D.F. OF ESTIMATES IS STORED.
[4] A THE FOLLOWING GLOBAL VARIABLES ARE EXPECTED:-
[5] A INFORMATION GENERATED BY SETUP
[6] A SIG, S.D. OF EXPERIMENTAL ERROR
[7] A PU,PL, P*1 MATRICES CONTAINING UPPER AND LOWER PARAMETER BOUNDARIES
[8] A NSIM, NUMBER OF SIMULATIONS
[9] A PST, P*1 MATRIX CONTAINING PARAMETER STARTING VALUES
[10] A PIN, P*1 MATRIX CONTAINING TRUE PARAMETER VALUES
[11] A CALLS FUNCTIONS MFOZ, RNORMAL AND ESTIMATE
[12] MFOZ PIN
[13] SJ+1
[14] TBIN←,BIN
[15] BINM←pBIN
[16] BIND←TBIN
[17] N←((pFOZ)[1])
[18] SIMO:Y←RNORMAL N
[19] DIVI+1
[20] Y←N 1pY
[21] Y←(SIG*Y)+FOZ
[22] Po←PST
[23] ESTIMATE
[24] I+1
[25] J+1
[26] LA:←LB*(J=(pBIN)+1)
[27] K+0
[28] LC:K←K+1
[29] →LD*(K=((pBIN)[J]))
[30] →LC*(PI[J]>BINL[K;J])
[31] LD:IT←K-1
[32] →LD2*(K=1)
[33] →LD3*(K=((pBIN)[J]))
[34] DIVI+DIVI*(BINL[K;J]-BINL[(K-1);J])
[35] →LD4
[36] LD2:DIVI+DIVI*(BINL[1;J]-PL[J;J])
[37] →LD4
[38] LD3:DIVI+DIVI*(PU[J;J]-BINL[(K-1);J])
[39] LD4:ITI←J
[40] LG:←LF*(ITI=pPIN[1]+1)
[41] IT←IT*((pBIN)[ITI])
[42] ITI←ITI+1
[43] →LG
[44] LF:I←I+IT
[45] J←J+1
[46] →LA
[47] LB:TBIN[I]←TBIN[I]+1
[48] BIND[I]←BIND[I]+(1+(NSIM*DIVI))
[49] SJ←SJ+1
[50] →SEND*(SJ=NSIM+1)
[51] →SIMO
[52] SEND:BIN←BINM p TBIN
[53] BIND←BINM p BIND
V

```

```

V MFOZ P
[1] A CALCULATES DEPENDENT VARIABLES FOR TRUE MODEL, IN N*1 MATRIX FOZ
[2] A THE FOLLOWING GLOBAL VARIABLES ARE EXPECTED:-
[3] A Z, N*1 MATRIX OF INDEPENDENT VARIABLES
[4] A Y, N*1 MATRIX OF DEPENDENT VARIABLES
[5] A CURRENTLY SET UP FOR A LOGISTIC MODEL
[6] FOZ←(pY)pFOZ←0
[7] FOZ[1;1]←10*(P[2;1]*(P[1;1]+Z[1;1]))
[8] FOZ[1;1]←P[3;1]*FOZ[1;1]+(1+FOZ[1;1])
V

```

```

V Z+RNORMAL N;V
[1] +2+V/, (N≥1)^(N=[N])^0=ppH
[2] +0,p[ ]+HO GO., Z+,
[3] Z+{(2147483647)*7(2,[N+2])p2147483647
[4] Z+N*{,1 2+.OZ[2;]*O2)*V,V+{-2*-Z[1;]}*0.5
V

```

The following printout shows part of a run involving 500 simulated data sets. The analysis represents a continuation directly from the session on ELISA data given above in Example 1. Set *PST* and *PIN* to be the same as the estimates *PI* generated by the real data analysis, and the fixed standard deviation to be equivalent to the estimate obtained from the raw data. After running the simulations the frequency table is given by matrix variable *BIN*, while the class boundaries are shown by the matrix variable *BINL*. Inspection of *BIN* shows the strong interdependent relationship between the three estimates γ , a and v ; behaviour that is usually only represented indirectly by the covariance matrix for estimates when using other nonlinear estimation packages.

```

PST+PIN+PI
SIG+{(RSS÷13)*0.5)
SIG
0.04132816099

NSIM+500
IRL
1805305474
SETUP
ENTER CLASS DATA FOR PARAMETER 1
[]: -2.85 -2.8 -2.75 -2.7 -2.65 -2.6 -2.55
ENTER CLASS DATA FOR PARAMETER 2
[]: 75 .8 .85 .9 .95 1.0 1.05
ENTER CLASS DATA FOR PARAMETER 3
[]: 1.2 1.25 1.3 1.35 1.4 1.45 1.5

```

```

BINL
-2.875 0.725 1.175
-2.825 0.775 1.225
-2.775 0.825 1.275
-2.725 0.875 1.325
-2.675 0.925 1.375
-2.625 0.975 1.425
-2.575 1.025 1.475
-2.525 1.075 1.525

```

```

BIN
0 0 0 0 0 0 0 0 2 ... 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 5 0 ... 0 0 0 0 0 0 0 3 1 0
0 0 0 0 0 0 0 0 1 0 ... 0 0 0 0 0 0 3 14 1 0
0 0 0 0 0 0 0 2 0 0 ... 0 0 0 0 0 12 7 0 0
0 0 0 0 0 0 0 0 0 0 ... 0 0 0 0 1 1 1 0 0
0 0 0 0 0 0 0 0 0 0 ... 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 ... 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 ... 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 ... 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 ... 0 0 0 0 0 0 0 0 0 0

```

... and so on

A matrix variable (*BIND*, not shown) is also produced to formally represent the simulated probability distribution of estimates. Actually when an estimation model is linear the estimate distribution will be multivariate normal, but in a nonlinear case like this the situation is more intricate. Such simulations can be used to check the adequacy of analytic approximations, such as those given by asymptotic maximum likelihood theory or saddlepoint approximations [17]. Further, the system can be easily used to investigate the behaviour of estimates when the regression model used for estimation differs from the model that is actually generating the data. It could then be used to augment studies of discrimination between models [18].

Conclusions

The function *ESTIMATE* can be used for application of the Gauss-Newton method to nonlinear estimation, where an independent constant standard deviation error model is appropriate for a set of data. Fairly obvious extensions could be made to deal with non-normal or non-independent errors, or to other techniques of estimation. It might be sensible to include *ESTIMATE*, or some function related to it, in any potential nonlinear regression volume for the ASL system. A different approach for ASL might be to attempt to re-write generalised linear models as generalised nonlinear models [19], by removing the distinction between the linear predictor and the link function.

I have not made any attempt so far to adapt the functions to ASL standards [20]. Also I have not attempted to make the defined functions maximally efficient in APL terms, and they could probably be written with fewer lines of code. However it is an open question whether general purpose APL functions run more or less efficiently if they are maximally compacted to take into account the elegance of the language.

The extension to probability distributions for estimates is a more specialised area which may not be of universal interest, but is important and demonstrates the kind of investigation which can easily be pursued in the APL milieu.

References

- [1] D.R. Appleton, *Curve-fitting in APL*, Vector, Vol.8, No.3 (1992), pp.23-28.
- [2] T. Amemiya, *Advanced econometrics*, Basil Blackwell, Oxford (1985) 521pp.
- [3] J.M.T. Thompson, and H.B. Stewart, *Nonlinear dynamics and chaos*, John Wiley, Chichester (1986) 376pp.

- [4] D. Rodbard, *Statistical quality control and routine data processing for radioimmunoassays and immunoradiometric assays*, Clinical Chemistry, Vol.20 (1974), pp.1255-1270.
- [5] P.J. Hingley, and E.J. Ouldrige, *The use of a logistic model for the quantitative interpretation of indirect sandwich enzyme labelled immunosorbent assays (ELISA) for antibodies and antigens in foot and mouth disease*, Computers in biology and medicine, Vol.15, No.3 (1985), pp.137-152.
- [6] E.M.E. Abu Elzein, and J.R. Crowther, *Enzyme labelled immunological assay techniques in foot and mouth disease virus research*, Journal of Hygiene, Vol.80 (1978), pp.391-399.
- [7] N.R. Draper, and H. Smith, *Applied regression analysis*, John Wiley, New York, Second edition (1981) 709pp.
- [8] R.I. Jennrich, and P.F. Sampson, *Application of stepwise regression to non-linear estimation*, Technometrics, Vol.10 (1968), pp.63-72.
- [9] G.A.F. Seber, and C.J. Wild, *Nonlinear regression*, John Wiley, New York (1989), 768pp.
- [10] D.M. Bates, and D.G. Watts, *Nonlinear regression analysis and its applications*, John Wiley, New York (1988), 365pp.
- [11] A.R. Gallant, *Nonlinear statistical models*, John Wiley, New York (1987), 610pp.
- [12] D.A. Ratkowsky, *Nonlinear regression modelling*, Marcel Dekker, New York (1983), 276pp.
- [13] G.J.S. Ross, *Nonlinear estimation*, Springer-Verlag, New York (1990), 189pp.
- [14] P.J. Hingley, *Antibody affinity distribution-based models for vaccine potency assessment*, in *Theoretical Immunology*, A.S. Perelson (ed.), Reading, Massachusetts (1988), Part one, pp.311-325.
- [15] F.J. Anscombe, *Computing in statistical science through APL*, Springer-Verlag, New York (1981), 426pp.
- [16] D. Marske, *Biochemical oxygen demand data interpretation using sum of squares surface*, unpub. M.Sc. thesis, University of Wisconsin-Madison (1967).
- [17] O.E. Barndorff-Nielsen, *Parametric statistical models and likelihood*, Lecture notes in statistics, Vol.50, Springer-Verlag, New York (1988).
- [18] D.S. Borowiak, *Model discrimination for nonlinear regression models*, Marcel Dekker, New York (1989), 177pp.
- [19] P. McCullagh, and J.A. Nelder, *Generalized linear models*, Chapman and Hall, Second edition, London (1989), 511pp.
- [20] D.S. Eastwood, *ASL Standards*, Vector, Vol.8, No.3 (1992), pp.46-51.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: Using DOS File Functions	Duncan Pearson	124
Technical Correspondence		126
• • ? (Random Jottings) <i>DOIF</i> Considered Harmful	Dave Piper	133

Hackers' Corner: Using DOS File Functions

by Duncan Pearson

With DOS being a file based system, a large number of devices operate by behaving as files. Examples are the printer and serial ports and some network devices. DEC Pathworks, for instance has a facility called TTT which associates a process on a VAX with a pseudo-file on the PC. The normal `□NREAD` function has the disadvantage, when used to read from this type of device, that if you try to read beyond the end-of-file you get an error and no data at all. "And so," thought I (having just invested in a copy of "Advanced MSDOS Programming" by Ray Duncan) "why not use the DOS file functions via `□INT 33`?"

Here I came across a problem. If a DOS function needs a text string, or returns text data then the segment address and offset of the text (or the buffer for data) need to be passed in the DS:DX register pair. Unfortunately `□INT`, the normal method of calling interrupts, does not allow you to set DS.

The function `DOSΔINT`, which follows, gets round this problem.

The function passes the symbol table pointer of the variable containing the text in the register DI. Then a gobblet of 8086 machine code reads the address pointed to by DI into DS and calls the interrupt. The extra lines of machine code are for saving and restoring the previous value of DS and for putting into DX the offset of the actual data in the variable (10 for a text vector).

The assembler for the gobblet is:

```
push ds      ; save the previous contents of DS
mov  ds,[di]  ; move into DS the address pointed to by DI
mov  dx,10    ; move into DX the offset of the data in the var
int  33       ; call the DOS interrupt
pop  ds      ; restore the previous value of DS
ret          ; return to APL
```

and the function which uses it is :

```

▽ OUTREGS←INREGS DOSΔINT DS_DX;CODE;RESSIZE
[1]  ▹ Call the DOS interrupt with DS:DX pointing to an APL vector
[2]  ▹ REGS is an integer vector of the registers AX BX CX DX BP & SI
[3]  ▹ DS_DX is the symbol table ptr (ΔSTPTR) of the variable to which
[4]  ▹ DS:DX should point.
[5]  ▹ The contents of DX will be overwritten by 10 which is the
[6]  ▹ offset into a text vector of the actual data
[7]
[8]  ▹ push ds ; mov ds,[di] ; mov dx,10 ; int 33 ; pop ds ; ret
[9]  CODE← 30 142 29 186 10 0 205 33 31 203
[10]
[11] ▹ Ensure that ΔCALL returns all of AX ... DI Flags DS and ES
[12] RESSIZE←10 ΔPOKE 886
[13]
[14] ▹ Call the code with DS_DX in the DI register
[15] OUTREGS←((6+INREGS),DS_DX)ΔCALL ΔAV(ΔIO+CODE)
[16]
[17] ▹ Restore the previous behaviour of ΔCALL
[18] 0 0 ΔRESSIZE ΔPOKE 886
▽

```

Here also are two functions to illustrate its use. These use the DOS functions 61 and 62 to open and read from a file. In the first case the filename is passed to the function and in the second the file data is passed back to APL. The general method is to set up a variable as a text vector in APL, call the *DOSΔINT* function with the symbol table pointer of the variable as right argument, then interpret the contents of the registers returned.

```

▽ HANDLE←FILE_OPEN NAME;REGS
[1]  ▹ Open a file with the DOS interrupt, returning the file handle
[2]
[3]  ▹ Null terminate the name as required by DOS
[4]  NAME←NAME,ΔTCNUL
[5]
[6]  ▹ Call the DOS fn 61 (open file) in read/write mode (AX←256161 2)
[7]  ▹ With DS_DX pointing at the name
[8]  REGS←15618 DOSΔINT ΔSTPTR 'NAME'
[9]
[10] ▹ Check the flags register for an error, which would be in AX
[11] ΔERROR(2|REGS[ΔIO+7])/'Dos file error ',ΔREGS[ΔIO]
[12]
[13] ▹ If successful return the file handle from AX
[14] HANDLE←REGS[ΔIO]
▽

```

```

V DATA←HANDLE FILE_READ BYTES;REGS
[1] A Read from a file with the DOS interrupt
[2]
[3] A Create a blank result of the correct size
[4] DATA←BYTESp' '
[5]
[6] A Call the DOS fn 63 (read file) with the handle in BX and the
[7] A number of bytes to read in CX. DS_DX will point to the data area
[8] A of DATA
[9] REGS←(16128,HANDLE,BYTES)DOSAINTR STPTR 'DATA'
[10]
[11] A Check the flags register for an error, which would be in AX
[12] ERROR(2|REGS[IO+7])/'Dos file error ',REGS[IO]
[13]
[14] A If successful, AX holds the no of bytes actually read, which are
[15] A returned
[16] DATA←REGS[IO]+DATA
V

```

As mentioned above this last function has the advantage over `INREAD` that it will not cause an error if an attempt is made to read beyond the end of a file.

TECHNICAL CORRESPONDENCE

From: Norman Thomson

14 February 1992

David Appleton is right to observe (Vector Vol. 8 No. 3) that curve fitting routines for non-linear functions should be included in ASL, and indeed more generally there should be awareness that although it is only linear fits which come as a primitive with APL, the domino function nevertheless allows fitting of a much wider range of non-linear functions at the cost of a very modest amount of programming.

I take issue with David in using a technique which requires the analyst to work out his partial derivatives in algebra - surely a denial of what a computer is for, quite apart from the unnecessary wear on the execute key!

Take as a starting point David's example function:

$$y = a - (a - b) \exp(-cx)$$

and define as *FN* its realisation as an APL function (*C* stands for coefficients):

```
[0]  Z←C FN X
[1]  Z←C[1]+(-/C[2 1])×*-C[3]×X
```

Partial derivatives of a function *F* are estimated by defining an operator:

```
[0]  Z←C(F PDERIV X)N;T
[1]  N is index of coefficient whose partial deriv. is reqd
[2]  Z←((C+T×N=1ρC)F X)-C F X)÷T←ΔX3[N]
```

David in his derivative function uses a blanket .01% for ΔX , however intervals can be defined for each coefficient separately as the rightmost part of the above function line implies:

```
ΔX3
0.01 0.01 0.001
```

Now use domino to do the fit:

```
[0]  Z←X(F FIT Y)C
[1]  Z←C+(Y-C F X)⊞[1](←C)F PDERIV X)∘1ρC
```

Given the following observed readings which are approximately the same as David's:

```
      X
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
      Y
21 27 29 31 30 37 35 32 34 37 39 37 37 40 36 37 40 41 40 38
```

and a first guess at the coefficients:

```
C0
40 20 0.2
```

running the function *FIT* gives:

```
      X FN FIT Y C0
39.303 22.524 0.20628
```

Now develop a means of finding an iterative solution with prescribed accuracy. Start by defining a completely general operator which executes a dyadic function *P* feeding back the result until a condition defined by the predicate function *Q* is attained:

```
[0]  Z←L(P UNTIL Q)R
[1]  →0 IF Q Z←R
[2]  Z←L P UNTIL Q L P R
```

The function *Q* defines a stopping criterion which might be that all the coefficients are within *EPS* of the previous iteration. The function *ALLNEAR* uses the *P* and *L* of *UNTIL*:

```
[0]  Z←ALLNEAR C
[1]  Z←^/EPS>|C-C P L
```

```
EPS
0.00001
```

A small amendment must be made to *FIT*:

```
[0]  Z←A(F FIT1 Y)B
[1]  Z←R+(Y-R F L)⊞=[1](←R)(F PDERIV L)⌈⌈ρR
```

which now has dummy arguments *A* and *B* since it is the derived function (*F FIT1 Y*) which is the left operand of *UNTIL* and the *L* and *R* are those of *UNTIL*. The iterated solution is:

```
X FN FIT1 Y UNTIL ALLNEAR C0
39.281 22.493 0.20907
```

To obtain a trace of the steps towards convergence add $\square\leftarrow$ at an appropriate point in the operator *UNTIL*:

```
[2]  Z←L P UNTIL Q L P  $\square\leftarrow$ R
```

and rerun:

```
X FN FIT1 Y UNTIL ALLNEAR C0
40 20 0.2
39.303 22.524 0.20628
39.289 22.502 0.2086
39.283 22.495 0.20899
39.282 22.494 0.20905
39.281 22.493 0.20906
39.281 22.493 0.20907
```

The above analysis establishes the point that only a small amount of APL programming need be added to domino to generalise from linear to non-linear fitting in a manner consistent with the overall programming philosophies of APL2.

From: David Appleton

10 March 1992

I am delighted that Norman Thomson and Ken Iverson were interested enough to comment on my article on Stirling numbers. As a result, I have learned enough J to understand and execute the wonderfully concise and interesting expression in the latter's letter, thanks to my colleague Chris Brunsdon. Unfortunately, I ran out of workspace on my micro long before I could execute it with a sufficiently high parameter value (60) to compare the results with those of the programs I presented.

It might help anyone interested in this correspondence if I gave an APL version of the method (index origin is zero as in J).

```

K←⊔N
D←K∘.-K
S←D≥0
F←⊔(ρD)ρ:K
A←K∘.×K
B←(F×S)÷!D×S
S1←B⊞A
S2←A⊞B

```

The relationship between the two kinds of Stirling number is certainly made very clear, though as a practical method of calculation it is not as useful as Norman's recursive function.

Perhaps I should come clean! I wrote the original article because the expression $\pm 1 + (7 \times P) \rho \cdot \times + \backslash (\cdot N) \cdot$ amused me, and I thought it might amuse others. I am sure that J can produce the same sort of expression without using character vectors, but I am afraid I cannot yet construct it. However, I also had a real problem involving Stirling numbers, or, as it turned out, the ratios of Stirling numbers, and I needed a good algorithm. Recursion seems to fit the bill best, and my eventual function to calculate $t \sigma_{t-1} / \sigma_t$ was (with index origin back to 1):

```

VZ←STIRRAT T;U
[1] →(T=1)/0,Z←1
[2] Z←(T×(⊖1⊔U)÷T-1+(⊔T-1)×U←⊖1⊔U←STIRRAT T-1),0
V

```

If this looks very like Norman's function, that is because I modelled the final version on it!

More on Strands in PLUS II

From: David Piper

25 May 92

Further to my letter on "STSC's Double Whammy", I have discovered a nasty little bug that seems to have worked its way into the interpreter. In my original letter, I included some discussion of the way in which numeric strands are differentiated from other types of strand during indexing. The bug appears to originate from this dichotomy.

The following session log should illustrate the problem quite nicely:

```

      a+3
      ρ1 2 3
3
      1 2 3[1]
1
      1 2 3[2]
2
      1 2 3[3]
3
      1 2 3=1 2 a
1
      ρ1 2 a
3
      1 2 a[1]
1 2
      1 2 a[2]
3
      1 2 a[3]
INDEX ERROR
      1 2 a[3]
      ^   ^

```

All the primitive functions like *MATCH* and *SHAPE* appear to consider the numeric strand

```
1 2 3
```

to be identical to the strand

```
1 2 a
```

as long as

```
a+3
```

Bracket indexing however does not. Bracket indexing considers the numeric strand to be a discrete item in its own right, thus for bracket indexing the shape of the vector is 2 items, not 3; the first item is the numeric strand (1 2), the second item is the numeric scalar 3.

The behaviour gets even more bizarre. Consider the following example:

```

      1 2 a[1p1 2 a]
INDEX ERROR
      1 2 a[1p1 2 a]
      ^      ^

```

You cannot index this vector by its own index set! ...

```

      (1 2 a)[1p1 2 a]
1 2 3

```

... but simply by placing the strand in parentheses, you can. In fact all the problems are cured as long as the strand is parenthesized before you index into it.

The correct behaviour is given by the parenthesized form, the software that I have written to analyze functions for changes which have to be made to accommodate the evolution levels 1 and 2 assume the correct behaviour, so that parenthesizing the strand before indexing gives the correct results.

DB Piper
11 Lingfield Road
Eastwick Park
Evesham
WR11 6XG

P.S. Since writing this letter, I have found the problem to be present in APL*Plus II version 3.5 also.

Cloning After Dark in APL

From: Duncan Pearson

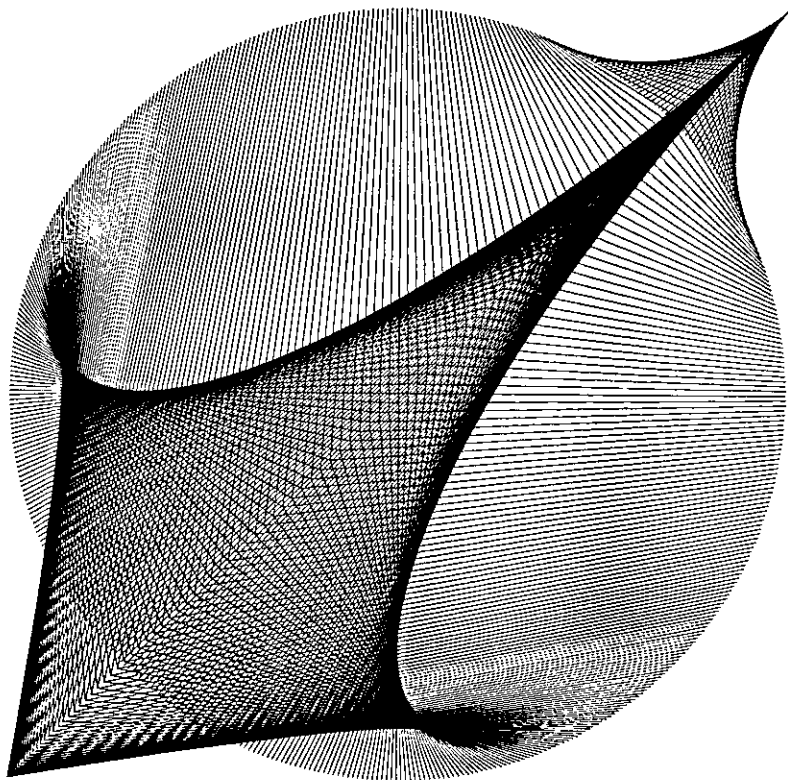
May 1992

Screen savers, programs that do funny things to your screen after prolonged inactivity, are all the rage with Windows users. One of the most popular programs, After Dark, has a module which draws a number of lines on your screen to form a very pleasing pattern.

It can be reproduced by the inevitable APL one-liner which I offer below:

```
PLOT 100(.5, [IO+.1]2*(\LINES)÷LINES)+.×1-[IO-?2 2 2ρ4, COMPLEXITY
```

LINES is the number of lines required, *COMPLEXITY* is a positive integer (try 4, more than about 6 makes a bit of a mess) and *PLOT* is a function that takes an $n \times 2 \times 2$ matrix and draws a line between $ARG[i;1;]$ and $ARG[i;2;]$ for all i (APL*PLUS's *GLINE* is such a function).



◦ . ? (Random Jottings) DOIF Considered Harmful

by Dave Piper

Introduction

From time to time attempts have been made to add structure to APL code through the use of defined functions. The *IF* function is probably the most widely accepted and successful of these attempts, allowing us to write branch statements in the form:

```
+label IF condition
```

In the last year or so, I have come across another function in the same vein - *DOIF*. *DOIF* can be used in the situation where you wish to execute some code only if a condition is true, for example to assign a value to the elided left argument of an ambivalent function:

```
[...] →DOIF 0=⊖NC 'left' ◊ left+....
```

To read this line of code, simply ignore the branch arrow the rest follows relatively easily.

To understand how *DOIF* works. If the condition on the right of *DOIF* evaluates to 1, an empty vector is returned. No branch occurs and execution continues with the statement following the diamond. If the condition evaluates to 0, the line number of the next line ($\ominus LC+1$) is returned. A branch to the next line is executed; statements after the diamond are ignored.

The Culprit

Here is a listing of a typical *DOIF* function:

```

∇ R←DOIF W
[1]  A FN: Return next line number in CALLING fn if <-W>.
[2]  A W - Boolean scalar                      Evaluated condition
[3]  A R - Numeric vector                      Next line number or empty
[4]  R+(-W)/1+~1+2+⊖LC
∇

```

What criticisms can we make about *DOIF* and its use? I want to concentrate on three specific areas:

- use of statement separators
- use of computed goto
- poor APL style

Diamonds are Forever?

The *DOIF* function relies on the diamond statement separator. Without this only a single statement can be placed on a line of code; no statements can follow the *DOIF* itself. Not all versions of APL support statement separators. By encouraging their use (through the use of *DOIF*), portability of code is being impaired. Equally programmers who are not familiar with statement separators will be hard put to read the line of code and understand what it is intended to do.

We can further criticize the use of the diamond separator. Consider a situation where several statements must be executed if our condition is true. Each of these must be present on the same line of code — the natural result is a tendency to produce very long lines of code with resulting reductions in readability and ease of comprehension.

The use of comments will also be affected. Only one comment can be used per line of code restricting our description of an entire “block” of code to a single in-line comment. Of course we are free to use leading and trailing full-line comments but these can lack the immediacy and impact of comments at the end of a line of code.

Poor Quality Diamonds

It can be argued that the excessive use of diamondized lines of code leads to the creation of poor quality code. The statement separator creates within APL a new form of structure — the line of code is elevated to the lofty position of “module”. Traditionally, the only form of module supported by APL has been the defined function (and defined operators in some APLs). Unfortunately, the only characteristic that a diamondized line shares with the defined function is the ability to cram in a lot of processing.

If we look at the two standard measures of module quality [see Page-Jones, Myers, Yourdon] — Coupling and Strength (=cohesion), the diamondized line

will do pretty poorly. The only form of coupling (i.e. links to/from other modules) that can exist between lines of code is COMMON coupling. Common coupling is jargon for the use of "global" variables — if you think about it, there is no way to make a variable local to a line of code.

Similarly, in terms of strength it is unlikely that a single diamondized line will embody the whole of an APL function (indeed the use of *DOIF* precludes this possibility). Thus a "sequential strength" module is the best we can hope for when using diamondized lines. Without the use of statement separators these considerations would not be relevant. A line of code is not normally considered to be a module so the coupling and strength measures would not apply. It is the over-use of statement separators changing a simple line of code into a module which makes these concerns important.

The precise deficiencies of "sequential strength" or "common coupling" are not important, the simple fact is that the use of diamondized lines as modules of code can easily reduce the quality of the code we write. Even if this trap is avoided, more expertise and care is required to ensure the continued creation of high quality code. Expertise is always at a premium.

No More Automation!

Diamonds also reduce the opportunities for "automated" coverage testing. The built in monitor facilities found in some implementations of APL can be used to examine the coverage of a test, since they indicate which lines of code have been executed. The coverage of the monitor facilities is by line of code rather than statement. However, as long as a diamondized line of code involves no branch statements then we can rely on all the statements on the line being executed if the line is executed at all. If a branch is included, then some of the statements on the line may have been executed while others are not. *DOIF* always introduces such a branch.

Note that this argument is one aspect of a more general point — the use of diamondized lines of code may make it significantly more difficult to detect CPU hot-spots by blurring the CPU usage of several statements into a single figure. With well written code, we always have the option of removing the diamonds and re-executing the statements as individual lines of code. Unfortunately because of the computed branch generated by *DOIF*, this option is lost to us.

Computed Goto

The *DOIF* function computes the destination of the branch based on the value returned by the right argument. Of course, the language purists can argue that all conditional branch statements in APL are computed. The real point is that the destination of the branch is not fixed by a line label.

This can affect us in several ways. For example, when delivering the application code to the production environment, we may wish to remove comments and "diamondize" the code. This reduces the size of the delivered code (and possibly improves the performance of the code, especially in APL*PLUS/PC). At the very least, the speed of loading the code from workspace or file will be improved as a result of the reduction in volume of code.

Diamondization usually works by merging together unlabelled lines of code. The use of *DOIF* prevents the diamondization of code because of the branches to unlabelled lines. Without labels, the statements are merged together making it impossible to branch to the statement we require. Consider the following function which computes a simple average if the left argument is elided, or an N-period moving average if specified:

```

▽ R←A AVE W;iset;IO
[1]  A FN: Average or moving average on vector W
[2]  A A - Numeric scalar           Periodicity of average
[3]  A W - Numeric vector           Data to average
[4]  A R - Numeric vector           N-period moving average
[5]  IO←0
[6]  →DOIF 0=nc 'A' ◊ A←pW          A Simplifies index set
[7]  iset←(11+(pW)-A)◊.+.1A         A Default to simple average
[8]  R←(+/W[iset])÷A                A Generate the index set
[9]  A                               A Sum over period and divide
                                   In APL2: R←(A+/W)÷A
▽

data←4 7 2 9 10 15 13 16
AVE data                               A Simple average
9.5
4 AVE data                             A 4 period moving average
5.5 7 9 11.75 13.5

```

The diamondized and decommented version of this function would look something like this:

```

▽ R←A AVE W;iset;IO
[1]  IO←0 ◊ →DOIF 0=nc 'A' ◊ A←pW ◊ iset←(11+(pW)-A)◊.+.1A ◊
R←(+/W[iset])÷A

```

When called dyadically, the branch statement will attempt to GOTO line 2, which is outside the scope of the function. Execution of the *AVE* function will terminate at this point without calculating a result; if an attempt is made to assign the result a *VALUE ERROR* will be reported.

APL Style

Style is, of course, a subjective thing. Even so, within the APL world there is a degree of consensus about what constitutes good style. Branching is often considered a necessary evil — “if you can avoid it do so” seems to be the argument. Such arguments are supported by measures of program complexity that are published from time-to-time which frequently assign the highest complexity ratings to “GOTOs”.

The problem with functions like *DOIF* is that they encourage the use of branching even if it is not strictly necessary.

“Aha”, reasons the programmer,

“I need to perform these calculations only if the following condition is true”.
More thinking quickly follows

“... we have a *DOIF* utility which makes this form of conditional execution so easy ...”.

In the blink of an eyelid, the following code is produced:

```

      V ADJUSTED←TYPE PMT_ADJUST PAYMENT;adjustment;limit;taxamt;taxable
[1]  A FN: Adjust the payments if <TYPE> ....
[2]  A TYPE      - Numeric scalar      Type of payment
[3]  A PAYMENT   - Numeric vector      Vector of payments
[4]  A ADJUSTED  - Numeric vector      Adjusted payments
[5]  adjustment←0                      Assume no adjustment
[6]  A          Adjustment calculation
[7]  →DOIF TYPE<24 28 ∘ limit←PMT_TAXLIMIT ∘ taxable←PAYMENT>limit ∘
taxamt←taxable×PMT_TAXRATE×PAYMENT-limit ∘ adjustment←PMT_CHARGE+taxamt
[8]  ADJUSTED←PAYMENT-adjustment      A Adjust the payment
      V

```

The code is not well structured, for example the statements calculating the actual adjustment could easily be subsumed into a sub-function, so our code becomes:

```

V ADJUSTED+TYPE PMT_ADJUST PAYMENT;adjustment
[1]  A FN: Adjust the payments if <TYPE> ....
[2]  A TYPE      - Numeric scalar           Type of payment
[3]  A PAYMENT   - Numeric vector           Vector of payments
[4]  A ADJUSTED  - Numeric vector           Adjusted payments
[5]      adjustment+0                      A Assume no adjustment
[6]  A      Adjustment calc'n
[7]      +DOIF TYPE<24 28 > adjustment+PMT_ADJ_CALC PAYMENT
[8]  ADJUSTED+PAYMENT-adjustment          A Adjust the payment
V

```

Now the line involving *DOIF* is so simple, indeed the whole function is so short that perhaps we can remove the *DOIF* altogether:

```

V ADJUSTED+TYPE PMT_ADJUST PAYMENT;adjustment
[1]  A FN: Adjust the payments if <TYPE> ....
[2]  A TYPE      - Numeric scalar           Type of payment
[3]  A PAYMENT   - Numeric vector           Vector of payments
[4]  A ADJUSTED  - Numeric vector           Adjusted payments
[5]      adjustment+0                      A Assume no adjustment
[6]      +none IF ~TYPE<24 28              A Adjust if 24 or 28
[7]      adjustment+PMT_ADJ_CALC PAYMENT  A Adjustment calc'n
[8]  none:ADJUSTED+PAYMENT-adjustment      A Adjust the payment
V

```

I would argue that the last listing is at least as readable as that involving *DOIF*, so we have lost nothing here. Indeed, we have gained many benefits by using an explicit label.

Flexibility of Design

We can, of course, go further and argue that the use of the branch is not a very flexible approach. If the specification changes so that the payment type is now a vector of potentially different types (one type per payment) we are presented with a difficult choice. We could call the function once for every single payment being made but performance may suffer as a result. Alternatively, the code can be modified to cope with the new specification.

Perhaps the following would have been a better solution to start with:

```

      ▽ ADJUSTED←TYPE PMT_ADJUST PAYMENT;adjustment;nonzero
[1]   A FN: Adjust the payments if <TYPE> ....
[2]   A TYPE      - Numeric scalar           Type of payment
[3]   A PAYMENT   - Numeric vector          Vector of payments
[4]   A ADJUSTED  - Numeric vector          Adjusted payments
[5]   nonzero←TYPE<24 28                    A Nonzero adjustments
[6]   A                                     Calculate adjustments
[7]   adjustment←nonzero×PMT_ADJ_CALC PAYMENT
[8]   ADJUSTED←PAYMENT-adjustment          A Adjust the payment
      ▽

```

No branching at all...

A frequently voiced maxim about designing and writing APL code suggests that it is important to make defined functions behave as much like primitives as possible (the "as possible" is crucial). Scalar extension is a fundamental principle within APL, and it is scalar extension that the branching solution makes much more difficult to achieve.

Conclusions

The problems surrounding the *DOIF* function originate from two aspects of its use:

- its reliance on statement separators (diamonds)
- the use of an unlabelled branch

The requirement to use statement separators means:

- *DOIF* is not universal
- very long lines of code can result
- comments will probably be more sparse
- comments will be less "immediate" (farther from the code)
- it will be more difficult to ensure high quality code

The most important advantage claimed for the use of *DOIF* is an improvement in the readability of code. Even this seems a spurious claim because of the increased length of lines and the reduction in commenting.

The use of an unlabelled branch means we cannot change the statement structure of the function by, for example, splitting multi-statement lines so that:

- opportunities for automatic coverage testing are reduced
- detecting performance problem areas may be more difficult
- “diamondization” for code delivery is much more difficult

DOIF alters the perceived complexity of writing a branching form of code rather than using “logical calculations” [McIntyre pp 566-567]. The programmer may as a result:

- create inflexible designs
- give insufficient thought to the modular design of code
- generate additional branches which are not really needed
- use branches when another technique would be more appropriate
- increase the complexity of defined functions

These arguments do not just apply to the *DOIF* function. In general any function which attempts to introduce structured programming constructs, but which relies on unlabelled branching and/or the use of statement separators will suffer from the same problems.

To put it more simply, any utility function which relies on, or encourages, bad programming practices can only create more problems than it solves.

References

- [1] McIntyre D B, *Language as an Intellectual Tool: From Hieroglyphics to APL*, IBM Systems Journal Vol 30 No 4 G321-0105-00
- [2] Myers G J, *Reliable Software Through Composite Design*, Van Norstrand Reinhold 0-442-25620-6
- [3] Page-Jones M, *The Practical Guide to Structured Systems Design*, Prentice Hall 0-13-690777-6
- [4] Taylor A R, Private communications 1991, 1992,
- [5] Yourdon, *Modern Structured Analysis*, Prentice Hall 0-13-598632-X

Appendix 1 - Removing DOIF

Study of some code involving *DOIF*s which was subsequently re-written using a "two-phase" approach indicates the veracity of the above discussion. The code was first modified to use standard branching techniques, together with the diamondized line being "exploded". Thus the sample function above would have become:

```

      V ADJUSTED←TYPE PMT_ADJUST PAYMENT;adjustment;limit;taxamt;taxable
[1]  A FN: Adjust the payments if <TYPE> ....
[2]  A TYPE      - Numeric scalar           Type of payment
[3]  A PAYMENT   - Numeric vector          Vector of payments
[4]  A ADJUSTED  - Numeric vector          Adjusted payments
[5]  adjustment←0                          A Assume no adjustment
[6]  →end IF ~TYPE<24 28
[7]  limit←PMT_TAXLIMIT
[8]  taxable←PAYMENT>limit
[9]  taxamt←taxable×PMT_TAXRATE×PAYMENT-limit
[10] adjustment←PMT_CHARGE+taxamt
[11] end:ADJUSTED←PAYMENT-adjustment      A Adjust the payment
      V

```

On a personal note, this form of code was found to be far easier to understand than the original; I find it particularly hard to remember that the branch occurs if the condition is FALSE. I also find it impossible to ignore the branch arrow when just reading the code.

In the second phase, an analysis was made to see if was possible to remove the branch altogether. In some cases (such as that above) the changes required were trivial. In others it was found a complete restructuring of the code was needed to achieve the elimination of branching logic introduced by *DOIF*.

This was particularly true of functions which contained a significant amount of branching for logic control (good examples would include functions controlling a screen dialogue). In these cases the logic control branching and the *DOIF* branching were so intertwined that a significant amount of effort was required to complete the modifications.

At the end of the effort, the number of branches in the function had been reduced significantly. Some of the branches which at first appeared to be required for logic control (i.e. were not initially associated with a statement containing *DOIF*) were found to exist because of the use of *DOIF* and the branching style of programming it introduced. Thus the total reduction in complexity was much more significant than the mere removal of the *DOIF*s would have suggested.

Appendix 2 - Re-writing DOIF

Some die-hard users of *DOIF* may argue that the difficulties it introduces can be overcome. For example by providing a label as the left argument, the concerns about branching to an unlabelled line and the need for statement separators can both be removed. Our sample function would then look like this:

```

V ADJUSTED+TYPE PMT_ADJUST PAYMENT;adjustment;limit;taxamt;taxable
[1]  FN: Adjust the payments if <TYPE> ....
[2]  TYPE      - Numeric scalar           Type of payment
[3]  PAYMENT   - Numeric vector          Vector of payments
[4]  ADJUSTED  - Numeric vector          Adjusted payments
[5]  adjustment+0                          Assume no adjustment
[7]  +end DOIF TYPE€24 28                  Adjustment calculation
[8]  limit+PMT_TAXLIMIT
[9]  taxable+PAYMENT>limit
[10] taxamt+taxable×PMT_TAXRATE×PAYMENT-limit
[11] adjustment+PMT_CHARGE+taxamt
[12] end:ADJUSTED+PAYMENT-adjustment      Adjust the payment
V

```

And the *DOIF* function would be:

```

V R+A DOIF W
[1]  FN: Return <A> if <~W>.
[5]  R+(-W)/A
V

```

But hang on, what does *IF* look like?

```

V R+A IF W
[1]  FN: Return <A> if <W>
[5]  R+W/A
V

```

It is immediately apparent that this form of *DOIF* is identical to a "branch on not condition" form of *IF* (i.e. *IFNOT*). For myself, the use of *IF* and *~* is much clearer than *DOIF* or *IFNOT*.

Index to Advertisers

Cocking and Drury	52
Cybex	2
Dyadic Systems Ltd	6
Iverson Software Inc	84
Lingo Allegro U.S.A. Inc	22
MicroAPL	21
Zark	8
APL Booklist (Renaissance Data Systems)	106

All queries regarding advertising in VECTOR should be made to Gill Smith, at Brook House, Gilling East, YORK. Tel: 04393-385.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Jonathan Barman,
Hill Top House,
East Garston,
NEWBURY, Berks RG16 7HD
Tel: 048839-575

(not after 10.00pm please!)

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members news) should be sent to Vector Production, c/o Adrian Smith, Brook House, Gilling East, YORK Tel: 04393-385 (6.00pm - midnight).

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box): 92/93

UK private membership	£12	☐
Overseas private membership	£20	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£100	☐
Corporate membership overseas	£155	☐
Sustaining membership	£430	☐
Non-voting student membership (UK only)	£6	☐

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____|____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

Signature: _____

The British APL Association will take the completion of this form as evidence that you do not object to your membership details being held on our computer. For details of the British APL Association's exemption from the Data Protection Act please see Vector, the Journal of the BAA, Vol.8 No.4

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Access or Visa number. Send the completed form to:

British APL Association, FREEPOST (SG923), 8 Cardigan Road, LONDON, E3 5HU

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1992/93 Committee

Chairman:	David Eastwood 071-922 8866	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Secretary:	Anthony Camacho 0272-730036	11 Auburn Road, Redland BRISTOL, BS6 6LS.
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Jonathan Barman, 048839-575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Activities:	Peter Donnelly 0256-811125	Dyadic Systems Ltd., Riverside View, Basing Road, Old Basing, BASINGSTOKE, Hants RG24 0AL
Education:	Dr Alan Mayer 0792-295296	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Technical:	Duncan Pearson 0904-603510	c/o Operational Research Nestlé-Rowntree, YORK YO1 1XY
Projects:	George MacLeod 0732-885126	Rusper, Sandy Lane, Ivy Hatch, Seven Oaks, Kent TN15 0PD
Publicity:	Misha Jovanovic 0753-853141	99 Oxford Road, WINDSOR, Berks SL4 5DX
Recruitment:	Dr Peter Branson 081-848 8989	Electronic Data Systems, Stockley Park, UXBRIDGE, Middx UB11 1BQ
Administration:	Rowena Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Jonathan Barman	048839-575
Secretary:	Anthony Camacho	0272-730036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	Ray Cannon (0252-874697), Richard Weber, Sylvia Camacho, Duncan Pearson	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' == an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Malton Road
Gillies Street, Bath, BA1 5NS
Tel 0225 462602
Fax 0225 444552

Compass FMD Ltd
15/16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
Surrey Research Park
Guildford, Surrey GU2 5YD
Tel 01833 602219
Fax 01833 602279

Dyadic Systems Ltd
Riverside View, Basingstoke
Odebasin, BASINGSTOKE
Hants, RG22 0AL
Tel 02534 111125
Fax 02534 111180

HMV Trading Systems Ltd
Hamilton House
1 Temple Avenue
LONDON EC4A 3QA
Tel 071 4834242
Fax 071 4834825

Cockington & Dury Ltd
180 Tottenham Court Road
LONDON W1P 9HE
Tel 071 4868213
Fax 071 4860524

MicroAPL Ltd
South Bank, Tottenham
90 Tottenham Road
LONDON SE1 1EN
Tel 071 4922886

Barbers Ltd
85 Fleet Street
LONDON EC4A 3AA
Tel 071 250 1122
Fax 071 5838248

Enter Cyrix Systems
22 Farnham Road
LONDON W2 4YA
Tel 071 2215624

Imperial Ltd
Rusper, Sandhurst
Wyke, WIMBORNE
Kent TN15 0PD
Tel 0742 685126

Dutch APL User Group
Postbus 1821, 6500 EA Nieuwegein
Netherlands
Tel 0847 42087